Rossmann Stores
Sales and Customer Prediction

Alireza Hosseinzadeh

1 Introduction

Rossmann, Germany's second-largest drugstore chain, operates over 3000 stores across multiple countries. Accurate sales forecasts and customer number predictions are crucial for effective staff scheduling, inventory management, and promotional strategies. This report outlines the methodology and results of the sales and customer prediction model, aiming to enhance Rossmann's operational efficiency and strategic decision-making.

2 Data Description

The analysis used two primary datasets provided by Rossmann: sales records and store specifications. The sales dataset includes approximately one million records from 1115 stores across Germany from January 1, 2013, to July 31, 2015. Key variables that are identified by the business and expect to influence sales and customer numbers are brought in appendix A.

Additional data sets were used to improve prediction accuracy. One data set mapped stores to 13 of the 16 states in Germany, which is important due to state-specific holidays and school regulations. Another data set provided aggregated weather information for each state on the sales data dates, including max temperature and min humidity.

3 Data Preprocessing

3.1 Missing and Incorrect Values Treatment

In the first step the provided data sets were analysed for missing values, outliers, or incorrect values compared to the provided data dictionary.

There are 180 stores missing 184 days of data in the middle of the series between 1 July 2014 to 31 Dec 2014. Compared to the 1 million given data records they can be neglected, since we are relying on the characteristics of stores, not the stores' IDs. After inspection, we find out that the specific store had been under refurbishment in that period. These missing records will have a little effect on the prediction of the test data points which are 8 months away from the last day of the missing data.

Starting with the test data set, 11 rows have missing Open values, which are assumed to represent closed stores and are thus imputed with 0. Closed stores have zero Sales and Customers, so to simplify the models, these records are removed from both test and train data sets, treating Sales and Customer values as constants. To make model accuracy comparable, the accuracy is adjusted by weighting 17% for predictions with 100% accuracy and 83% for the model's accuracy on the training data, since actual values for the test data are unavailable.

The train data set doesn't have any missing values, nevertheless, 54 data rows have 0 Sales (52 of these have 0 Customers), while recorded as being open. Assuming that these observations were mistakenly recorded as open, we drop them as closed stores. Otherwise, they will cause a peak in the corresponding distribution, as shown in Figure 3-1.

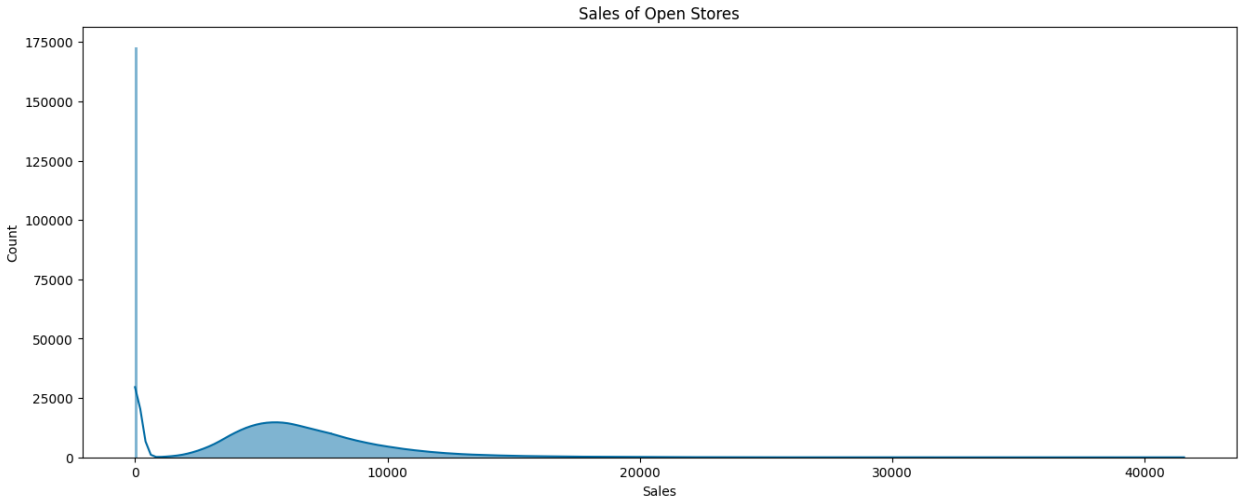


Figure 3-1 Sales with zero values, causing a peak in the distribution.

For the store data set, Figure 3-2 shows that except for CompetitionDistance with 3 missing values there are a lot of missing values in CompetitionOpenSinceMonth and Year and Promo2SinceWeek and Year, as well as PromoInterval (It would have been better if it were named Promo2Interval, as it corresponds to Promo2, not Promo which is day specific.).

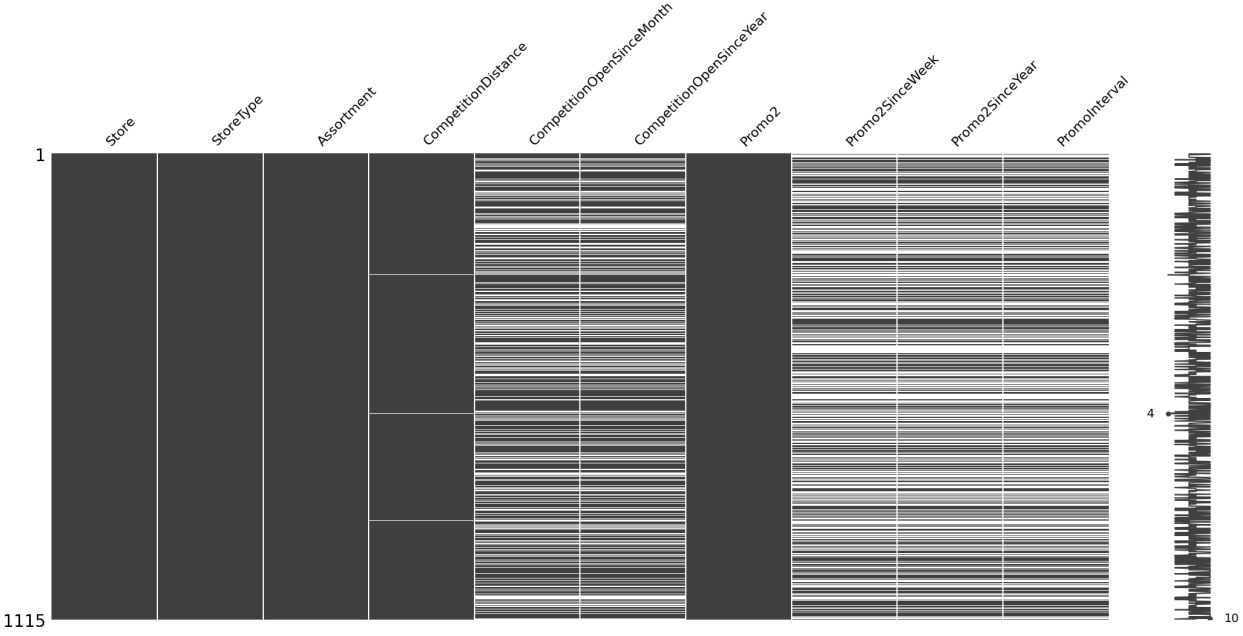


Figure 3-2Missing number matrix of the store data set

Given the missing number correlation shown in Figure 3.3, the missing data for Competition can be considered completely random. We imputed the missing distances with the median distance to

avoid extreme values and replaced the missing competition dates with the minimum values in the data set, assuming these correspond to initial stores where these fields were not recorded.

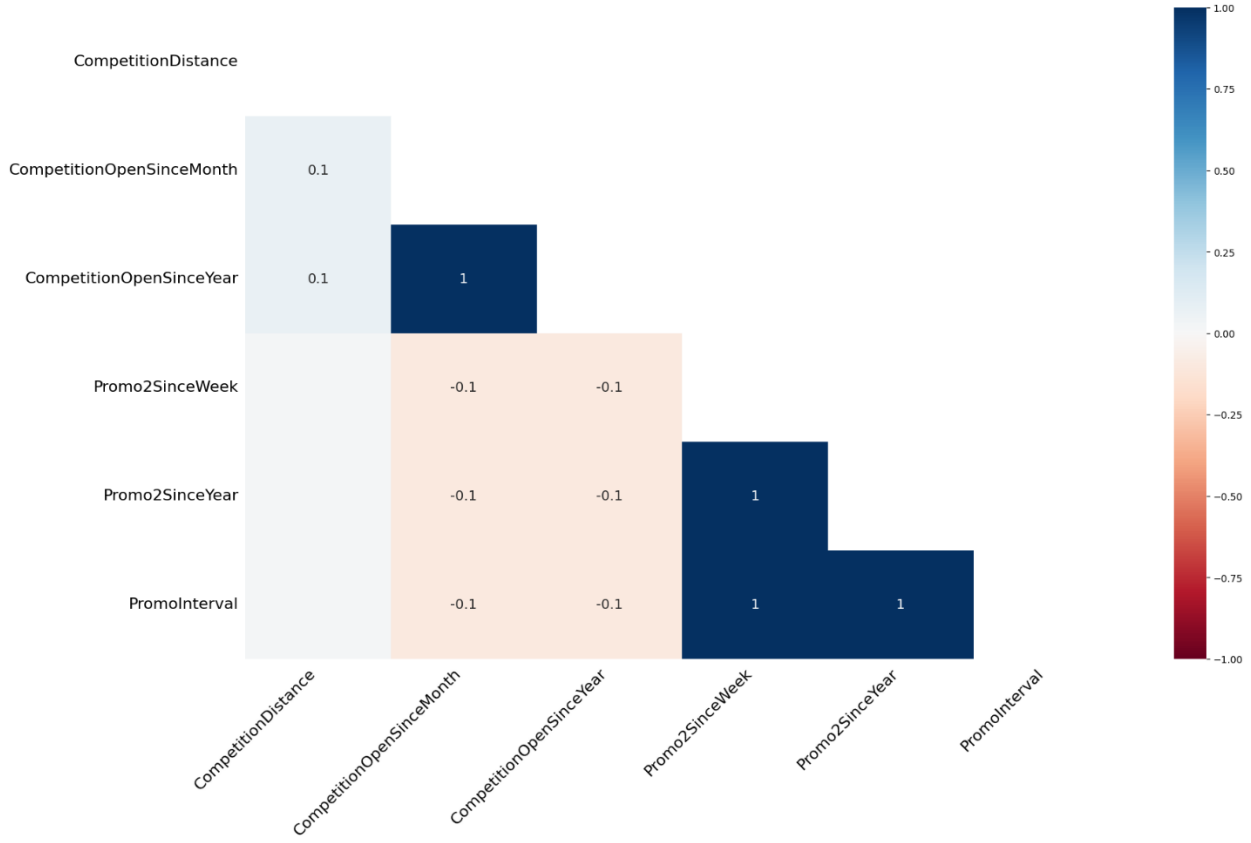


Figure 3-3 Missing number correlation heatmap.

However, for Promo2 related missing values, after taking some queries, we can realise that these are in fact systematic missing values and correspond to the cases for which Promo2 was not active (with the value of 0). Therefore, we replace the Promo2SinceWeek and Promo2SinceYear with -1, and Promo2Interval with 'None'.

3.2 Data Transformation and Feature Generation

After ensuring the compatibility of the data with the data dictionary, considering the form of the distribution in Figure 3-1, we check the skewness of Sales. As shown in Figure 3-4, the distribution is skewed to right with the skewness of 1.59.

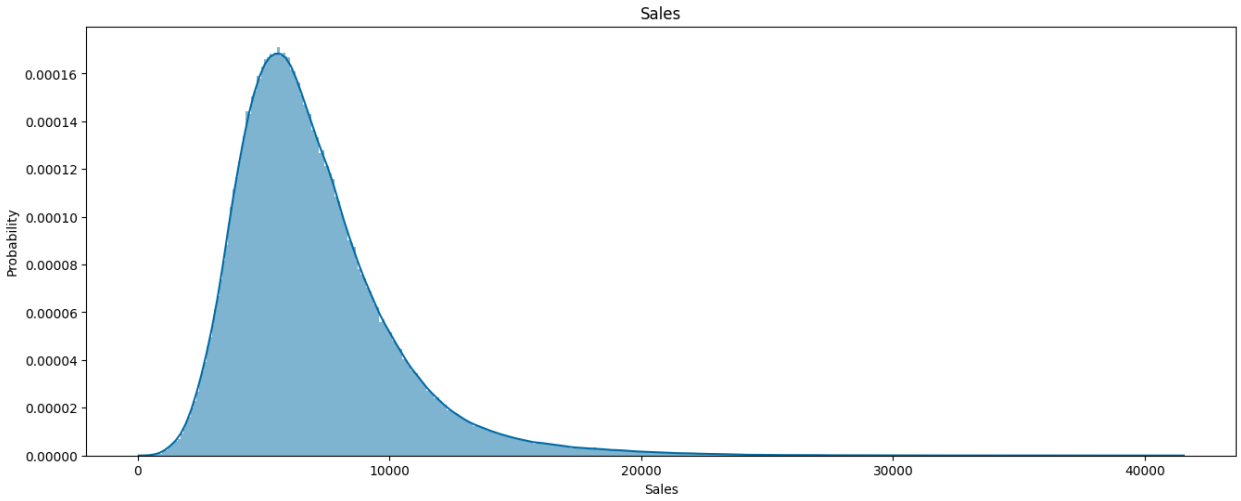


Figure 3-4 Distribution of Sales, skewed to right.

Taking the logarithm of Sales, will reduce the skewness to -0.11, which can be fairly considered as being symmetrical. Nevertheless, it's highly platykurtic with the kurtosis of 0.65, far from being 3, corresponding to the normal distribution.

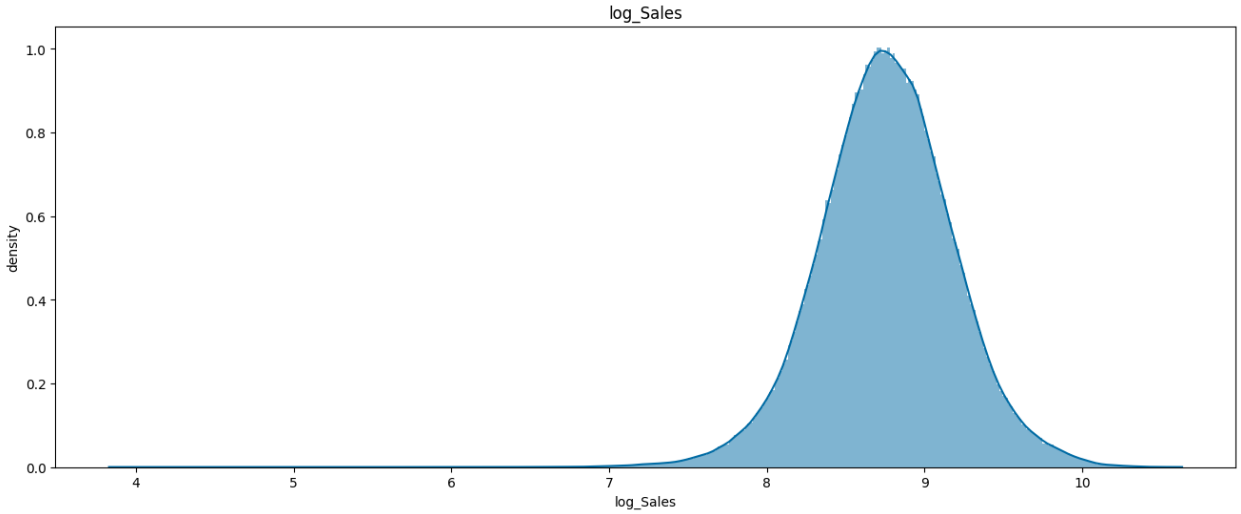


Figure 3-5 Log sale, being approximately symmetrical.

Winsorizing and removing outliers did not resolve the issue. Box-Cox transformations with lambdas between -3 and 3 were also ineffective. These steps were repeated for Customers with the same results. Given the high correlation between Sales and Customers, a new variable, sales per customer, was generated. This variable, Figure 3-6, despite a left-side bump, has a skewness of

0.59 and kurtosis of 2.76, approximating a normal distribution, which is crucial for predictive models. However, further Winsorizing, outlier removal, and other transformations did not improve its distribution.

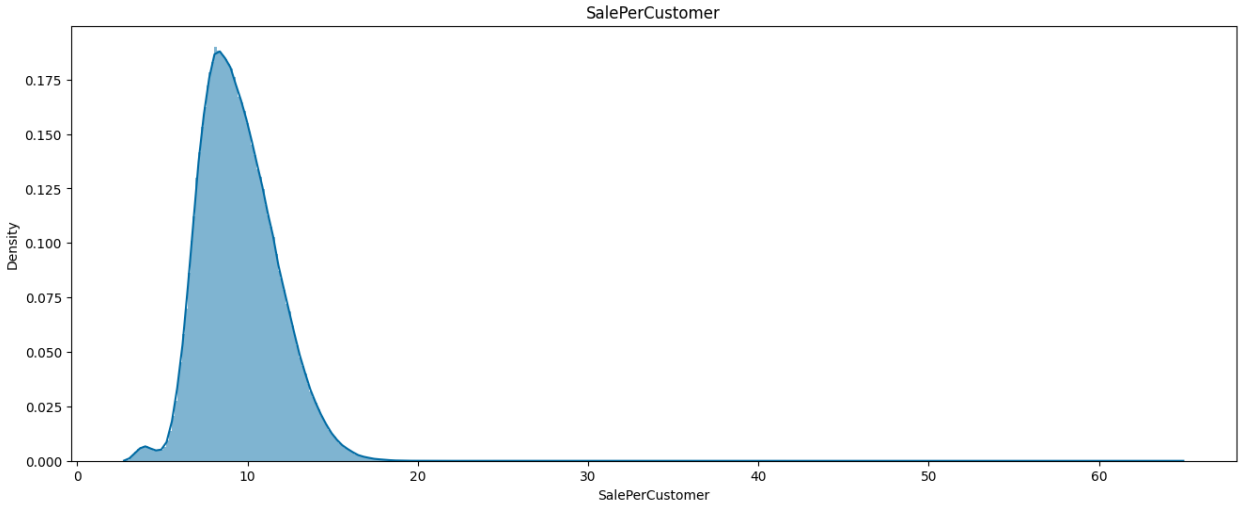


Figure 3-6 Sales per customer having a fairly normal distribution.

The other variable that needs some transformation is Date. This variable is better to be set as the index, which we do it on the final merged data set. Nevertheless, for the prediction models that don't exploit the autocorrelation in the data, we extract year and month. Besides year and month, day is also extracted to check the existence of patterns in the exploratory data analysis (EDA) section.

Among the ratio-scaled variables, as can be seen in the violin plot of Figure 3-7, competition distance has very thick tail on the right. Winsorizing or outlier removal will cause dramatic losses in the data set. By applying several transformations, we understand that the log-inverse of squared root of this variable will be bounded with a very low skewness. While not as critical as some other characteristics, considering symmetry and outliers can significantly improve the performance and reliability of a regression model and ensure more accurate and interpretable results.

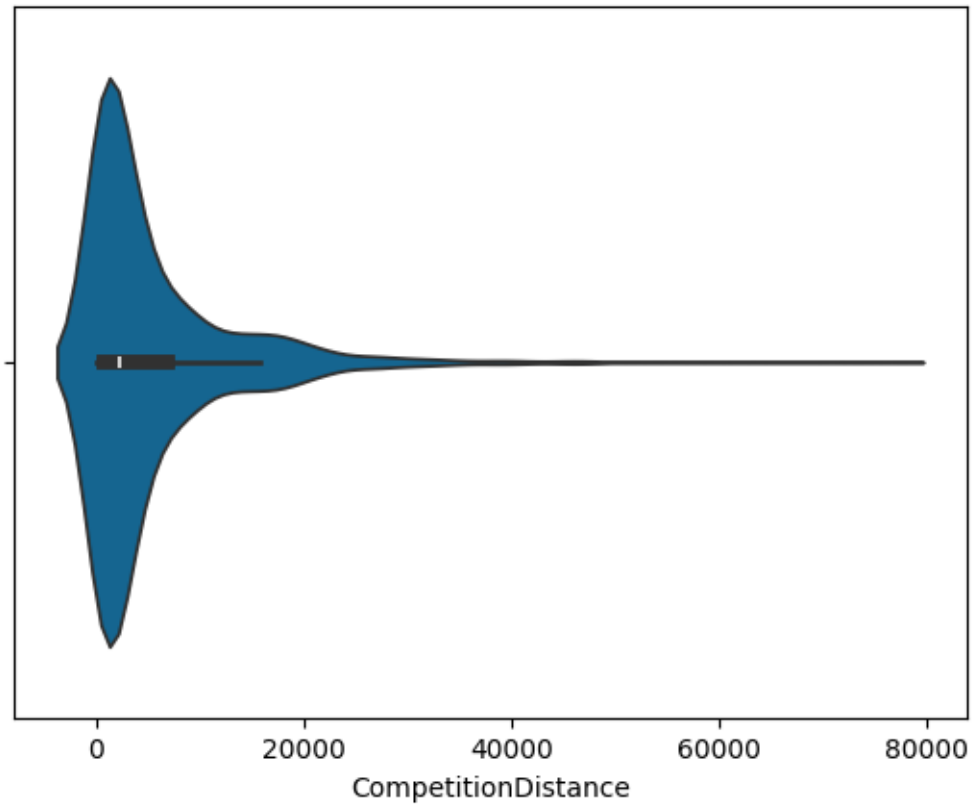


Figure 3-7Competition distance violin plot.

It is meaningless to consider the week, month, or year of when competition or promotion (2) has started for a store as regressor variables. Instead, considering the short (6 week) duration that a prediction model is intended to be applied to, we consider the duration that competition and promotion 2 have been active, and eliminate the other variables. As a result, Promo2 can also be deleted, since a store that is not engaged in it will have a Promo2Duration of 0.

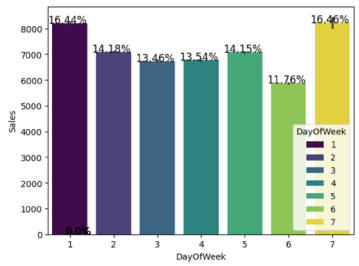
After ordinal encoding the Assortment variable, we merged the datasets as mentioned in section 2. Other categorical variables were not label encoded since their original names are more expressive, and mathematical operations like correlation analysis on these nominal variables are meaningless. Data types were optimized to minimize memory usage. One-hot encoding was applied to a copy of the final data frame, named "new total data frame," preserving the original for models that can effectively use categorical encoding.

A new attribute combining State and StateHoliday was created to capture their interacting effects. Despite State also influencing school holidays, this interaction was not included due to its minor impact on sales, as determined by EDA, to prevent an excessive number of variables. However, the potential applicability of Simpson's paradox should be inspected.

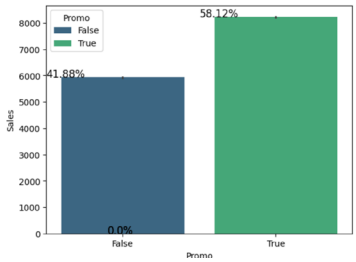
4 Exploratory Data Analysis

Bar charts of categorical variables against sale, shown in Figure 4-1 (a)-(i), provide the following insights:

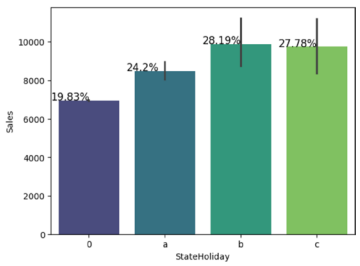
1) 1. Sales were higher on Mondays, likely because shops are closed on Sundays.
2) 2. Promotions lead to increased sales.
3) 3. Most stores close on state holidays, and all schools close on public holidays and weekends.
4) 4. The lowest sales occur on state holidays, especially Christmas.
5) 5. More stores were open on school holidays than state holidays, leading to higher sales.
6) 6. Type B stores had the highest average sales.
7) 7. Assortment level B stores had the highest average sales.
8) 8. Promo2 resulted in slightly higher sales due to increased store participation.
9) 9. Berlin had the highest sales among the states.
10) 10. Saturdays have the lowest sales, making them ideal for refurbishment.



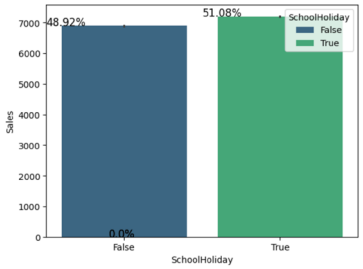
(a)

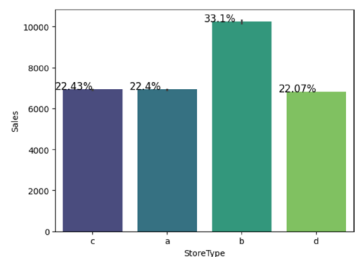


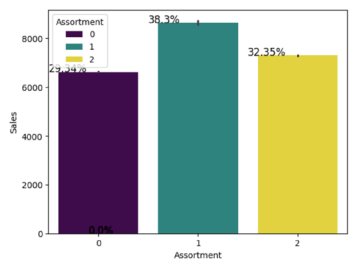
(b)



(c)







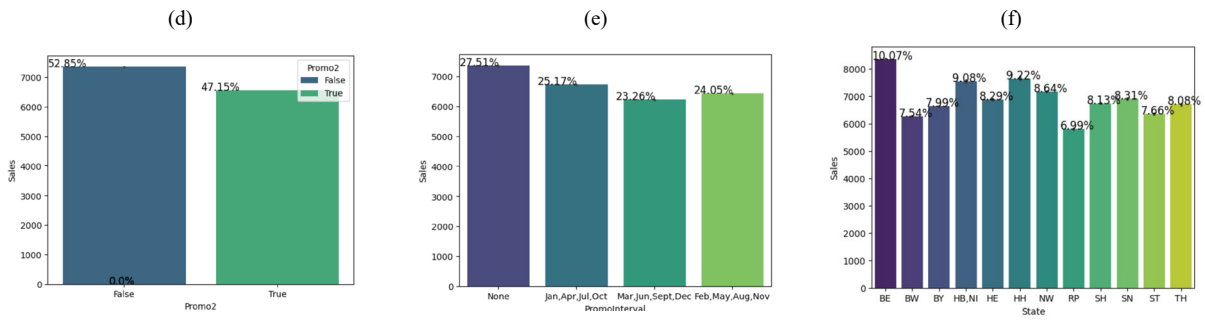


Figure 4-1Bar charts of categorical variables, (a) Day of Week, (b) Promotion, (c) State Holiday, (d) School Holiday, (e) Store Type, (f) Assortment, (g) Promotion 2 , (h) Promotion 2 Interval, (i) State, against sales.

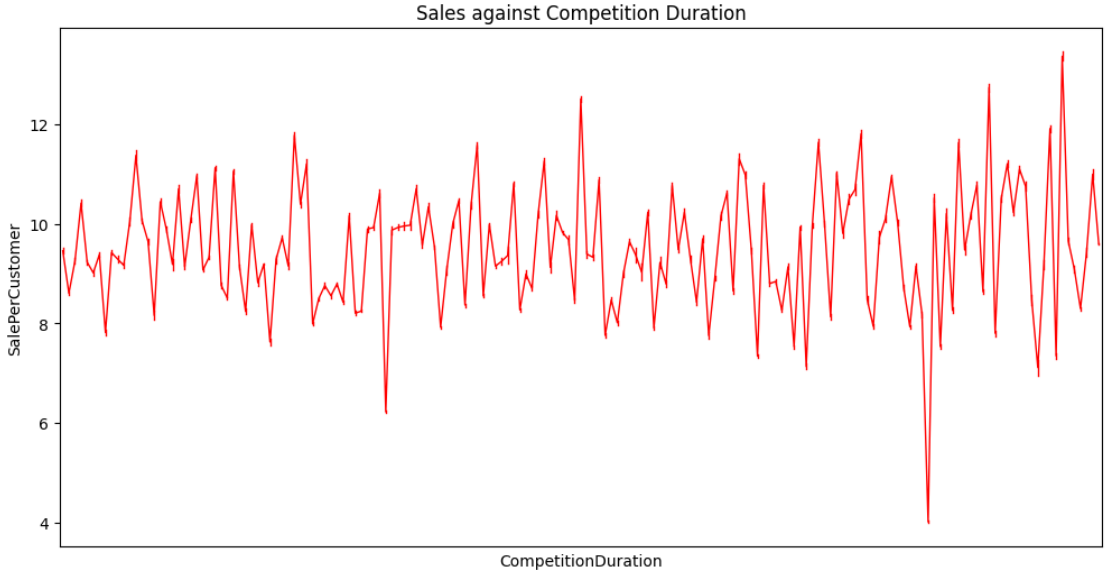
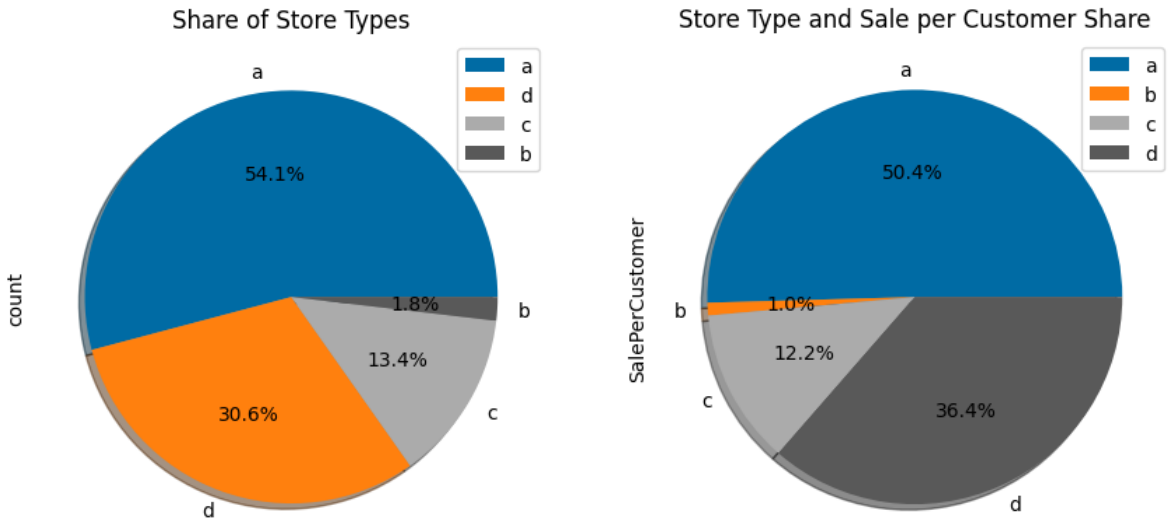


Figure 4-2Effect of competition duration on sales.

As shown in Figure 4-2, competition duration merely doesn't seem to have a significant role in determining sales per customer.



(a) (b)

Figure 4-3 The share of the type of store. in (a) sales per customer, (b) in the number of owned stores.

From Figure 4-3 (a), we see that the majority of the stores are physical stores. Upon further exploration, from Figure 4-3 (b), it can be observed that the highest sales belonged to the store type a. This can be due to the high number of type a stores in the data set. Stores type a and c had a similar kind of sales and customer share. These insights can be used to help the business in making the decision on where to open new stores considering the existing store types, and what products or services to offer.

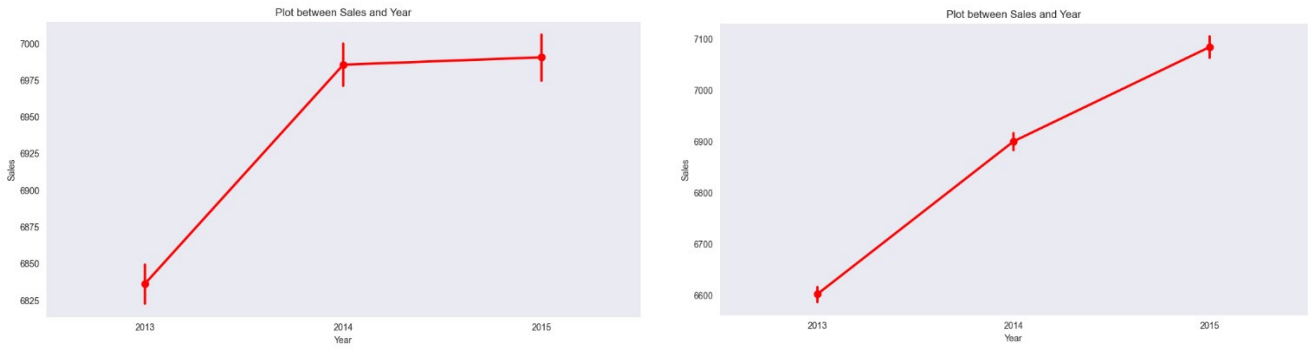
Although two and a half years are not enough for identifying trends, we observe an increasing trend in years, seasonality (oscillation) in months, and noise in days. These patterns, should be taken into consideration along with the event-driven causes for properly modelling sales.

(a) (b) (c)

Figure 4-4 (a) Increasing trend in years, (b) seasonality (oscillation) in months, (c) noise in days, patterns in sales data.

Although the overall trend of sales over years as shown in Figure 4-4 (a), is increasing, in Figure 4-5 we observe that for physical stores it becomes steady and the increase is due to online stores.

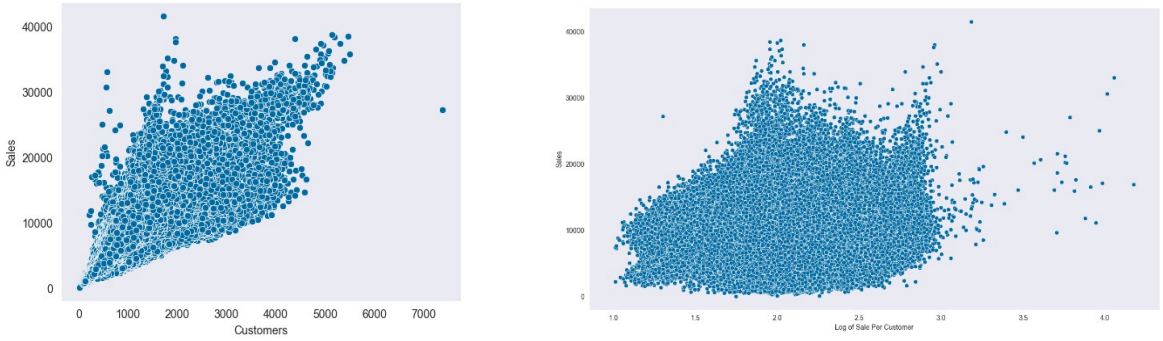
Considering the share of stores in Figure 4-3 (a), this could be dangerous for the business facing it's competitors and it should invest more on online shops.



(a) (b)

Figure 4-5 Sales trend for (a) physical store, (b) online stores.

As can be seen by the scatter plots in Figure 4-6, sales and customers are highly positively correlated, whereas (log) sales per customer becomes (linearly) independent from sales. These results are confirmed by the upcoming correlation heatmap.



(a) (b)

Figure 4-6 Scatter plot of (a) Sales and Customers, (b) Sales and Sales per Customer.

Visual analysis of the relation between sales and the other interval-scaled and ratio-scaled variables, such as maximum temperature, and minimum humidity are useless and we have to resort to numeric analysis of them.

The correlation heatmap of the numeric variables, shown in Figure 4-7, indicates that the following factors have the highest effect on SalePerCustomer:

1) Promo2Duration, which subsumes the availability of Promo2;
2) log_InvSqrtCompDist;

3) Assortment;
4) Promo;

Nevertheless, correlation analysis can't recognise the nonlinear relationships.

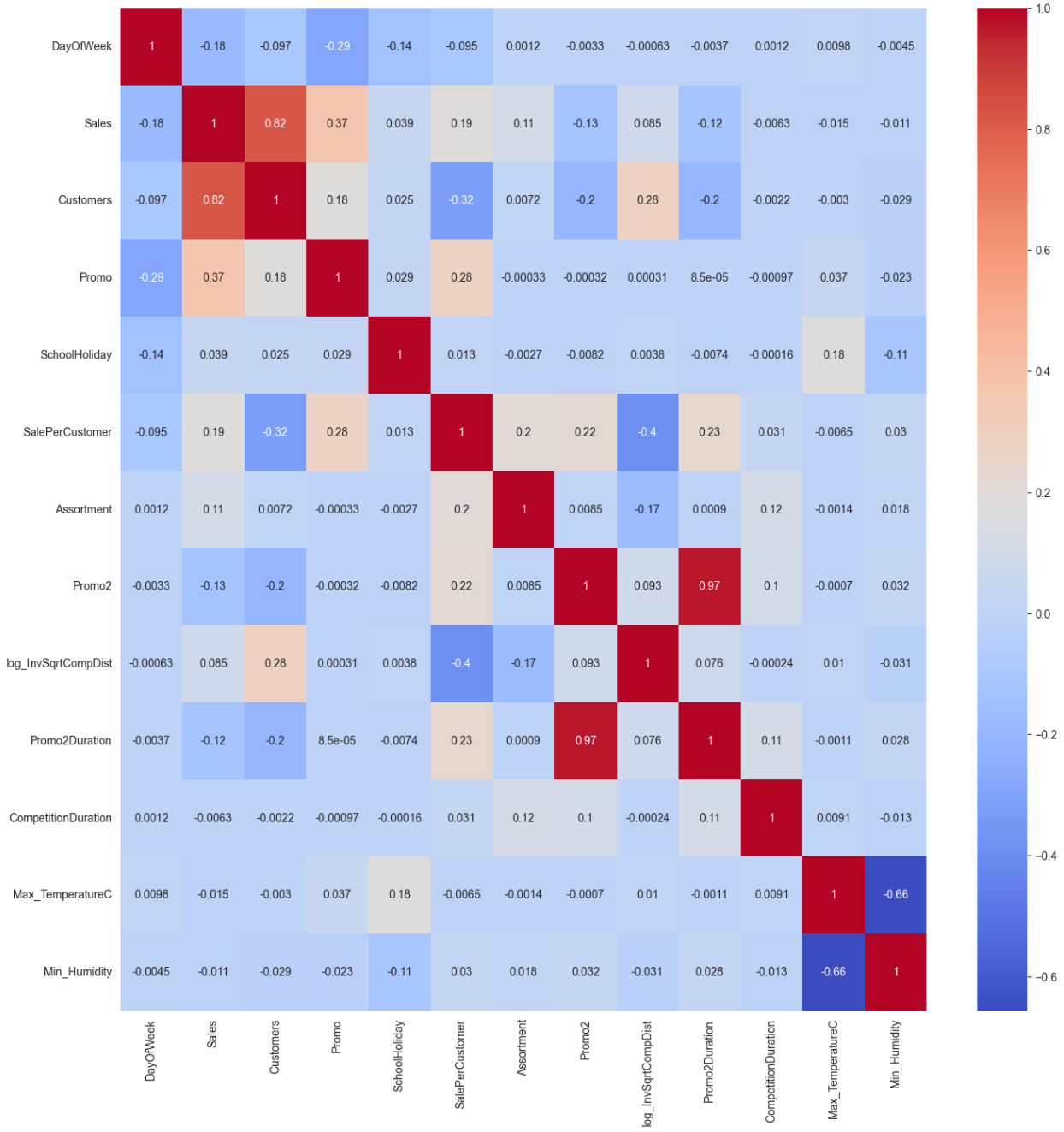


Figure 4-7Correlation heatmap between numeric variables.

From the performed EDA we understand that the key factors are both pattern-based and event-driven.

5 Prediction

The following models were developed and tested on the data:

1) Linear Regression
2) LASSO
3) Ridge
4) Elastic Net
5) Regression Tree
6) Random Forest
7) Ada Boost
8) LSTM

For LSTM since we want to evaluate the generalization capabilities of the models on time-series data, we used hold-out cross-validation and exclude the ending 20 percent of the data for testing. For the other models, since they don't exploit autocorrelation, 20% were held out randomly. These test data points are different from the test data set, as we don't have the true values of the test data set. The hyperparameters of some of the models were optimized by evaluating them for different values.

To avoid multicollinearity, variance inflation factors of all the variables were calculated and based on the EDA a set of variables was selected as independent (input) variables for the models other than LSTM. Root Mean Square Error (RMSE) was used as the main measure of performance, instead of the Root Mean Square Percentage Error (RMSPE) to penalize the errors equally. XG Boost and Elastic Net, outperformed the other models.

Nevertheless, LSTM, which is a type of recurrent neural network (RNN) architecture that is particularly well-suited to sequence prediction problems, such as temporal sequences and their long-range dependencies more accurately than conventional RNNs, can't be compared to the above methods since its training and testing data points were different and customers and sales were used as output variables. Since LSTM generates features automatically the final model is not comprehendible and the previous models are preferrable in terms of interpretability.

6 Conclusion

The report developed a robust predictive model for Rossmann stores, identifying key factors influencing sales and customer numbers. Advanced models like XGBoost and LSTM significantly improved prediction accuracy, offering insights for strategic decision-making:

- Promotions and holidays significantly impact sales.

- Competition distance influences store performance based on proximity to competitors.

- Urban stores with higher competition density show different sales patterns compared to rural stores.

Continuous model updates with new data will maintain predictive performance. Further research could explore external factors like economic indicators and digital marketing impacts. Adding geospatial data could also reveal the effect of locality on sales.

7 References

- Kaggle Rossmann Store Sales competition

https://www.kaggle.com/c/rossmann-store-sales

Bring the other data sets

- Weather data at the 16 different states:

https://www.kaggle.com/competitions/rossmann-store-sales/discussion/17058#97075

https://storage.googleapis.com/kaggle-forum-message-attachments/97075/3138/Brandenburg.csv

https://storage.googleapis.com/kaggle-forum-message-attachments/97075/3139/Sachsen.csv

https://storage.googleapis.com/kaggle-forum-message-attachments/97075/3140/Berlin.csv

https://storage.googleapis.com/kaggle-forum-message-attachments/97075/3141/NordrheinWestfalen.csv

https://storage.googleapis.com/kaggle-forum-message-attachments/97075/3142/SchleswigHolstein.csv

https://storage.googleapis.com/kaggle-forum-message-attachments/97075/3143/Niedersachsen.csv

https://storage.googleapis.com/kaggle-forum-message-attachments/97075/3144/Th%C3%BCringen.csv

https://storage.googleapis.com/kaggle-forum-message-attachments/97075/3145/MecklenburgVorpommern.csv

https://storage.googleapis.com/kaggle-forum-message-attachments/97075/3146/SachsenAnhalt.csv

https://storage.googleapis.com/kaggle-forum-message-attachments/97075/3147/RheinlandPfalz.csv

https://storage.googleapis.com/kaggle-forum-message-attachments/97075/3149/Saarland.csv

https://storage.googleapis.com/kaggle-forum-message-attachments/97075/3149/Saarland.csv

https://storage.googleapis.com/kaggle-forum-message-attachments/97075/3150/Bremen.csv

https://storage.googleapis.com/kaggle-forum-message-attachments/97075/3151/BadenW%C3%BCrttemberg.csv

https://storage.googleapis.com/kaggle-forum-message-attachments/97075/3152/Bayern.csv

https://storage.googleapis.com/kaggle-forum-message-attachments/97075/3153/Hessen.csv

- Data on mapping of stores to states:

https://www.kaggle.com/c/rossmann-store-sales/forums/t/17048/putting-stores-on-the-map

8 Appendices

8.1 Appendix A: Data Dictionaries

- Sales (Train and Test) Data:
  - Store: Unique identifier for each store.
  - Date: Date of the sales record.
  - Sales: Sales turnover for a particular day in Euros. (for test it's not given)
  - Customers: Number of customers on a given day. (for test it's not given)
  - Open: Indicator of whether the store was open (0 = closed, 1 = open).
  - Promo: Indicates whether a store is running a promotion on a particular day.
  - StateHoliday: Indicates whether the day is a state holiday.
  - SchoolHoliday: Indicates whether the day is a school holiday.
- Store Data:
  - Store: Unique identifier for each store.
  - StoreType: Differentiates between 4 different store models: a, b, c, d.
  - Assortment: Describes the assortment level: a = basic, b = extra, c = extended.
  - CompetitionDistance: Distance to the nearest competitor store.
  - CompetitionOpenSinceMonth: Month the nearest competitor store opened.
  - CompetitionOpenSinceYear: Year the nearest competitor store opened.
  - Promo2: Continuous and consecutive promotion for some stores (0 = store is not participating, 1 = store is participating).
  - Promo2SinceWeek: Calendar week when the store started participating in Promo2.
  - Promo2SinceYear: Year when the store started participating in Promo2.
  - PromoInterval: Consecutive intervals Promo2 is active.

8.2 Appendix B: Python Code

Meta Data

Problem Description

Rossmann has provided a comprehensive dataset containing approximately one million sales records from 1,115 stores across Germany, spanning from January 1, 2013, to July 31, 2015.

The task is to predict daily sales for these stores over a 6-week period, from August 1, 2015, to September 17, 2015.

Train and Test Data Set

Sale specific data regarding each store for the given date, constituting of:

• Store: a unique store ID for each store ;

• DayOfWeek: The day of week , staring from Monday as 1;

• Date: the date in a first day format;

• Sales: the turnover for any given day in Euros;

• Customers: the number of customers on a given day;

• Open: an indicator for whether the store was open: 0 = closed, 1 = open;

• Promo: indicates whether a store is running a promo on that day;

• StateHoliday: indicates a state holiday. Normally all stores, with few exceptions, are closed on state holidays. Note that all schools are closed on public holidays and weekends. a = public holiday, b = Easter holiday, c = Christmas, 0 = None;

• SchoolHoliday: indicates if the (Store, Date) was affected by the closure of public schools;

Store Data Set

The data corresponding to the stores:

• Store: The unique store ID for each store, as the primary key;

• StoreType: differentiates between 4 different store models: a, b, c, d;

• Assortment: describes an assortment level: a = basic, b = extra, c = extended;

• CompetitionDistance: distance in meters to the nearest competitor store;

• CompetitionOpenSinceMonth: gives the month of the time the nearest competitor was opened;

• CompetitionOpenSinceYear: gives the of the time the nearest competitor was opened;

• Promo2: Promo2 is a continuing and consecutive promotion for some stores: 0 = store is not participating, 1 = store is participating;

• Promo2SinceWeek: describes the year and calendar week when the store started participating in Promo2;

• Promo2SinceYear: describes the year and calendar week when the store started participating in Promo2;

• PromoInterval: describes the consecutive intervals Promo2 is started, naming the months the promotion is started anew. E.g. "Feb,May,Aug,Nov" means each round starts in February, May, August, November of any given year for that store;

0. Install packages

To install the necessary packages, excute either of 0.1, 0.2, or 0.3.

0.1. Install the packages (versions and dependencies might not conform!)

```
In [1]: # pip install numpy
In [2]: # pip install pandas
In [3]: # pip install matplotlib
In [4]: # pip install seaborn
In [5]: # pip install plotly
In [6]: # pip install scipy
In [7]: # pip install statsmodels
In [8]: # pip install scikit-learn
In [9]: # pip install xgboost
In [10]: # pip install missingno
In [11]: # pip install lightgbm
In [12]: # pip install shap
```

In [13]:
```
# pip install lime
```

In [14]:
```
# pip install requests
```

In [15]:
```
# pip install --upgrade ipywidgets
```

In [16]:
```
# pip install --upgrade jupyter
```

In [17]:
```
# pip freeze > requirements.txt
```

0.2. Install requirements

In [18]:
```
# pip install -r requirements.txt
```

0.3. Create and install the UDatEMan virtual environment

Follow the given instructions:

- Run Anaconda Prompt as Administrator.
- Navigate to the pertinent directory. (cd C:...)
-

1. Import Necessary Packages

In [364…
```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt
import matplotlib.style as style

import seaborn as sns

from statsmodels.stats.outliers_influence import variance_inflation_factor

from sklearn.preprocessing import StandardScaler

from sklearn.model_selection import train_test_split, GridSearchCV, cross_val_sc

from sklearn.linear_model import LinearRegression, Ridge, Lasso, ElasticNet

from sklearn.tree import DecisionTreeRegressor

from sklearn.metrics import r2_score, mean_squared_error as mse

from sklearn.ensemble import RandomForestRegressor, AdaBoostRegressor

import xgboost as xgb
```

```
import shap

from lime.lime_tabular import LimeTabularExplainer

from scipy.stats.mstats import winsorize

import missingno as msno

import requests

import os

from tensorflow.keras.models import Model
from tensorflow.keras.layers import LSTM, Dense, Input
from tensorflow.keras.optimizers import Adam
```

2. Initialise Settings

In [20]:
```
# To display graphs entirely in the workbook
%matplotlib inline
```

In [21]:
```
# Setting the style
style.use('tableau-colorblind10')
```

In [22]:
```
# TODO Turn on at last
# import warnings
# warnings.filterwarnings("ignore")
```

3. Load Data Sets

3.1. Test

In [23]:
```
# Load Rossmann's test data set
test_df = pd.read_csv(r"Data\test.csv", low_memory=False, )
# low_memory=False to avoid the potential mixed type inference
# View test_df data frame
test_df.head()
```

Out[23]:

	Store	DayOfWeek	Date	Sales	Customers	Open	Promo	StateHoliday	Schc
0	1	4	17/09/2015	NaN	NaN	1.0	1	0	
1	3	4	17/09/2015	NaN	NaN	1.0	1	0	
2	7	4	17/09/2015	NaN	NaN	1.0	1	0	
3	8	4	17/09/2015	NaN	NaN	1.0	1	0	
4	9	4	17/09/2015	NaN	NaN	1.0	1	0	

3.2. Train

In [24]:
```
# Load Rossmann's train data set
train_df = pd.read_csv(r"Data\train.csv", low_memory=False, )
# View train_df data frame
train_df.head()
```

Out[24]:

	Store	DayOfWeek	Date	Sales	Customers	Open	Promo	StateHoliday	Sch
0	1	5	31/07/2015	5263	555	1	1	0	
1	2	5	31/07/2015	6064	625	1	1	0	
2	3	5	31/07/2015	8314	821	1	1	0	
3	4	5	31/07/2015	13995	1498	1	1	0	
4	5	5	31/07/2015	4822	559	1	1	0	

3.3. Store

In [25]:
```
# Load Rossmann's store data set
store_df = pd.read_csv(r"Data\store.csv", low_memory=False, )
# View store_df data frame
store_df.head()
```

Out[25]:

	Store	StoreType	Assortment	CompetitionDistance	CompetitionOpenSinceMonth	C
0	1	c	a	1270.0	9.0	
1	2	a	a	570.0	11.0	
2	3	a	a	14130.0	12.0	
3	4	c	c	620.0	9.0	
4	5	a	a	29910.0	4.0	

3.4. ALTERNATIVELY

In [26]:
```
# train_df = pd.read_csv(r"Data\train.csv", low_memory=False, parse_dates=True,
# test_df = pd.read_csv(r"Data\test.csv", low_memory=False, parse_dates=True, da
# store_df = pd.read_csv(r"Data\store.csv", low_memory=False, parse_dates=True,

## parse_dates=['Date']: consider 'Date' as date format.
## parse_dates=True: pandas will try to infer the date format for each column.
## we could have first imported the data and after that convert it to date forma
## train.index = pd.to_datetime(train.index, dayfirst=True)
## low_memory=False: not be conservative with allocating memory, not import in c
## index_col='Date': specifying which column should be used as the index of the
```

3.5. Additional Data Sets

The states are important Since public holidays as well as school holidays in Germany differ from state to state.

We can also involve the weather data (max temperature and min precipitation) based on state and date, which have impact on physical stores.

The other external data sets that could have been included in the analysis include:

- External data on World Cup dates (Special events)
- Macro indicators data
- Google trends data for keyword "ROSSMANN" for every state as of today

3.5.1. store-state

In [27]:
```
# we can use store_states.csv to map the stores to the states
store_state_df = pd.read_csv('./Data/store_states.csv')
```

In [28]:
```
store_state_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1115 entries, 0 to 1114
Data columns (total 2 columns):
 #   Column  Non-Null Count  Dtype 
---  ------  --------------  ----- 
 0   Store   1115 non-null   int64 
 1   State   1115 non-null   object
dtypes: int64(1), object(1)
memory usage: 17.6+ KB
```

In [29]:
```
store_state_df['State'].unique()
```

Out[29]:
```
array(['HE', 'TH', 'NW', 'BE', 'SN', 'SH', 'HB,NI', 'BY', 'BW', 'RP',
       'ST', 'HH'], dtype=object)
```

In [30]:
```
store_state_df.duplicated().any()
```

Out[30]:
```
False
```

3.5.2. Date-state-Weather

In [31]:
```
data_urls = {
    "Brandenburg": "https://storage.googleapis.com/kaggle-forum-message-attachme
    "Sachsen": "https://storage.googleapis.com/kaggle-forum-message-attachments/
    "Berlin": "https://storage.googleapis.com/kaggle-forum-message-attachments/9
    "Nordrhein Westfalen": "https://storage.googleapis.com/kaggle-forum-message-
    "Schleswig Holstein": "https://storage.googleapis.com/kaggle-forum-message-a
    "Niedersachsen": "https://storage.googleapis.com/kaggle-forum-message-attach
    "Thüringen": "https://storage.googleapis.com/kaggle-forum-message-attachment
    "Mecklenburg Vorpommern": "https://storage.googleapis.com/kaggle-forum-messa
    "Sachsen Anhalt": "https://storage.googleapis.com/kaggle-forum-message-attac
    "Rheinland Pfalz": "https://storage.googleapis.com/kaggle-forum-message-atta
    "Hamburg": "https://storage.googleapis.com/kaggle-forum-message-attachments/
```

```
    "Saarland": "https://storage.googleapis.com/kaggle-forum-message-attachments
    "Bremen": "https://storage.googleapis.com/kaggle-forum-message-attachments/9
    "Baden Württemberg": "https://storage.googleapis.com/kaggle-forum-message-at
    "Bayern": "https://storage.googleapis.com/kaggle-forum-message-attachments/9
    "Hessen": "https://storage.googleapis.com/kaggle-forum-message-attachments/9
}
```

In [33]:
```
## It has already been done. If need to do it again, uncomment.
# # Directory to save the downloaded files
# output_dir = "datasets"

# # Create the directory if it does not exist
# os.makedirs(output_dir, exist_ok=True)

# # Function to download a file
# def download_file(url, filename):
#     response = requests.get(url)
#     response.raise_for_status()  # Check for request errors
#     with open(filename, 'wb') as file:
#         file.write(response.content)

# # Download each file
# for name, url in data_urls.items():
#     file_path = os.path.join(output_dir, f"{name}.csv")
#     print(f"Downloading {name} from {url} to {file_path}")
#     download_file(url, file_path)

# print("All files have been downloaded.")
```

4. Data Preprocessing

There are 180 stores missing 184 days of data in the middle of the series between 1 July 2014 to 31 Dec 2014. Compared to the 1 million given data records they can be neglected, since we are relying on the characteristics of stores, not the stors id. Especially, they will have a little effect on the prediction of the test data points which are 8 months away from the last day of the missing data We observe that for a period one of the stores has no record. After inspection, we find out that the specific store had been under refurbishment in that period.

4.1. Data Preparation

4.1.1. M[illegible]

[illegible]t

In [34]:
```
# Obtaining general info of the 'test_df' data frame.
test_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 41088 entries, 0 to 41087
Data columns (total 9 columns):
 #   Column         Non-Null Count  Dtype  
---  ------         --------------  -----  
 0   Store          41088 non-null  int64  
 1   DayOfWeek      41088 non-null  int64  
 2   Date           41088 non-null  object 
 3   Sales          0 non-null      float64
 4   Customers      0 non-null      float64
 5   Open           41077 non-null  float64
 6   Promo          41088 non-null  int64  
 7   StateHoliday   41088 non-null  object 
 8   SchoolHoliday  41088 non-null  int64  
dtypes: float64(3), int64(4), object(2)
memory usage: 2.8+ MB
```

In [35]:
```
# The complement of the above info
test_df.isnull().sum()
```

Out[35]:
```
Store               0
DayOfWeek           0
Date                0
Sales           41088
Customers       41088
Open               11
Promo               0
StateHoliday        0
SchoolHoliday       0
dtype: int64
```

In [36]:
```
# The number of test_df records with 0 or null 'Open'.
test_df.loc[(test_df['Open']==0) | test_df['Open'].isnull()].count()
```

Out[36]:
```
Store           5995
DayOfWeek       5995
Date            5995
Sales              0
Customers          0
Open            5984
Promo           5995
StateHoliday    5995
SchoolHoliday   5995
dtype: int64
```

In [37]:
```
# Obtainng the proportion of the closed stores for correcting the accuracy of th
test_closed_proportion = round(
    test_df.loc[(test_df['Open']==0) | test_df['Open'].isnull(), 'Store'].count(
    5
    )
print('The proportion of the closed stores in the test data set is:', f'{test_cl
```

```
The proportion of the closed stores in the test data set is: 14.591%
```

In [38]:
```
test_df = test_df.loc[~((test_df['Open']==0) | test_df['Open'].isnull())]
```

In [39]:
```
test_df = test_df.drop(['Open'], axis=1)
```

In [40]:
```
# Dropping the 'Sales' and 'Customers' from the data frame.
test_df.drop(['Sales', 'Customers'], axis=1, inplace=True)
```

In [41]:
```
# Final check of the null values for the test data frame.
test_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 35093 entries, 0 to 41087
Data columns (total 6 columns):
 #   Column         Non-Null Count  Dtype 
---  ------         --------------  ----- 
 0   Store          35093 non-null  int64 
 1   DayOfWeek      35093 non-null  int64 
 2   Date           35093 non-null  object
 3   Promo          35093 non-null  int64 
 4   StateHoliday   35093 non-null  object
 5   SchoolHoliday  35093 non-null  int64 
dtypes: int64(4), object(2)
memory usage: 1.9+ MB
```

4.1.1.2. Train

In [42]:
```
# Obtaining general info of the 'train_df' data frame.
train_df.info()
```

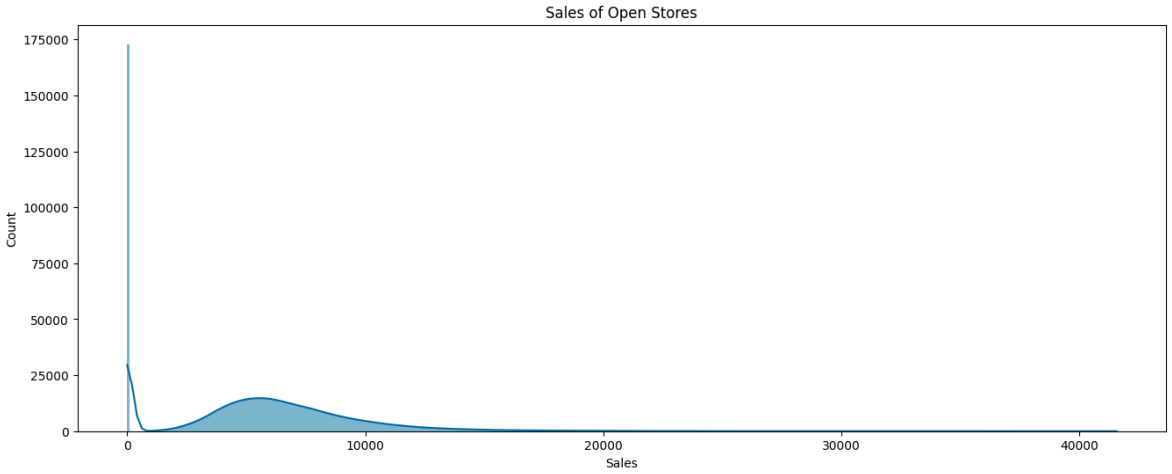
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1017209 entries, 0 to 1017208
Data columns (total 9 columns):
 #   Column         Non-Null Count    Dtype 
---  ------         --------------    ----- 
 0   Store          1017209 non-null  int64 
 1   DayOfWeek      1017209 non-null  int64 
 2   Date           1017209 non-null  object
 3   Sales          1017209 non-null  int64 
 4   Customers      1017209 non-null  int64 
 5   Open           1017209 non-null  int64 
 6   Promo          1017209 non-null  int64 
 7   StateHoliday   1017209 non-null  object
 8   SchoolHoliday  1017209 non-null  int64 
dtypes: int64(7), object(2)
memory usage: 69.8+ MB
```

There are no null values in the train data set. Nevertheless, following the steps that were applied on 'test_df' for stores that were closed, we observe that in 54 additional instances, 'Sales' is 0 (within which in 52 data rows, 'Customers' is also 0). Since the number is insignificant compared to the total number of observations, we will consider them as mistakes in inputting data, and drop them along the closed stores.

Or we can get rid of these zero sales by considring them as outliers, they will distort the shape of the sales distribution by giving a peak at the beginning since sales is non-negative.

In [44]:
```
# Create a new figure with a specified size
fig = plt.figure(figsize=(16,6))

# Add a subplot to the figure
ax1 = fig.add_subplot()
```

```
# Set the label for the x-axis
ax1.set_xlabel('Sales')

# Set the label for the y-axis
ax1.set_ylabel('Count')

# Set the title for the subplot
ax1.set_title(label='Sales of Open Stores', loc='center')

# Create a histogram using seaborn's histplot function
sns.histplot(train_df['Sales'], bins=400, kde=True, linewidth=0)

# Display the plot
plt.show()
```



In [45]:
```
# The number of test_df records with 0 or null 'Open'.
train_df.loc[(train_df['Open']==0)].count()
```

Out[45]:
```
Store            172817
DayOfWeek        172817
Date             172817
Sales            172817
Customers        172817
Open             172817
Promo            172817
StateHoliday     172817
SchoolHoliday    172817
dtype: int64
```

In [46]:
```
train_df.loc[(train_df['Open']==1) & (train_df['Sales']==0)].count()
```

Out[46]:
```
Store            54
DayOfWeek        54
Date             54
Sales            54
Customers        54
Open             54
Promo            54
StateHoliday     54
SchoolHoliday    54
dtype: int64
```

In [47]:
```
train_df.loc[(train_df['Open']==1) & (train_df['Sales']==0) & (train_df['Custome
```

Out[47]:
```
Store            52
DayOfWeek        52
Date             52
Sales            52
Customers        52
Open             52
Promo            52
StateHoliday     52
SchoolHoliday    52
dtype: int64
```

In [48]:
```
# Obtainng the proportion of the closed stores for correcting the accuracy of th
train_closed_proportion = round(
    train_df.loc[(train_df['Open']==0) | (train_df['Sales']==0), 'Store'].count(
    5
    )
print('The proportion of the closed stores in the test data set is:', f'{train_c
```

```
The proportion of the closed stores in the test data set is: 16.995%
```

In [49]:
```
train_df = train_df.loc[~((train_df['Open']==0) | (train_df['Sales']==0))]
```

In [50]:
```
train_df = train_df.drop(['Open'], axis=1)
```

In [51]:
```
train_df.info()
```

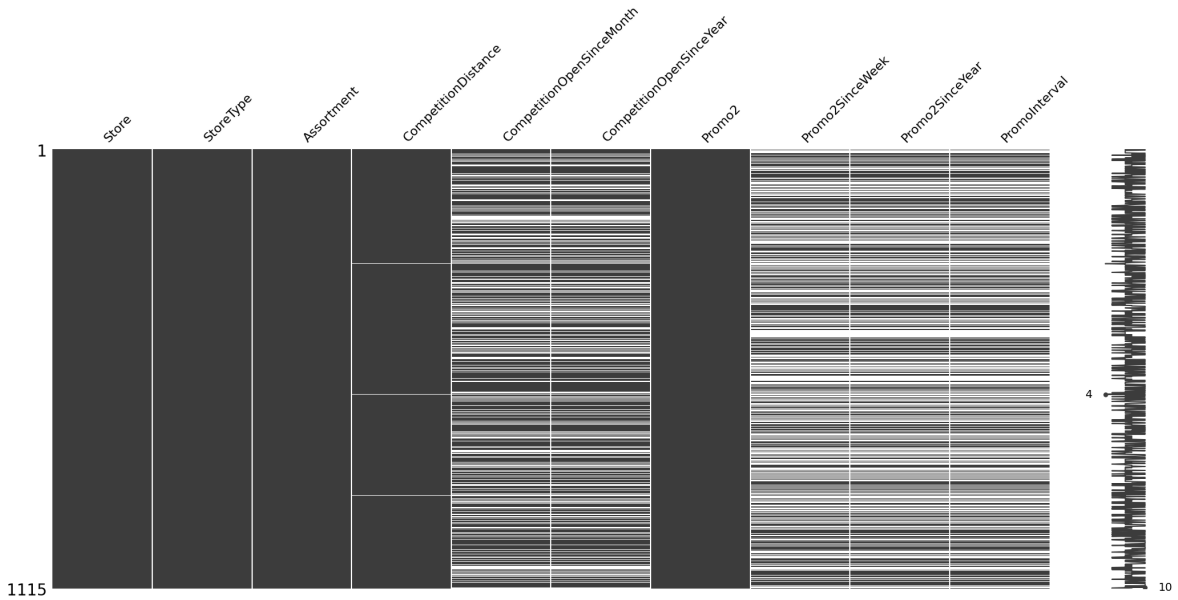
```
<class 'pandas.core.frame.DataFrame'>
Index: 844338 entries, 0 to 1017190
Data columns (total 8 columns):
 #   Column         Non-Null Count   Dtype 
---  ------         --------------   ----- 
 0   Store          844338 non-null  int64 
 1   DayOfWeek      844338 non-null  int64 
 2   Date           844338 non-null  object
 3   Sales          844338 non-null  int64 
 4   Customers      844338 non-null  int64 
 5   Promo          844338 non-null  int64 
 6   StateHoliday   844338 non-null  object
 7   SchoolHoliday  844338 non-null  int64 
dtypes: int64(6), object(2)
memory usage: 58.0+ MB
```

4.1.1.3. Store

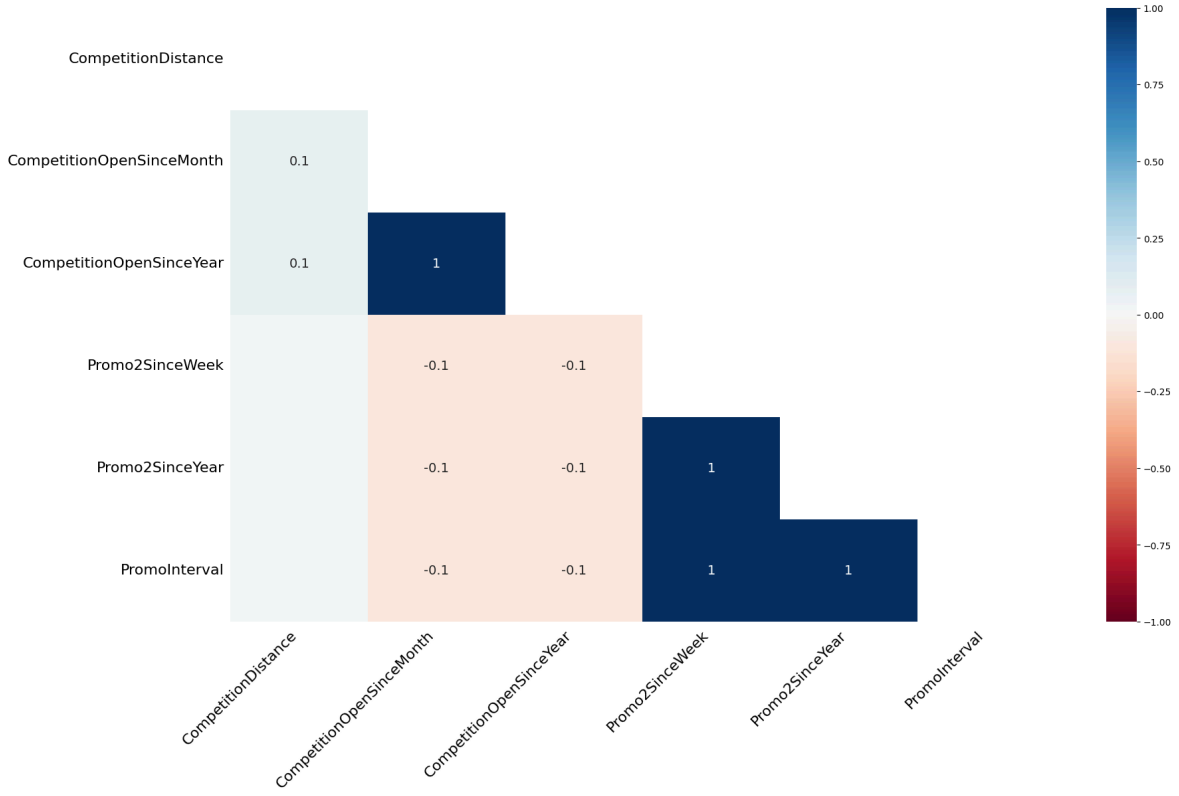
In [52]:
```
# Obtaining general info of the 'train_df' data frame.
store_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1115 entries, 0 to 1114
Data columns (total 10 columns):
 #   Column                     Non-Null Count  Dtype
---  ------                     --------------  -----
 0   Store                      1115 non-null   int64
 1   StoreType                  1115 non-null   object
 2   Assortment                 1115 non-null   object
 3   CompetitionDistance        1112 non-null   float64
 4   CompetitionOpenSinceMonth  761 non-null    float64
 5   CompetitionOpenSinceYear   761 non-null    float64
 6   Promo2                     1115 non-null   int64
 7   Promo2SinceWeek            571 non-null    float64
 8   Promo2SinceYear            571 non-null    float64
 9   PromoInterval              571 non-null    object
dtypes: float64(5), int64(2), object(3)
memory usage: 87.2+ KB
```

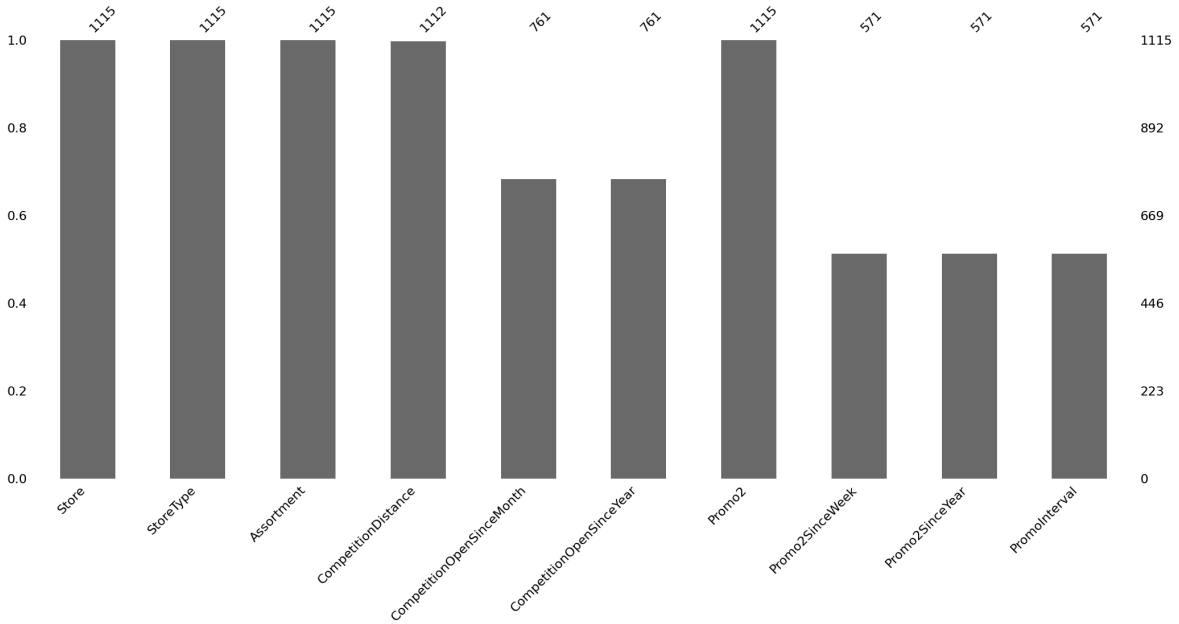
In [53]:
```
msno.matrix(store_df);
```



In [54]:
```
# Heatmap of missing data correlations
msno.heatmap(store_df);
```



In [55]:
```
# Bar chart of missing data
msno.bar(store_df);
```



3 missing values in CompetitionDistance

and so many in CompetitionOpenSinceMonth, CompetitionOpenSinceYear, Promo2SinceWeek, Promo2SinceYear, PromoInterval

In [56]:
```
# Missing Completely At Random (MCAR): doesn't depend on the values of the data
# Treating the null values of 'CompetitionDistance', 'CompetitionOpenSinceMonth'

# Impute CompetitionDistance with the median CompetitionDistance (median instead
store_df['CompetitionDistance'] = store_df['CompetitionDistance'].fillna(store_d

# Impute the CompetitionSinceMonth and CompetitionSinceYear with the minimum val
```

```
store_df['CompetitionOpenSinceMonth'] = store_df['CompetitionOpenSinceMonth'].fi
store_df['CompetitionOpenSinceYear'] = store_df['CompetitionOpenSinceYear'].fill
```

In [57]:
```
store_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1115 entries, 0 to 1114
Data columns (total 10 columns):
 #   Column                     Non-Null Count  Dtype
---  ------                     --------------  -----
 0   Store                      1115 non-null   int64
 1   StoreType                  1115 non-null   object
 2   Assortment                 1115 non-null   object
 3   CompetitionDistance        1115 non-null   float64
 4   CompetitionOpenSinceMonth  1115 non-null   float64
 5   CompetitionOpenSinceYear   1115 non-null   float64
 6   Promo2                     1115 non-null   int64
 7   Promo2SinceWeek            571 non-null    float64
 8   Promo2SinceYear            571 non-null    float64
 9   PromoInterval              571 non-null    object
dtypes: float64(5), int64(2), object(3)
memory usage: 87.2+ KB
```

In [58]:
```
store_df.loc[store_df['Promo2']==0].count()
```

Out[58]:
```
Store                        544
StoreType                    544
Assortment                   544
CompetitionDistance          544
CompetitionOpenSinceMonth    544
CompetitionOpenSinceYear     544
Promo2                       544
Promo2SinceWeek                0
Promo2SinceYear                0
PromoInterval                  0
dtype: int64
```

We observe that all the 1115 - 571 = 544 missing values correspond to the cases where there was no Promo2. Therefore, we will replace the values of Promo2SinceWeek' and 'Promo2SinceYear' with -1 to indicate a systematic missing value. For PromoInterval (which should have been named Promo2Interval), we raplce the null values with 'None'. Later we will change all the 'None' values to -1.

In [59]:
```
# Systematic missing values
# Treating the null values of 'Promo2SinceWeek', 'Promo2SinceYear', and 'PromoIn

# To indicate that these are systematic missing values they are replaced with -1
store_df['Promo2SinceWeek'] = store_df['Promo2SinceWeek'].fillna(-1)
store_df['Promo2SinceYear'] = store_df['Promo2SinceYear'].fillna(-1)
# Imputing the null cells with 'None'
store_df['PromoInterval'] = store_df['PromoInterval'].fillna('None')
```

In [60]:
```
store_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1115 entries, 0 to 1114
Data columns (total 10 columns):
 #   Column                     Non-Null Count  Dtype
---  ------                     --------------  -----
 0   Store                      1115 non-null   int64
 1   StoreType                  1115 non-null   object
 2   Assortment                 1115 non-null   object
 3   CompetitionDistance        1115 non-null   float64
 4   CompetitionOpenSinceMonth  1115 non-null   float64
 5   CompetitionOpenSinceYear   1115 non-null   float64
 6   Promo2                     1115 non-null   int64
 7   Promo2SinceWeek            1115 non-null   float64
 8   Promo2SinceYear            1115 non-null   float64
 9   PromoInterval              1115 non-null   object
dtypes: float64(5), int64(2), object(3)
memory usage: 87.2+ KB
```

4.1.2 Checking Compatibility of the Data Values with the Meta Data

In [61]:
```
test_df.describe().T
```

Out[61]:

	count	mean	std	min	25%	50%	75%	max
Store	35093.0	555.990055	320.142280	1.0	280.0	553.0	833.0	1115.0
DayOfWeek	35093.0	3.470977	1.722496	1.0	2.0	3.0	5.0	7.0
Promo	35093.0	0.462998	0.498636	0.0	0.0	0.0	1.0	1.0
SchoolHoliday	35093.0	0.500698	0.500007	0.0	0.0	1.0	1.0	1.0

In [62]:
```
train_df.describe().T
```

Out[62]:

	count	mean	std	min	25%	50%	75%	max
Store	844338.0	558.421374	321.730861	1.0	280.0	558.0	837.0	1115.0
DayOfWeek	844338.0	3.520350	1.723712	1.0	2.0	3.0	5.0	7.0
Sales	844338.0	6955.959134	3103.815515	46.0	4859.0	6369.0	8360.0	41551.0
Customers	844338.0	762.777166	401.194153	8.0	519.0	676.0	893.0	7388.0
Promo	844338.0	0.446356	0.497114	0.0	0.0	0.0	1.0	1.0
SchoolHoliday	844338.0	0.193578	0.395102	0.0	0.0	0.0	0.0	1.0

In [63]:
```
store_df.describe().T
```

Out[63]:

	count	mean	std	min	25%	50%
Store	1115.0	558.000000	322.017080	1.0	279.5	558.0
CompetitionDistance	1115.0	5396.614350	7654.513635	20.0	720.0	2325.0
CompetitionOpenSinceMonth	1115.0	5.248430	3.929836	1.0	1.0	4.0
CompetitionOpenSinceYear	1115.0	1974.167713	50.866086	1900.0	1900.0	2006.0
Promo2	1115.0	0.512108	0.500078	0.0	0.0	1.0
Promo2SinceWeek	1115.0	11.595516	15.925223	-1.0	-1.0	1.0
Promo2SinceYear	1115.0	1029.751570	1006.538860	-1.0	-1.0	2009.0

Useless note!

Taking into account Promo and Promo2 are binary variables, looking at the mean and standard deviation, considering their first and second order momentums, we can assume that they have a Bernoulli distribution.

In [64]:
```
(0.45 * 0.55)**0.5
```

Out[64]:
```
0.49749371855331
```

In [65]:
```
(0.51 * 0.49)**0.5
```

Out[65]:
```
0.4998999899979995
```

4.1.2.1 Checking the values of categorical variables

In [66]:
```
# Defining a function for checking the values of variables
def check_categorical_vals(df: pd.DataFrame, cat_vars: list):
    for col in df[cat_vars]:
        print(col)
        print(df[col].dtype)
        print(df[col].nunique())
        print(df[col].unique())
        print('_____________________________')
```

In [67]:
```
def create_categorical_variables_list(df: pd.DataFrame, exclude_list: list=['Stc
    return [col for col in df if col not in exclude_list]
```

In [68]:
```
# Listing the categorical variables in 'train_df' and 'test_df'
exclude_list = ['Store', 'Date', 'Sales', 'Customers', 'CompetitionDistance']
cat_vars = create_categorical_variables_list(train_df, exclude_list)
```

In [69]:
```
check_categorical_vals(train_df, cat_vars)
```

```
DayOfWeek
int64
7
[5 4 3 2 1 7 6]
--------------------------------------------------
Promo
int64
2
[1 0]
--------------------------------------------------
StateHoliday
object
4
['0' 'a' 'b' 'c']
--------------------------------------------------
SchoolHoliday
int64
2
[1 0]
--------------------------------------------------
```

In [70]:
```
# Checking the value of the categorical variables in 'test_df'
check_categorical_vals(test_df, cat_vars)
```

```
DayOfWeek
int64
7
[4 3 2 1 7 6 5]
--------------------------------------------------
Promo
int64
2
[1 0]
--------------------------------------------------
StateHoliday
object
2
['0' 'a']
--------------------------------------------------
SchoolHoliday
int64
2
[0 1]
--------------------------------------------------
```

We observe that StateHoliday is of String dtype, and there is no Easter or Christmas holiday in the given test interval.

In [71]:
```
# Listing the categorical variables in 'store_df'
cat_vars = create_categorical_variables_list(store_df, exclude_list)
```

In [72]:
```
# Checking the value of the categorical variables in 'test_df'
check_categorical_vals(store_df, cat_vars)
```

```
StoreType
object
4
['c' 'a' 'd' 'b']
____________________________
Assortment
object
3
['a' 'c' 'b']
____________________________
CompetitionOpenSinceMonth
float64
12
[ 9. 11. 12.  4. 10.  8.  1.  3.  6.  5.  2.  7.]
____________________________
CompetitionOpenSinceYear
float64
23
[2008. 2007. 2006. 2009. 2015. 2013. 2014. 2000. 2011. 1900. 2010. 2005.
 1999. 2003. 2012. 2004. 2002. 1961. 1995. 2001. 1990. 1994. 1998.]
____________________________
Promo2
int64
2
[0 1]
____________________________
Promo2SinceWeek
float64
25
[-1. 13. 14.  1. 45. 40. 26. 22.  5.  6. 10. 31. 37.  9. 39. 27. 18. 35.
 23. 48. 36. 50. 44. 49. 28.]
____________________________
Promo2SinceYear
float64
8
[-1.000e+00  2.010e+03  2.011e+03  2.012e+03  2.009e+03  2.014e+03
  2.015e+03  2.013e+03]
____________________________
PromoInterval
object
4
['None' 'Jan,Apr,Jul,Oct' 'Feb,May,Aug,Nov' 'Mar,Jun,Sept,Dec']
____________________________
```

The value are compatible with the provided meta data.

4.1.3 Data Transformation and Feature Generation

There is no point in converting the values of categorcial variables from string to numbers, other than lower memory consumption. However, since we will lose expressiveness and ultimately they should (are better to) be converted to dummy variables, we won't do it.

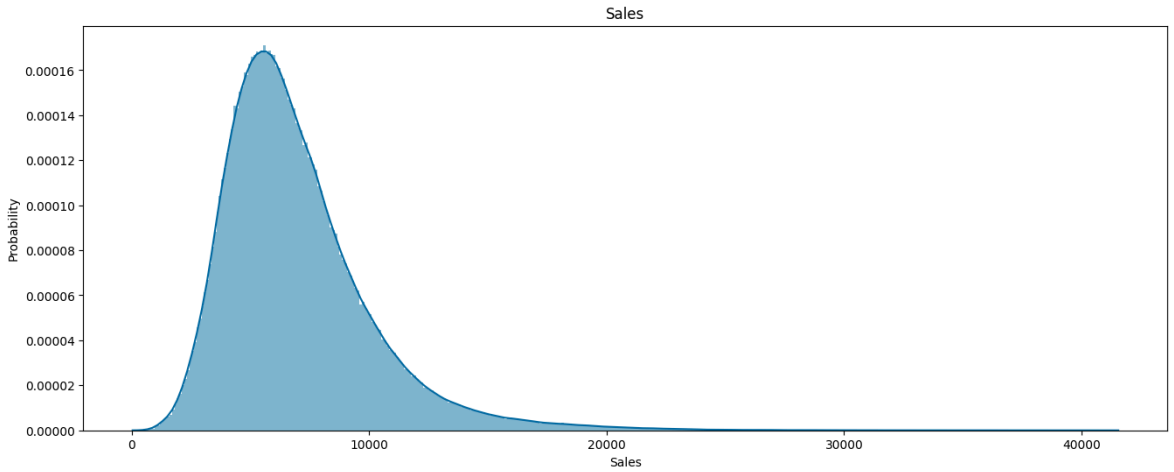
4.1.3.1 Sales

In [73]:
```
print(f'The skewness of Sales is {train_df['Sales'].skew():.2f}')
```

```
The skewness of Sales is 1.59
```

Sales is skewed to right.

In [74]:
```
# Create a new figure with a specified size
fig = plt.figure(figsize=(16,6))

# Add a subplot to the figure
ax1 = fig.add_subplot()

# Set the label for the x-axis
ax1.set_xlabel('Sales')

# Set the label for the y-axis
ax1.set_ylabel('Probability')

# Set the title for the subplot
ax1.set_title(label='Sales', loc='center')

# Create a histogram using seaborn's histplot function
sns.histplot(train_df['Sales'], bins=400, stat='density', kde=True, linewidth=0)

# Display the plot
plt.show()
```

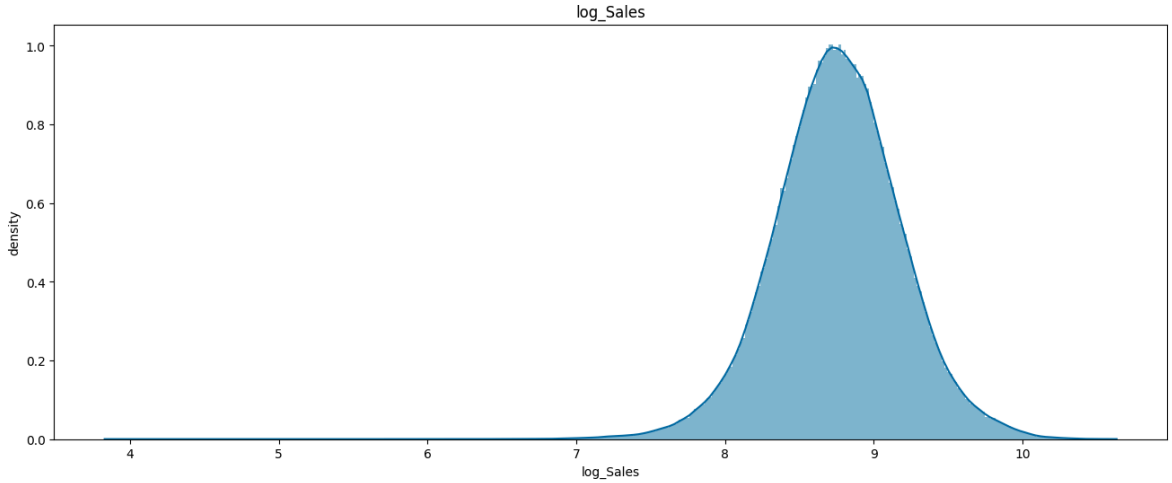


Since it's skewed we should use log-transformation

In [75]:
```
train_df['log_Sales'] = np.log(train_df.Sales)
```

In [76]:
```
# Create a new figure with a specified size
fig = plt.figure(figsize=(16,6))

# Add a subplot to the figure
ax1 = fig.add_subplot()

# Set the label for the x-axis
ax1.set_xlabel('log_Sales')

# Set the label for the y-axis
ax1.set_ylabel('density')

# Set the title for the subplot
ax1.set_title(label='log_Sales', loc='center')

# Create a histogram using seaborn's histplot function
```

```
sns.histplot(train_df['log_Sales'], bins=400, stat='density', kde=True, linewidt

# Display the plot
plt.show()
```



In [77]:
```
print(f'The skewness of Sales is {train_df['log_Sales'].skew():.2f}')
```

```
The skewness of Sales is -0.11
```

It's between -0.5 and 0.5, so we can consider it as approximately symmetric. Nevertheless, we can see there is still a thin tail on the left If we calculate the kurtosis we see that it is less than 3 (it's paltykurtic)

In [78]:
```
train_df['log_Sales'].kurtosis()
```

Out[78]:
```
0.6548797052359507
```

The kurtosis of the normal distribution is 3. The 'log_Sales' still doesn't have a normal distribution. Since we want to fit a linear regression model we should take this into consideration.

In [79]:
```
train_df.drop('log_Sales', axis=1, inplace=True)
```

Winsorizing removal before transforming

In [80]:
```
temp_train_df = train_df.copy()
```

In [81]:
```
temp_train_df['Sales'] = winsorize(temp_train_df['Sales'], limits=[0.05, 0.05])
```

In [82]:
```
temp_train_df['Sales'].skew(), temp_train_df['Sales'].kurt()
```

Out[82]:
```
(0.6697800884197596, -0.31297378957636823)
```

In [83]:
```
temp_train_df['Customer'] = winsorize(temp_train_df['Customers'], limits=[0.05,
```

In [84]:
```
temp_train_df['Customer'].skew(), temp_train_df['Customer'].kurt()
```

Out[84]:
```
(0.8614283328319557, 0.05888577736525846)
```

In [85]:
```
temp_train_df['Sales'].skew(), temp_train_df['Sales'].kurt()
```

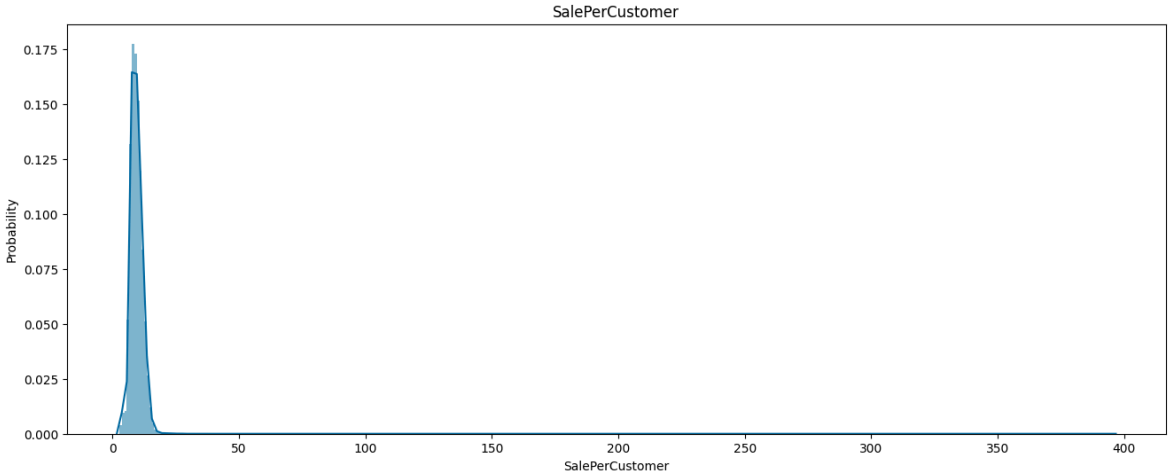
Out[85]: (0.6697800884197596, -0.31297378957636823)

In [86]:
```
temp_train_df['SalePerCustomer'] = temp_train_df['Sales'] / temp_train_df['Custc
```

In [87]:
```
temp_train_df['SalePerCustomer'].skew(), temp_train_df['SalePerCustomer'].kurt()
```

Out[87]: (7.481250540144326, 978.6413934233807)

In [88]:
```
# Create a new figure with a specified size
fig = plt.figure(figsize=(16,6))

# Add a subplot to the figure
ax1 = fig.add_subplot()

# Set the label for the x-axis
ax1.set_xlabel('SalePerCustomer')

# Set the label for the y-axis
ax1.set_ylabel('Probability')

# Set the title for the subplot
ax1.set_title(label='SalePerCustomer', loc='center')

# Create a histogram using seaborn's histplot function
sns.histplot(temp_train_df['SalePerCustomer'], bins=400, stat='density', kde=Tru

# Display the plot
plt.show()
```



That's awful!

And considerin the peak, IQR outlier removal will make it even worse!

In [89]:
```
del temp_train_df
```

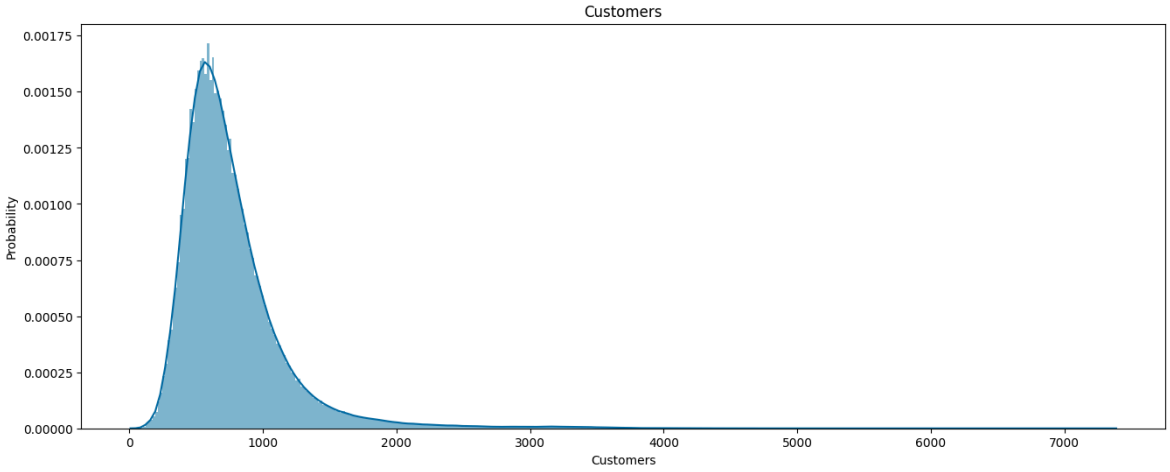
4.1.3.2. Customers

In [90]:
```
print(f'The skewness of Customers is {train_df['Customers'].skew():.2f}')
```

The skewness of Customers is 2.79

In [91]:
```
# Create a new figure with a specified size
fig = plt.figure(figsize=(16,6))
```

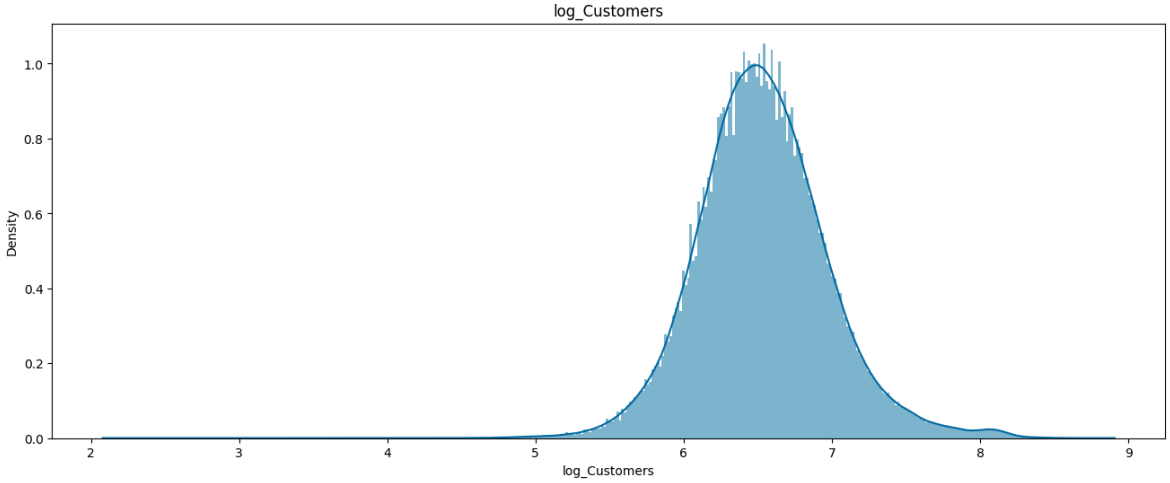
```
# Add a subplot to the figure
ax1 = fig.add_subplot()

# Set the label for the x-axis
ax1.set_xlabel('Customers')

# Set the label for the y-axis
ax1.set_ylabel('Probability')

# Set the title for the subplot
ax1.set_title(label='Customers', loc='center')

# Create a histogram using seaborn's histplot function
sns.histplot(train_df['Customers'], bins=400, stat='density', kde=True, linewidt

# Display the plot
plt.show()
```



Since it's skewed we should use log-transformation

In [92]:
```
train_df['log_Customers'] = np.log(train_df.Customers)
```

In [93]:
```
# Create a new figure with a specified size
fig = plt.figure(figsize=(16,6))

# Add a subplot to the figure
ax1 = fig.add_subplot()

# Set the label for the x-axis
ax1.set_xlabel('log_Customers')

# Set the label for the y-axis
ax1.set_ylabel('Density')

# Set the title for the subplot
ax1.set_title(label='log_Customers', loc='center')

# Create a histogram using seaborn's histplot function
sns.histplot(train_df['log_Customers'], bins=400, stat='density', kde=True, line

# Display the plot
plt.show()
```



In [94]:
```
print(f'The skewness of log_Customers is {train_df['log_Customers'].skew():.2f}'
```

```
The skewness of log_Customers is 0.31
```

It's between -0.5 and 0.5, so we can consider it as approximately symmetric. Nevertheless, we can see there is still a thin tail on the left If we calculate the kurtosis we see that it is less than 3 (it's paltykurtic)

In [95]:
```
train_df['log_Customers'].kurtosis()
```

Out[95]:
```
1.0689316134821158
```

The kurtosis of the normal distribution is 3. The 'log_Customers' still doesn't have a normal distribution. Since we wan to fit a linear regression model we should take this into consideration.

In [96]:
```
train_df.drop('log_Customers', axis=1, inplace=True)
```

4.1.3.3. SalesPerCustomer

In [97]:
```
train_df['SalePerCustomer'] = train_df.Sales / train_df.Customers
```

In [98]:
```
print(f'The skewness of Sales is {train_df['SalePerCustomer'].skew():.2}')
```

```
The skewness of Sales is 0.59
```

In [99]:
```
train_df['SalePerCustomer'].kurtosis()
```

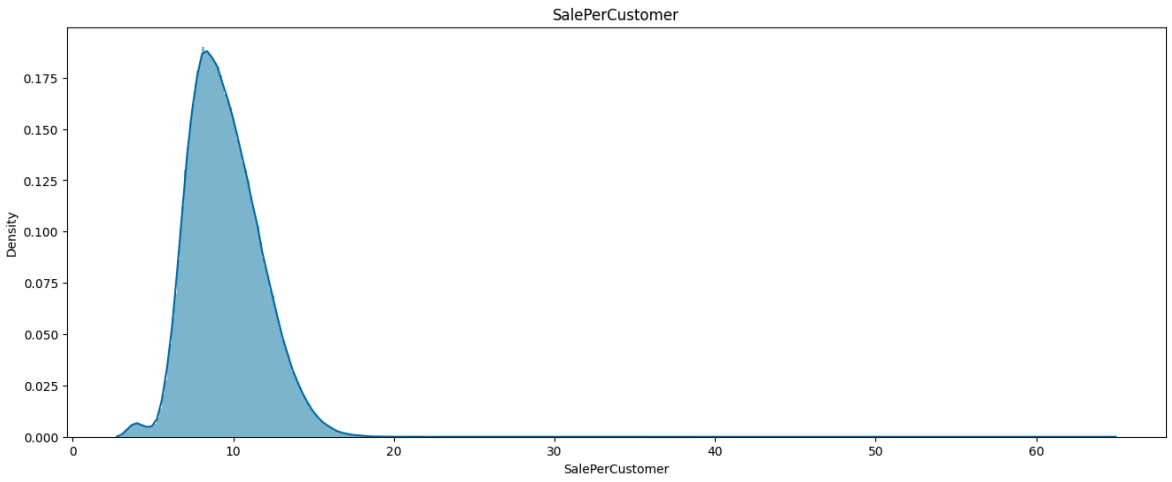
Out[99]:
```
2.759037254095608
```

We observe that SalesPerCustomer has a distribution that is very close to the normal distribution. The advantage of considering sale per customer is that we have to deal with only one output variable. Nevertheless, neither of Sales or CUstomers can be inferred from it.

In [100…
```
train_df['SalePerCustomer'].describe().T
```

Out[100…
```
count    844338.000000
mean          9.493641
std           2.197448
min           2.749075
25%           7.895571
50%           9.250000
75%          10.899729
max          64.957854
Name: SalePerCustomer, dtype: float64
```

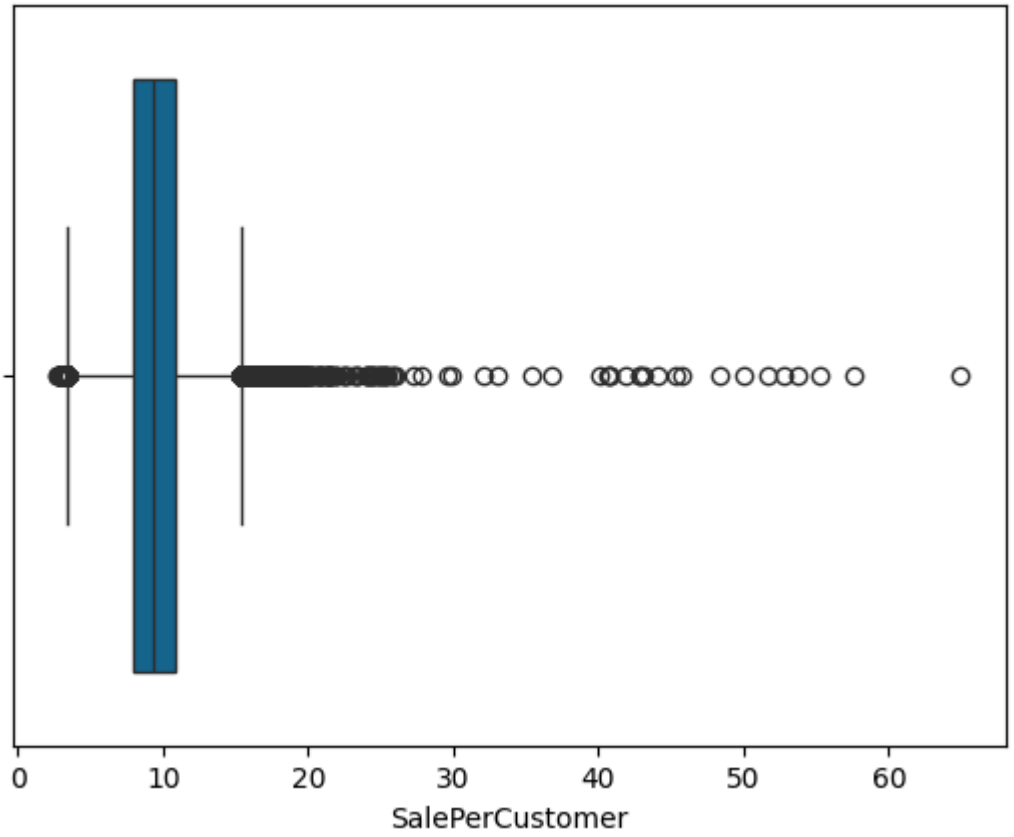
In [101…
```
# Create a new figure with a specified size
fig = plt.figure(figsize=(16,6))

# Add a subplot to the figure
ax1 = fig.add_subplot()

# Set the label for the x-axis
ax1.set_xlabel('SalePerCustomer')

# Set the label for the y-axis
ax1.set_ylabel('Density')

# Set the title for the subplot
ax1.set_title(label='SalePerCustomer', loc='center')

# Create a histogram using seaborn's histplot function
sns.histplot(train_df['SalePerCustomer'], bins=400, stat='density', kde=True, li

# Display the plot
plt.show()
```



Winsorizing after

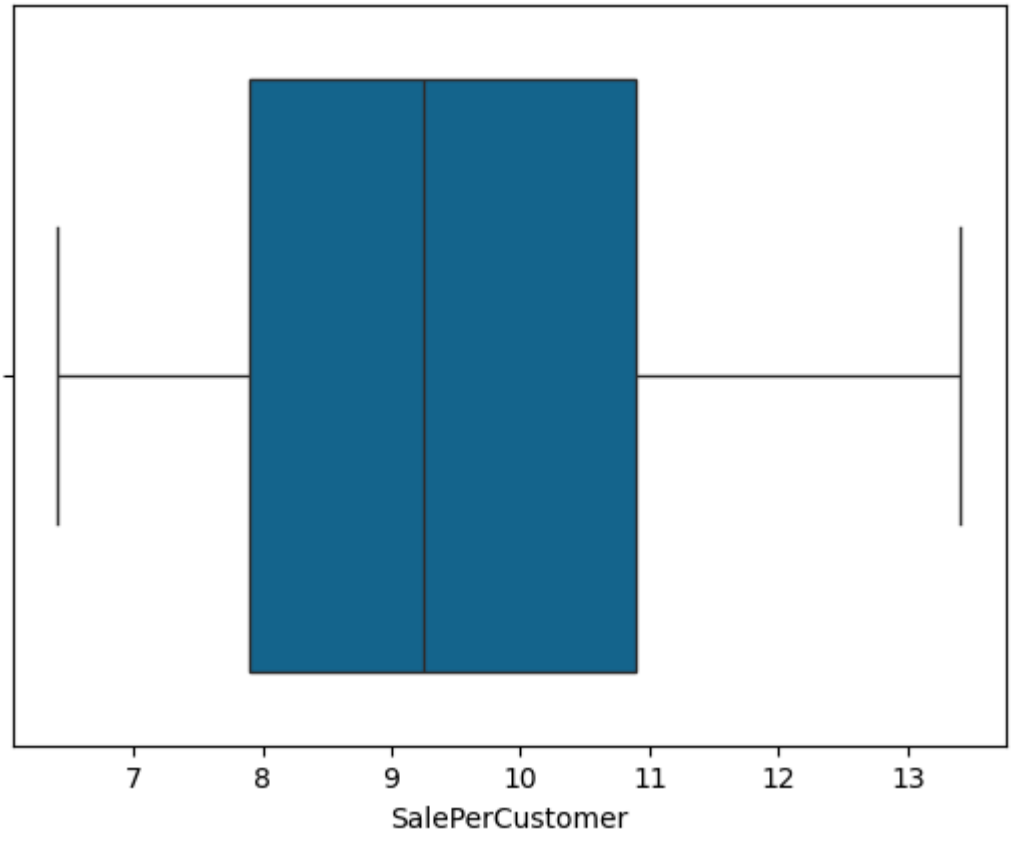
In [102…
```
temp_train_df = train_df.copy()
```

In [103…
```
sns.boxplot(x='SalePerCustomer', data=temp_train_df);
```



In [104…
```
temp_train_df['SalePerCustomer'] = winsorize(temp_train_df['SalePerCustomer'], l
```

In [105…
```
sns.boxplot(x='SalePerCustomer', data=temp_train_df);
```



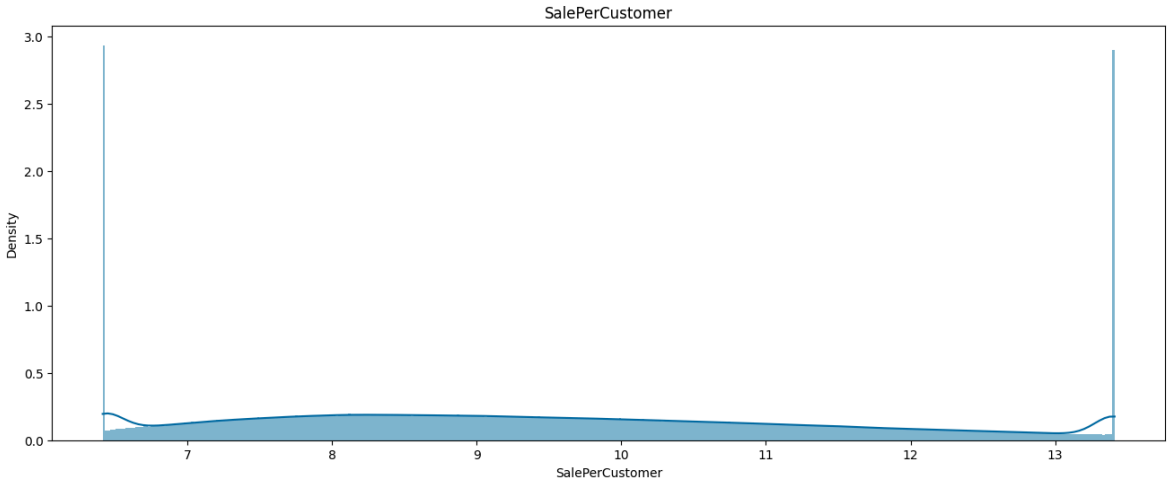
In [106…
```
temp_train_df['SalePerCustomer'].skew()
```

Out[106… 0.3475071427437537

In [107…
```
temp_train_df['SalePerCustomer'].kurtosis()
```

Out[107… -0.8181886356796237

In [108…
```
# Create a new figure with a specified size
fig = plt.figure(figsize=(16,6))

# Add a subplot to the figure
ax1 = fig.add_subplot()

# Set the label for the x-axis
ax1.set_xlabel('SalePerCustomer')

# Set the label for the y-axis
ax1.set_ylabel('Density')

# Set the title for the subplot
ax1.set_title(label='SalePerCustomer', loc='center')

# Create a histogram using seaborn's histplot function
sns.histplot(temp_train_df['SalePerCustomer'], bins=400, stat='density', kde=Tru

# Display the plot
plt.show()
```



We shouldn't Winsorize!

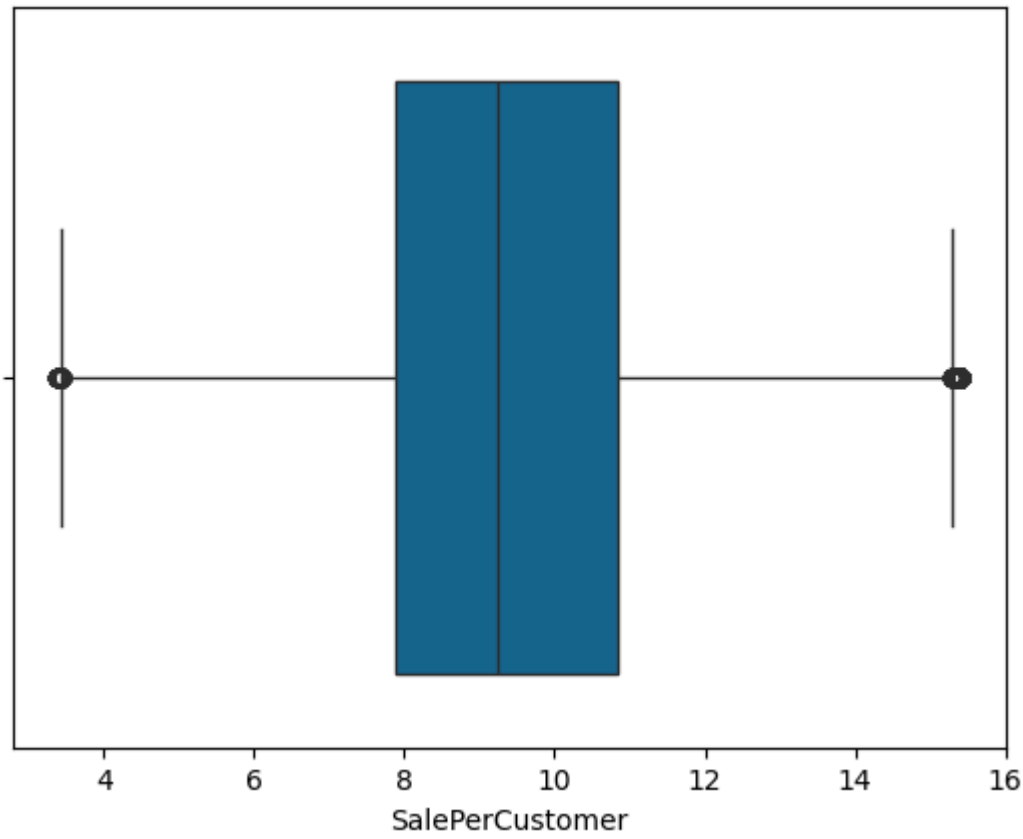
Outlier removal using the IQR after

In [109…
```
temp_train_df = train_df.copy()
```

In [110…
```
# Calculating the 1st, 3rd, and InterQuartile Range (IQR)
Q1 = temp_train_df['SalePerCustomer'].quantile(0.25)
Q3 = temp_train_df['SalePerCustomer'].quantile(0.75)
IQR = Q3 - Q1
```

In [111…
```
temp_train_df = temp_train_df[(temp_train_df['SalePerCustomer'] >= Q1 - 1.5*IQR)
```

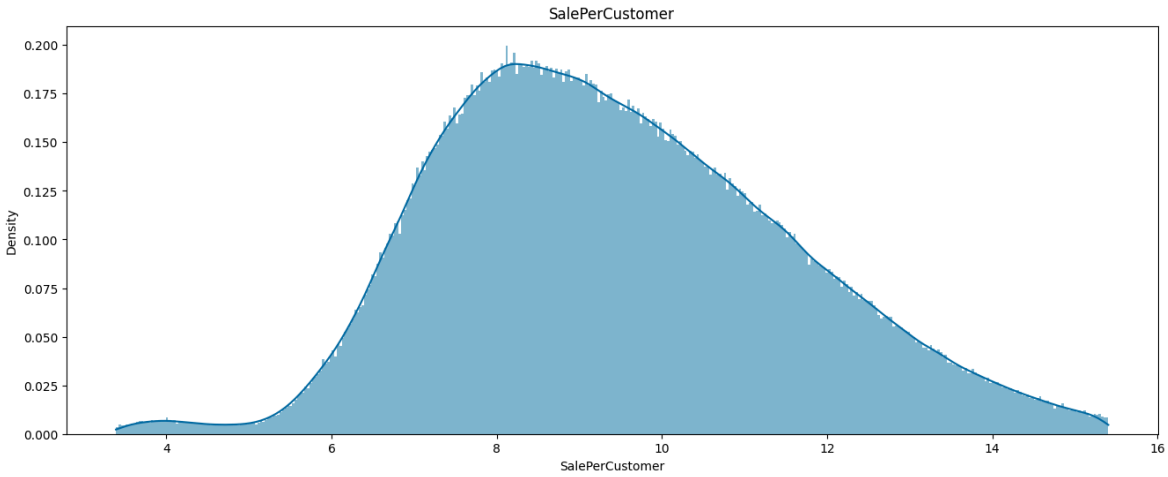
In [112…
```
sns.boxplot(x='SalePerCustomer', data=temp_train_df);
```



In [113…
```
temp_train_df['SalePerCustomer'].skew(), temp_train_df['SalePerCustomer'].kurt()
```

Out[113…
```
(0.32081688158880334, -0.23209609632323858)
```

In [114…
```
# Create a new figure with a specified size
fig = plt.figure(figsize=(16,6))

# Add a subplot to the figure
ax1 = fig.add_subplot()

# Set the label for the x-axis
ax1.set_xlabel('SalePerCustomer')

# Set the label for the y-axis
ax1.set_ylabel('Density')

# Set the title for the subplot
ax1.set_title(label='SalePerCustomer', loc='center')

# Create a histogram using seaborn's histplot function
sns.histplot(temp_train_df['SalePerCustomer'], bins=400, stat='density', kde=Tru

# Display the plot
plt.show()
```



The IQR outlier elimination makes compatibility of the data with the normal distribution worse and shouldn't be used.

4.1.3.3.1. Checking log_SalesPerCustomer

In [115…
```
train_df['log_SalePerCustomer'] = np.log(train_df.Sales / train_df.Customers)
```

In [116…
```
print(f'The skewness of Sales is {train_df['log_SalePerCustomer'].skew()}')
```

```
The skewness of Sales is -0.2999706390855646
```

In [117…
```
train_df['log_SalePerCustomer'].kurtosis()
```

Out[117…
```
0.7352351674844368
```

In [118…
```
train_df.drop('log_SalePerCustomer', axis=1, inplace=True)
```

4.1.3.3.2. Checking log_SalesPerlog_Customer

In [119…
```
train_df['log_SalePerlog_Customer'] = np.log(train_df.Sales) / np.log(train_df.C
```

In [120…
```
print(f'The skewness of Sales is {train_df['log_SalePerlog_Customer'].skew()}')
```

```
The skewness of Sales is -0.19613083269450354
```

In [121…
```
train_df['log_SalePerlog_Customer'].kurtosis()
```

Out[121…
```
0.4693621301654889
```

bseides being useless, how are we supposed to convert it to something related to sales and customers?!

In [122…
```
train_df.drop('log_SalePerlog_Customer', axis=1, inplace=True)
```

In [123…
```
train_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 844338 entries, 0 to 1017190
Data columns (total 9 columns):
 #   Column           Non-Null Count   Dtype  
---  ------           --------------   -----  
 0   Store            844338 non-null  int64  
 1   DayOfWeek        844338 non-null  int64  
 2   Date             844338 non-null  object 
 3   Sales            844338 non-null  int64  
 4   Customers        844338 non-null  int64  
 5   Promo            844338 non-null  int64  
 6   StateHoliday     844338 non-null  object 
 7   SchoolHoliday    844338 non-null  int64  
 8   SalePerCustomer  844338 non-null  float64
dtypes: float64(1), int64(6), object(2)
memory usage: 64.4+ MB
```

4.1.3.4. Date

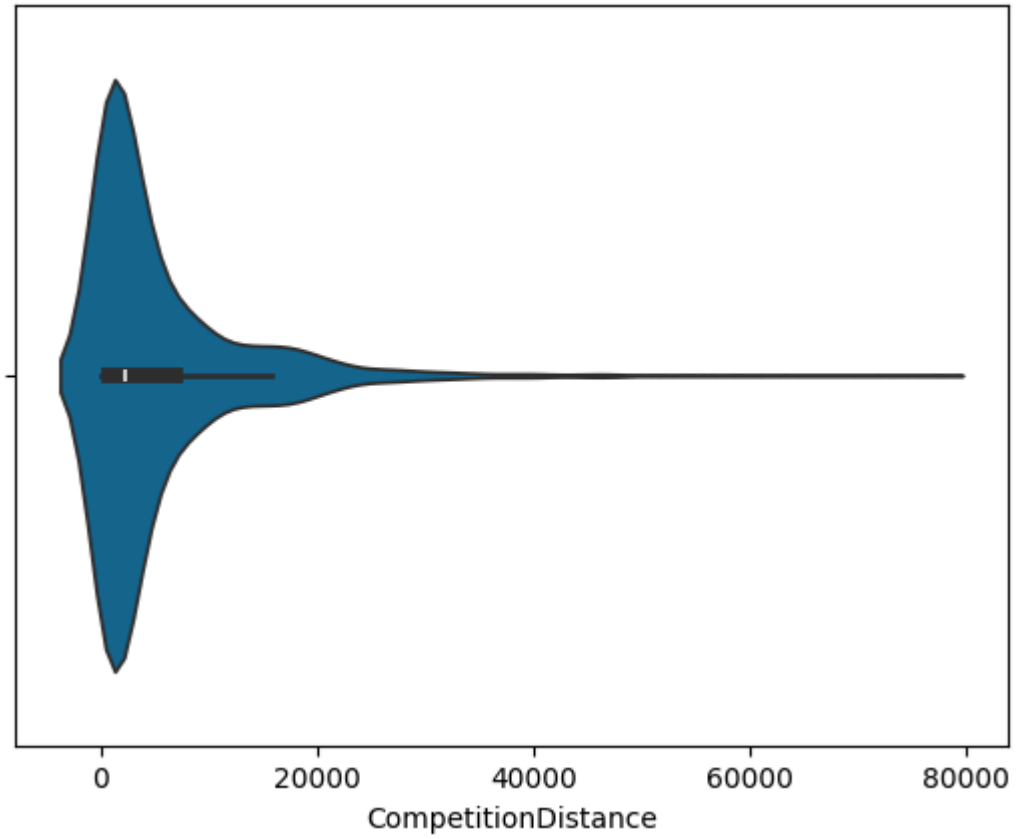
In [124…
```python
def extract_date_elements(df: pd.DataFrame, col_name='Date'):
    '''Convert Date, and Extract Year and Month'''
    df['Date'] = pd.to_datetime(df['Date'], format='%d/%m/%Y')
    df['Year'] = pd.DatetimeIndex(df['Date']).year
    df['Month'] = pd.DatetimeIndex(df['Date']).month
```

In [125…
```python
# Extract Date elements for 'train_df'
extract_date_elements(train_df)
```

In [126…
```python
# Extract Date elements for 'test_df'
extract_date_elements(test_df)
```

4.1.3.5. CompetitionDistance

In [127…
```python
sns.violinplot(x=store_df['CompetitionDistance']);
```



In [128…
```
store_df.loc[store_df['CompetitionDistance'] > 30000, 'Store'].count()
```

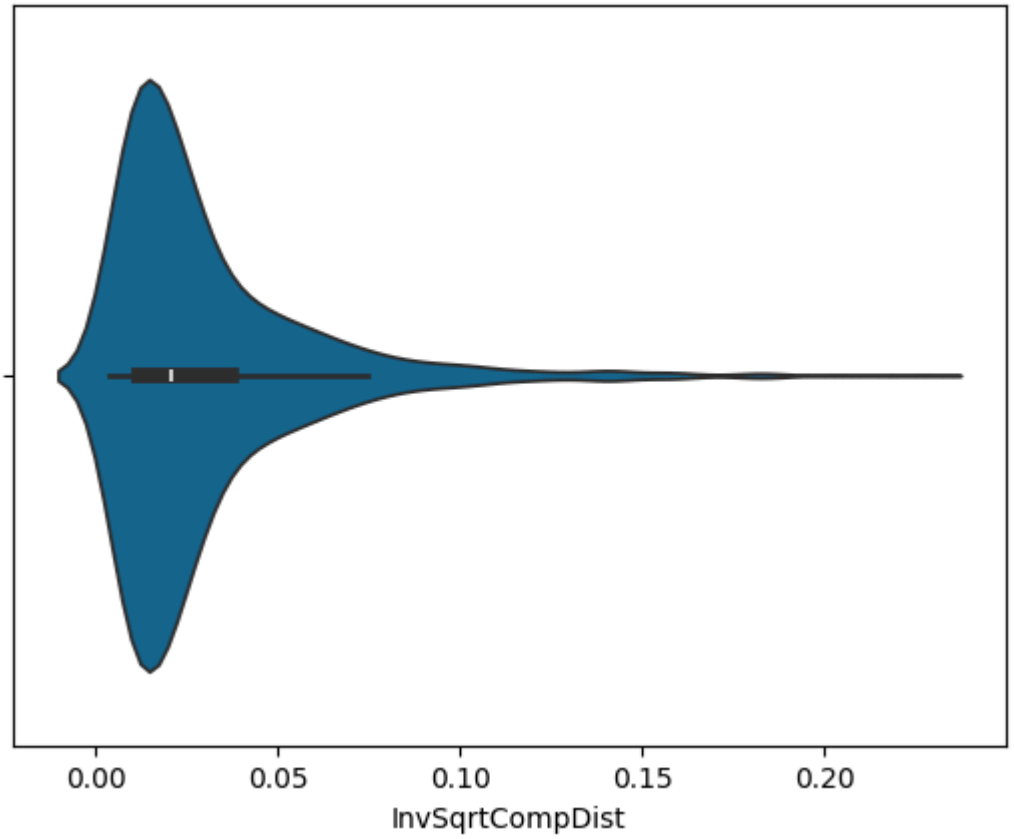
Out[128…
```
19
```

Winsorizing sounds a valid idea, nevertheless, before that we try the 1/sqrt(CompetitionDistance) transformation.

4.1.3.5.1. Inverse Squred Root Competition Distance

In [129…
```
store_df['InvSqrtCompDist'] = 1 / np.sqrt(store_df['CompetitionDistance'])
```

In [130…
```
sns.violinplot(x=store_df['InvSqrtCompDist']);
```



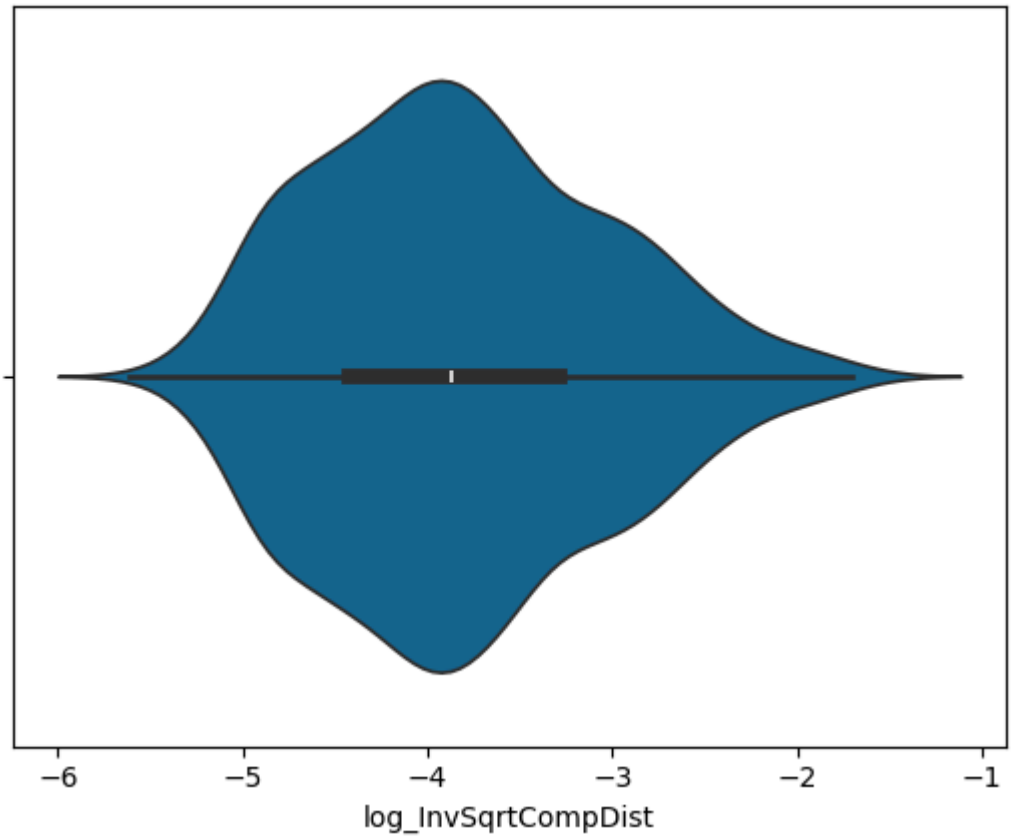
It's not that useful, so we drop it.

In [131…
```
store_df.drop('InvSqrtCompDist', axis=1, inplace=True)
```

4.1.3.5.2. Log Inverse Squred Root Competition Distance

In [132…
```
store_df['log_InvSqrtCompDist'] = np.log(1 / np.sqrt(store_df['CompetitionDistan
```

In [133…
```
sns.violinplot(x=store_df['log_InvSqrtCompDist']);
```



In [134…
```
store_df['log_InvSqrtCompDist'].skew(), store_df['log_InvSqrtCompDist'].kurt()
```

Out[134…
```
(0.35778091531811834, -0.4144648444769641)
```

It has a good symmetry. While not as critical as some other characteristics, considering symmetry and outliers can significantly improve the performance and reliability of a regression model. Addressing these aspects helps in meeting the assumptions of regression techniques and ensures more accurate and interpretable results.

4.1.3.5.3. Winsorizing

By Winsorizing we are replacing the values outside of the given bound, with the extreme values. With the applied transformation, we don't need Windosrizing anymore.

In [135…
```
# temp_store_df = store_df.copy()

# temp_store_df['CompetitionDistance'] = winsorize(temp_store_df['CompetitionDis

# sns.violinplot(x=temp_store_df['CompetitionDistance']);

# temp_store_df.loc[temp_store_df['CompetitionDistance'] > 17500, 'Store'].count
```

4.1.4. Data Normalization and Standardization

Depending on the prediction model we should do this and in some of the in-built functions these transformations exist and are automatically applied or required to be invoked.

4.2. Feature Selection

Dimensionality reduction is important in avoiding the curse of dimensionality.

We do this is linear regression, to avoid multicollinearity

This is also applied automatically in regression tree (random forest, ada boost, xg boost) by using the criteria for extending the tree

4.3. Feature Generation

We have already generated some new features.

This is done automatically in deep learning in the applied LSTM model.

Transforming categorical variables to the one-hot (1-out-of-k) encoding is done after integrating the data sets, since some of the categorical variables have interaction effects on each other.

Considering Promo2SinceWeek and Promo2SinceYear is meaningless. We should either transform them to categorical variables, which would make the model unnecessarily huge, or alternatively include the duration, in weeks, and drop them from the analysis.

Since the prediction is for six weeks (the duration is low) starting from the 1st of August of 2015, we will convert the aforementioned attribute to the correpsonding duration, and name it Promo2Duration. If the model is intended to be used for other dayes that are farther from the current prediction period, some other way has to be sorted out.

In [136…
```
new_store_df = store_df.copy()
```

4.3.1. Promo2Duration

In [137…
```
# Since some of the stores are not involved in Promo2, the Promo2Duration would
# (new_store_df['Promo2SinceWeek'] > 0) has been used as a mask for this purpose
new_store_df['Promo2Duration'] = \
((2015 - new_store_df['Promo2SinceYear'])*52 + (7*52 - new_store_df['Promo2Since
```

In [138…
```
new_store_df.head()
```

Out[138...

	Store	StoreType	Assortment	CompetitionDistance	CompetitionOpenSinceMonth	C
0	1	c	a	1270.0	9.0	
1	2	a	a	570.0	11.0	
2	3	a	a	14130.0	12.0	
3	4	c	c	620.0	9.0	
4	5	a	a	29910.0	4.0	

In [139...
```
new_store_df.drop(['Promo2SinceWeek', 'Promo2SinceYear'], axis=1, inplace=True)
```

4.3.2 CompetitionDuration

The same argument holds for 'CompetitionOpenSinceMonth' and 'CompetitionOpenSinceYear', but this time the duration is in months.

In [140...
```
new_store_df['CompetitionDuration'] = \
((2015 - new_store_df['CompetitionOpenSinceYear'])*12 + (7*12 - new_store_df['Cc
```

In [141...
```
new_store_df.drop(['CompetitionOpenSinceMonth', 'CompetitionOpenSinceYear'], axi
```

In [142...
```
new_store_df.head()
```

Out[142...

	Store	StoreType	Assortment	CompetitionDistance	Promo2	PromoInterval	log_Inv
0	1	c	a	1270.0	0	None	
1	2	a	a	570.0	1	Jan,Apr,Jul,Oct	
2	3	a	a	14130.0	1	Jan,Apr,Jul,Oct	
3	4	c	c	620.0	0	None	
4	5	a	a	29910.0	0	None	

4.4. Data Sets Integration

In [143...
```
# We will check if the completely match when merging the
train_df['Store'].nunique() == new_store_df['Store'].nunique()
```

Out[143... True

4.4.1 store, train, state_store

In [144...
```
# left join train_df with store_df on the 'store' column
total_df = pd.merge(train_df, new_store_df, on='Store', how='left').merge(store_
```

4.4.2. Weather

In [145…
```
integrated_df = total_df.copy()
```

In [146…
```
store_state_df['State'].unique()
```

Out[146…
```
array(['HE', 'TH', 'NW', 'BE', 'SN', 'SH', 'HB,NI', 'BY', 'BW', 'RP',
       'ST', 'HH'], dtype=object)
```

In [147…
```
weather_files = {
    'BE': 'Berlin.csv',
    'BW': 'Baden Württemberg.csv',
    'BY': 'Bayern.csv',
    'HB,NI': 'Niedersachsen.csv',
    'HE': 'Hessen.csv',
    'HH': 'Hamburg.csv',
    'NW': 'Nordrhein Westfalen.csv',
    'RP': 'Rheinland Pfalz.csv',
    'SH': 'Schleswig Holstein.csv',
    'SN': 'Sachsen.csv',
    'ST': 'Sachsen Anhalt.csv',
    'TH': 'Thüringen.csv'
}
```

In [148…
```
os.chdir(r'.\datasets')
```

In [149…
```
%pwd
```

Out[149…
```
'C:\\Drive D\\Python\\Rossmann\\datasets'
```

In [150…
```
# Dictionary to hold weather dataframes
weather_dfs = {}

for state, file in weather_files.items():
    # Load the weather data
    weather_dfs[state] = pd.read_csv(file, sep=';', parse_dates=['Date'])
```

In [151…
```
# Function to merge weather data for a given state
def merge_weather_data(df, state):
    weather_df = weather_dfs[state]
    return df.merge(weather_df[['Date', 'Max_TemperatureC', 'Min_Humidity']], on
```

In [152…
```
# Create an empty DataFrame to hold the merged results
merged_results = pd.DataFrame()

# Apply the function to each state's data
for state in weather_files.keys():
    state_df = integrated_df[integrated_df['State'] == state]

    if not state_df.empty:
        merged_df = merge_weather_data(state_df, state)
        merged_results = pd.concat([merged_results, merged_df], ignore_index=Tru

# Ensure that rows not matched by the weather data are included
unmatched_df = integrated_df[~integrated_df.index.isin(merged_results.index)]
final_df = pd.concat([merged_results, unmatched_df], ignore_index=True)
```

In [153…
```
final_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 844338 entries, 0 to 844337
Data columns (total 22 columns):
 #   Column                Non-Null Count   Dtype         
---  ------                --------------   -----         
 0   Store                 844338 non-null  int64         
 1   DayOfWeek             844338 non-null  int64         
 2   Date                  844338 non-null  datetime64[ns]
 3   Sales                 844338 non-null  int64         
 4   Customers             844338 non-null  int64         
 5   Promo                 844338 non-null  int64         
 6   StateHoliday          844338 non-null  object        
 7   SchoolHoliday         844338 non-null  int64         
 8   SalePerCustomer       844338 non-null  float64       
 9   Year                  844338 non-null  int32         
 10  Month                 844338 non-null  int32         
 11  StoreType             844338 non-null  object        
 12  Assortment            844338 non-null  object        
 13  CompetitionDistance   844338 non-null  float64       
 14  Promo2                844338 non-null  int64         
 15  PromoInterval         844338 non-null  object        
 16  log_InvSqrtCompDist   844338 non-null  float64       
 17  Promo2Duration        844338 non-null  float64       
 18  CompetitionDuration   844338 non-null  float64       
 19  State                 844338 non-null  object        
 20  Max_TemperatureC      844338 non-null  float64       
 21  Min_Humidity          844338 non-null  float64       
dtypes: datetime64[ns](1), float64(7), int32(2), int64(7), object(5)
memory usage: 135.3+ MB
```

In [154…
```
final_df.describe()
```

Out[154…

	Store	DayOfWeek	Date	Sales	Customers
count	844338.000000	844338.000000	844338	844338.000000	844338.000000
mean	558.421374	3.520350	2014-04-11 01:08:38.729702912	6955.959134	762.777166
min	1.000000	1.000000	2013-01-01 00:00:00	46.000000	8.000000
25%	280.000000	2.000000	2013-08-16 00:00:00	4859.000000	519.000000
50%	558.000000	3.000000	2014-03-31 00:00:00	6369.000000	676.000000
75%	837.000000	5.000000	2014-12-11 00:00:00	8360.000000	893.000000
max	1115.000000	7.000000	2015-07-31 00:00:00	41551.000000	7388.000000
std	321.730861	1.723712	NaN	3103.815515	401.194153

In [155…
```
# Free memory
del weather_dfs, weather_files
```

4.5. Categorical Encoding

4.5.1. Ordinal Encoding

In [156…
```
# Assortment is an ordinal variable, and to reduce the complexity we will consid
final_df.loc[final_df['Assortment']=='a', 'Assortment'] = 0
final_df.loc[final_df['Assortment']=='b', 'Assortment'] = 1
final_df.loc[final_df['Assortment']=='c', 'Assortment'] = 2
```

In [157…
```
final_df['Assortment'] = final_df['Assortment'].astype('int8')
```

If we used label (integer) encoding, we don't have any control on the assigned values), whereas here the variables is ordinal.

4.5.2. Data Type Transformation

Data type transformation is not related to categorical encoding, however, this is the best place to convert the dtypes.

4.5.2.1. Range of basic dtypes

4.5.2.1.1. Integers

In [158…
```
np.iinfo('int8')
```

Out[158…
```
iinfo(min=-128, max=127, dtype=int8)
```

In [159…
```
np.iinfo('int16')
```

Out[159…
```
iinfo(min=-32768, max=32767, dtype=int16)
```

In [160…
```
np.iinfo('int32')
```

Out[160…
```
iinfo(min=-2147483648, max=2147483647, dtype=int32)
```

In [161…
```
np.iinfo('int64')
```

Out[161…
```
iinfo(min=-9223372036854775808, max=9223372036854775807, dtype=int64)
```

4.5.2.1.2. Floating points

In [162…
```
np.finfo('float16')
```

Out[162…
```
finfo(resolution=0.001, min=-6.55040e+04, max=6.55040e+04, dtype=float16)
```

In [163…
```
np.finfo('float32')
```

Out[163…
```
finfo(resolution=1e-06, min=-3.4028235e+38, max=3.4028235e+38, dtype=float32)
```

In [164…
```
np.finfo('float64')
```

Out[164…
```
finfo(resolution=1e-15, min=-1.7976931348623157e+308, max=1.7976931348623157e+308, dtype=float64)
```

4.5.2.2. Features Range

Since 'Store' is just an index and has no meaningful order, we exclude it from the prediction.

Unless a predictive model is developed for each store, it won't make sense to include 'Store' in the prediction.

In [165…
```
final_df.drop('Store', axis=1, inplace=True)
```

4.5.2.3. Dtype Conversion

In [166…
```
# listing the dtype and the range of valaues for non-boolean variables.
df = final_df
for col in df.columns:
    if df[col].dtype != bool:
        print(f'{col}: f{df[col].dtype}')
        print(f'[{df[col].min()}, {df[col].max()}]')
        print('_____________________________________')
```

```
DayOfWeek: fint64
[1, 7]

------------------------------
Date: fdatetime64[ns]
[2013-01-01 00:00:00, 2015-07-31 00:00:00]

------------------------------
Sales: fint64
[46, 41551]

------------------------------
Customers: fint64
[8, 7388]

------------------------------
Promo: fint64
[0, 1]

------------------------------
StateHoliday: fobject
[0, c]

------------------------------
SchoolHoliday: fint64
[0, 1]

------------------------------
SalePerCustomer: ffloat64
[2.7490747594374536, 64.95785440613027]

------------------------------
Year: fint32
[2013, 2015]

------------------------------
Month: fint32
[1, 12]

------------------------------
StoreType: fobject
[a, d]

------------------------------
Assortment: fint8
[0, 2]

------------------------------
CompetitionDistance: ffloat64
[20.0, 75860.0]

------------------------------
Promo2: fint64
[0, 1]

------------------------------
PromoInterval: fobject
[Feb,May,Aug,Nov, None]

------------------------------
log_InvSqrtCompDist: ffloat64
[-5.618322407621445, -1.4978661367769954]

------------------------------
Promo2Duration: ffloat64
[0.0, 645.0]

------------------------------
CompetitionDuration: ffloat64
[76.0, 1463.0]

------------------------------
State: fobject
[BE, TH]

------------------------------
Max_TemperatureC: ffloat64
[-11.0, 38.0]

------------------------------
```

```
Min_Humidity: ffloat64
[4.0, 100.0]
```

In [167…
```
# We have assigned 'int8' to 'Assortment' before.
final_df = final_df.astype({
    'Promo': 'bool',
    'Promo2': 'bool',
    'SchoolHoliday': 'bool',
    'Customers': 'int16',
    'CompetitionDuration': 'int16',
    'Promo2Duration': 'int16',
    'Sales': 'int32',
    'CompetitionDistance': 'int32',
    'log_InvSqrtCompDist': 'float32',
    'Max_TemperatureC': 'float16',
    'Min_Humidity': 'float16',
})
```

In [168…
```
final_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 844338 entries, 0 to 844337
Data columns (total 21 columns):
 #   Column               Non-Null Count   Dtype
---  ------               --------------   -----
 0   DayOfWeek            844338 non-null  int64
 1   Date                 844338 non-null  datetime64[ns]
 2   Sales                844338 non-null  int32
 3   Customers            844338 non-null  int16
 4   Promo                844338 non-null  bool
 5   StateHoliday         844338 non-null  object
 6   SchoolHoliday        844338 non-null  bool
 7   SalePerCustomer      844338 non-null  float64
 8   Year                 844338 non-null  int32
 9   Month                844338 non-null  int32
 10  StoreType            844338 non-null  object
 11  Assortment           844338 non-null  int8
 12  CompetitionDistance  844338 non-null  int32
 13  Promo2               844338 non-null  bool
 14  PromoInterval        844338 non-null  object
 15  log_InvSqrtCompDist  844338 non-null  float32
 16  Promo2Duration       844338 non-null  int16
 17  CompetitionDuration  844338 non-null  int16
 18  State                844338 non-null  object
 19  Max_TemperatureC     844338 non-null  float16
 20  Min_Humidity         844338 non-null  float16
dtypes: bool(3), datetime64[ns](1), float16(2), float32(1), float64(1), int16(3),
int32(4), int64(1), int8(1), object(4)
memory usage: 72.5+ MB
```

We observe that after the transformation the required memory requirement decreases to almost half.

4.5.3. One-Hot Encoding

In [169…
```
# We preserve the original data frame for plotting purposes.
new_total_df = final_df.copy()
```

In [170…
```
# SateHoliday obviously depends on the State.
# This is also the case for SchoolHoliday, nevertheless, we ignore it to avoid t
new_total_df['StateHoliday_State'] = new_total_df['StateHoliday'] + '_' + new_tc
```

In [171…
```
# We have to include the basic effects.
new_total_df = pd.get_dummies(new_total_df, columns=['StateHoliday', 'State', 'S
```

Although the days of week have an order, the 1st and 5th days might have similar behaviour. The same is true for 'Month'.

Therefore, we convert them to dummy variables.

If we want to be more stringent, we should also include year as a categorical variable. (we pass!)

In [172…
```
# Converting nominal variables to dummy variables.
new_total_df = pd.get_dummies(new_total_df, columns=['PromoInterval', 'StoreType
```

In [173…
```
pd.set_option('display.max_info_columns', 200)
```

In [174…
```
new_total_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 844338 entries, 0 to 844337
Data columns (total 100 columns):
 #   Column                          Non-Null Count   Dtype
---  ------                          --------------   -----
 0   Date                            844338 non-null  datetime64[ns]
 1   Sales                           844338 non-null  int32
 2   Customers                       844338 non-null  int16
 3   Promo                           844338 non-null  bool
 4   SchoolHoliday                   844338 non-null  bool
 5   SalePerCustomer                 844338 non-null  float64
 6   Year                            844338 non-null  int32
 7   Assortment                      844338 non-null  int8
 8   CompetitionDistance             844338 non-null  int32
 9   Promo2                          844338 non-null  bool
 10  log_InvSqrtCompDist             844338 non-null  float32
 11  Promo2Duration                  844338 non-null  int16
 12  CompetitionDuration             844338 non-null  int16
 13  Max_TemperatureC                844338 non-null  float16
 14  Min_Humidity                    844338 non-null  float16
 15  StateHoliday_0                  844338 non-null  bool
 16  StateHoliday_a                  844338 non-null  bool
 17  StateHoliday_b                  844338 non-null  bool
 18  StateHoliday_c                  844338 non-null  bool
 19  State_BE                        844338 non-null  bool
 20  State_BW                        844338 non-null  bool
 21  State_BY                        844338 non-null  bool
 22  State_HB,NI                     844338 non-null  bool
 23  State_HE                        844338 non-null  bool
 24  State_HH                        844338 non-null  bool
 25  State_NW                        844338 non-null  bool
 26  State_RP                        844338 non-null  bool
 27  State_SH                        844338 non-null  bool
 28  State_SN                        844338 non-null  bool
 29  State_ST                        844338 non-null  bool
 30  State_TH                        844338 non-null  bool
 31  StateHoliday_State_0_BE         844338 non-null  bool
 32  StateHoliday_State_0_BW         844338 non-null  bool
 33  StateHoliday_State_0_BY         844338 non-null  bool
 34  StateHoliday_State_0_HB,NI      844338 non-null  bool
 35  StateHoliday_State_0_HE         844338 non-null  bool
 36  StateHoliday_State_0_HH         844338 non-null  bool
 37  StateHoliday_State_0_NW         844338 non-null  bool
 38  StateHoliday_State_0_RP         844338 non-null  bool
 39  StateHoliday_State_0_SH         844338 non-null  bool
 40  StateHoliday_State_0_SN         844338 non-null  bool
 41  StateHoliday_State_0_ST         844338 non-null  bool
 42  StateHoliday_State_0_TH         844338 non-null  bool
 43  StateHoliday_State_a_BE         844338 non-null  bool
 44  StateHoliday_State_a_BW         844338 non-null  bool
 45  StateHoliday_State_a_BY         844338 non-null  bool
 46  StateHoliday_State_a_HB,NI      844338 non-null  bool
 47  StateHoliday_State_a_HE         844338 non-null  bool
 48  StateHoliday_State_a_HH         844338 non-null  bool
 49  StateHoliday_State_a_NW         844338 non-null  bool
 50  StateHoliday_State_a_RP         844338 non-null  bool
 51  StateHoliday_State_a_SH         844338 non-null  bool
 52  StateHoliday_State_a_SN         844338 non-null  bool
 53  StateHoliday_State_a_ST         844338 non-null  bool
 54  StateHoliday_State_a_TH         844338 non-null  bool
```

```
 55  StateHoliday_State_b_BE          844338 non-null  bool
 56  StateHoliday_State_b_BW          844338 non-null  bool
 57  StateHoliday_State_b_BY          844338 non-null  bool
 58  StateHoliday_State_b_HB,NI       844338 non-null  bool
 59  StateHoliday_State_b_HE          844338 non-null  bool
 60  StateHoliday_State_b_HH          844338 non-null  bool
 61  StateHoliday_State_b_NW          844338 non-null  bool
 62  StateHoliday_State_b_RP          844338 non-null  bool
 63  StateHoliday_State_b_SH          844338 non-null  bool
 64  StateHoliday_State_c_BE          844338 non-null  bool
 65  StateHoliday_State_c_BW          844338 non-null  bool
 66  StateHoliday_State_c_BY          844338 non-null  bool
 67  StateHoliday_State_c_HB,NI       844338 non-null  bool
 68  StateHoliday_State_c_HE          844338 non-null  bool
 69  StateHoliday_State_c_HH          844338 non-null  bool
 70  StateHoliday_State_c_NW          844338 non-null  bool
 71  StateHoliday_State_c_RP          844338 non-null  bool
 72  StateHoliday_State_c_SH          844338 non-null  bool
 73  PromoInterval_Feb,May,Aug,Nov    844338 non-null  bool
 74  PromoInterval_Jan,Apr,Jul,Oct    844338 non-null  bool
 75  PromoInterval_Mar,Jun,Sept,Dec   844338 non-null  bool
 76  PromoInterval_None               844338 non-null  bool
 77  StoreType_a                      844338 non-null  bool
 78  StoreType_b                      844338 non-null  bool
 79  StoreType_c                      844338 non-null  bool
 80  StoreType_d                      844338 non-null  bool
 81  DayOfWeek_1                      844338 non-null  bool
 82  DayOfWeek_2                      844338 non-null  bool
 83  DayOfWeek_3                      844338 non-null  bool
 84  DayOfWeek_4                      844338 non-null  bool
 85  DayOfWeek_5                      844338 non-null  bool
 86  DayOfWeek_6                      844338 non-null  bool
 87  DayOfWeek_7                      844338 non-null  bool
 88  Month_1                          844338 non-null  bool
 89  Month_2                          844338 non-null  bool
 90  Month_3                          844338 non-null  bool
 91  Month_4                          844338 non-null  bool
 92  Month_5                          844338 non-null  bool
 93  Month_6                          844338 non-null  bool
 94  Month_7                          844338 non-null  bool
 95  Month_8                          844338 non-null  bool
 96  Month_9                          844338 non-null  bool
 97  Month_10                         844338 non-null  bool
 98  Month_11                         844338 non-null  bool
 99  Month_12                         844338 non-null  bool
dtypes: bool(88), datetime64[ns](1), float16(2), float32(1), float64(1), int16
(3), int32(3), int8(1)
memory usage: 105.5 MB
```

4.7. Export Data to CSV

In [175…
```
%pwd
```

Out[175…
```
'C:\\Drive D\\Python\\Rossmann\\datasets'
```

In [176…
```
new_store_df.to_csv('store_df.csv', index=False)
```

In [177…
```
test_df.to_csv('test_df.csv', index=False)
```

In [178…
```
train_df.to_csv('train_df.csv', index=False)
```

In [179…
```
total_df.to_csv('total_df.csv', index=False)
```

In [180…
```
final_df.to_csv('final_df.csv', index=False)
```

In [181…
```
new_total_df.to_csv('new_total_df.csv', index=False)
```

In [182…
```
# Free some memory!
del test_df, train_df, new_store_df, store_df, total_df
```

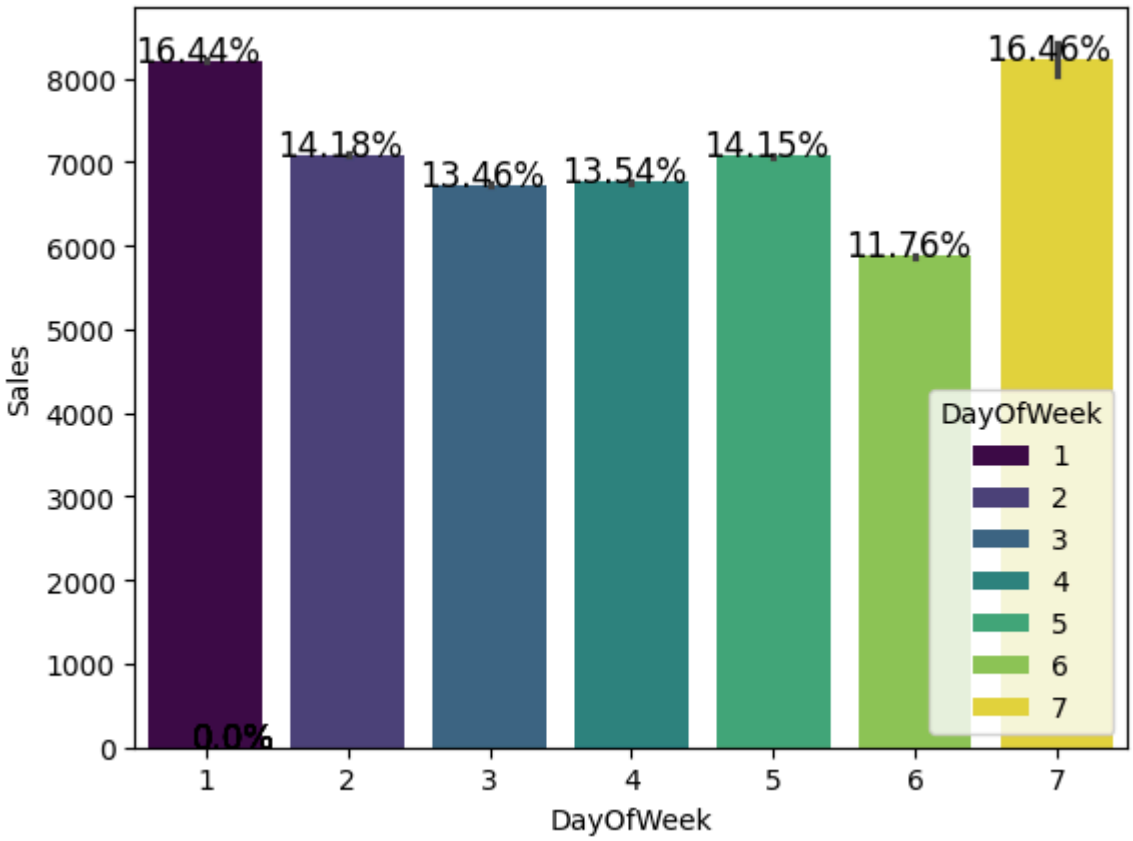
5. Exploratory Data Analysis

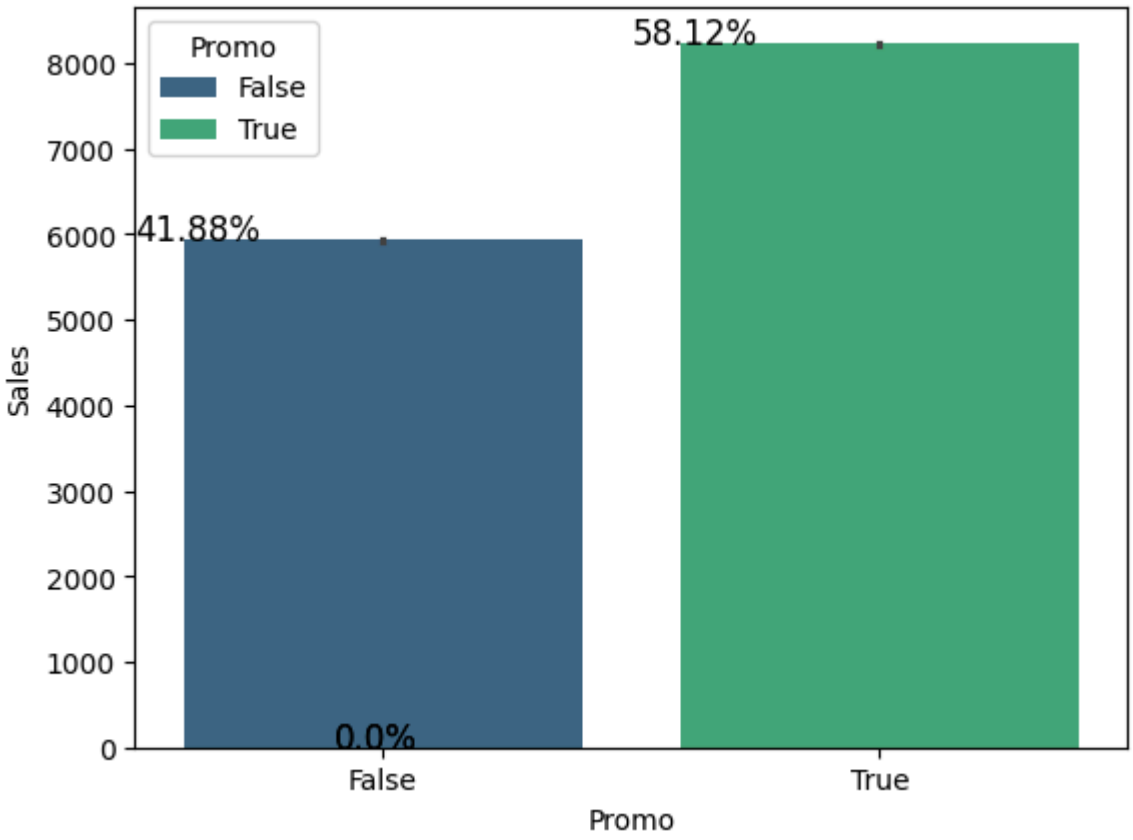
In this section we visualise Sales, Customers, and SalePerCustomer against the other variables to discover the trends and identify the key factors.

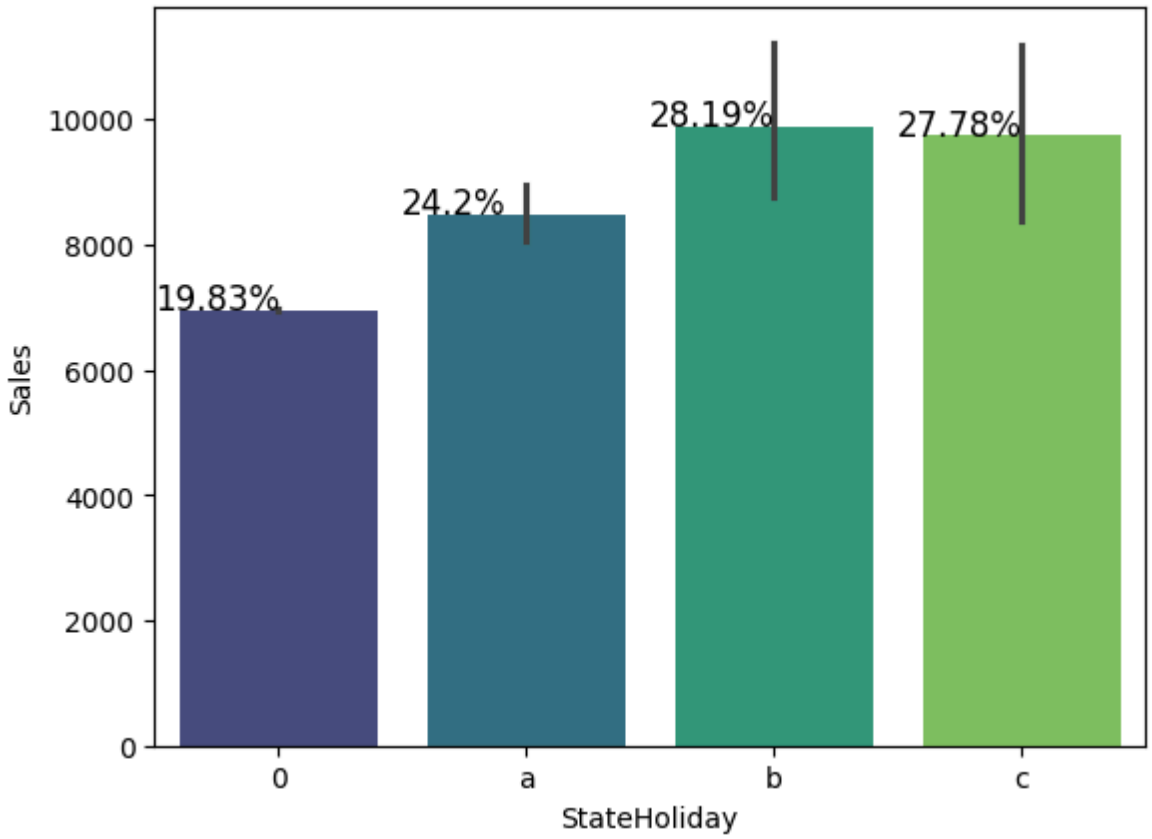
5.1. Effect of categorical variables on sales

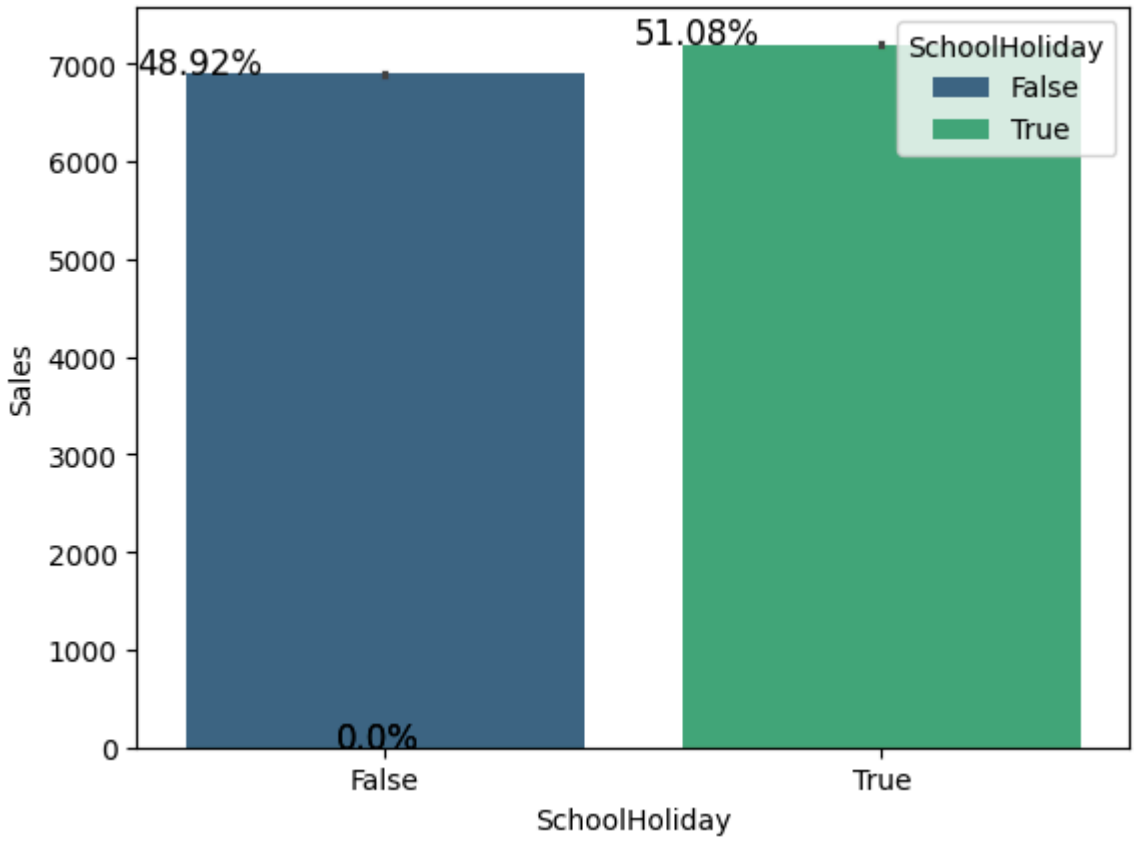
In [183…
```
# List categorical variables
categorical_variables = [
    'DayOfWeek',
    'Promo',
    'StateHoliday',
    'SchoolHoliday',
    'StoreType',
    'Assortment',
    'Promo2',
    'PromoInterval',
    'State',
]
```

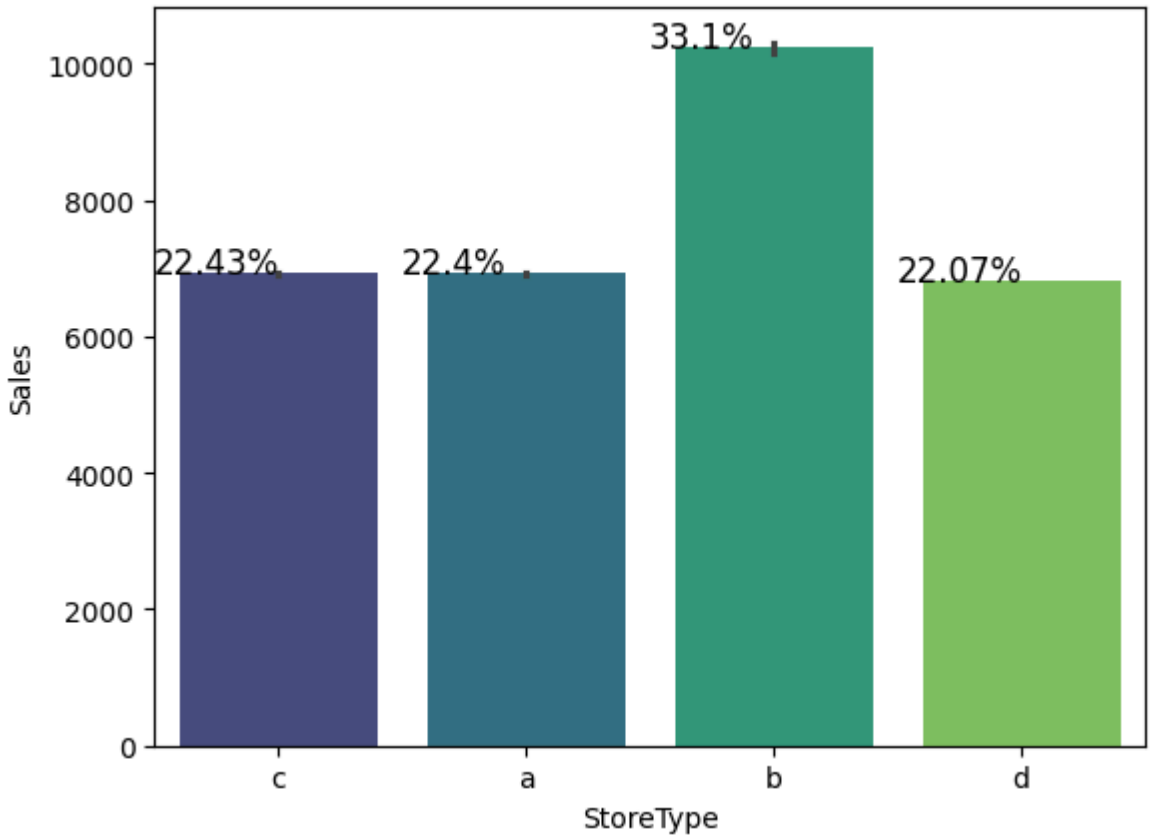
In [184…
```
# total_df, for which the categorical variables were not transfomred to dummy va
for value in categorical_variables:
    ax = sns.barplot(x=final_df[value], y=final_df['Sales'], hue=final_df[value]
    totals = []
    # for each patch in the barplot ax
    for i in ax.patches:
        # append height for each patch
        totals.append(i.get_height())

    # sum of each patch height for a plot
    total = sum(totals)

    for i in ax.patches:
        # text position and formula for percentage
        ax.text(
            i.get_x() - 0.1, i.get_height() + 0.5, str(round((i.get_height()/tot
        )
    plt.show()
```

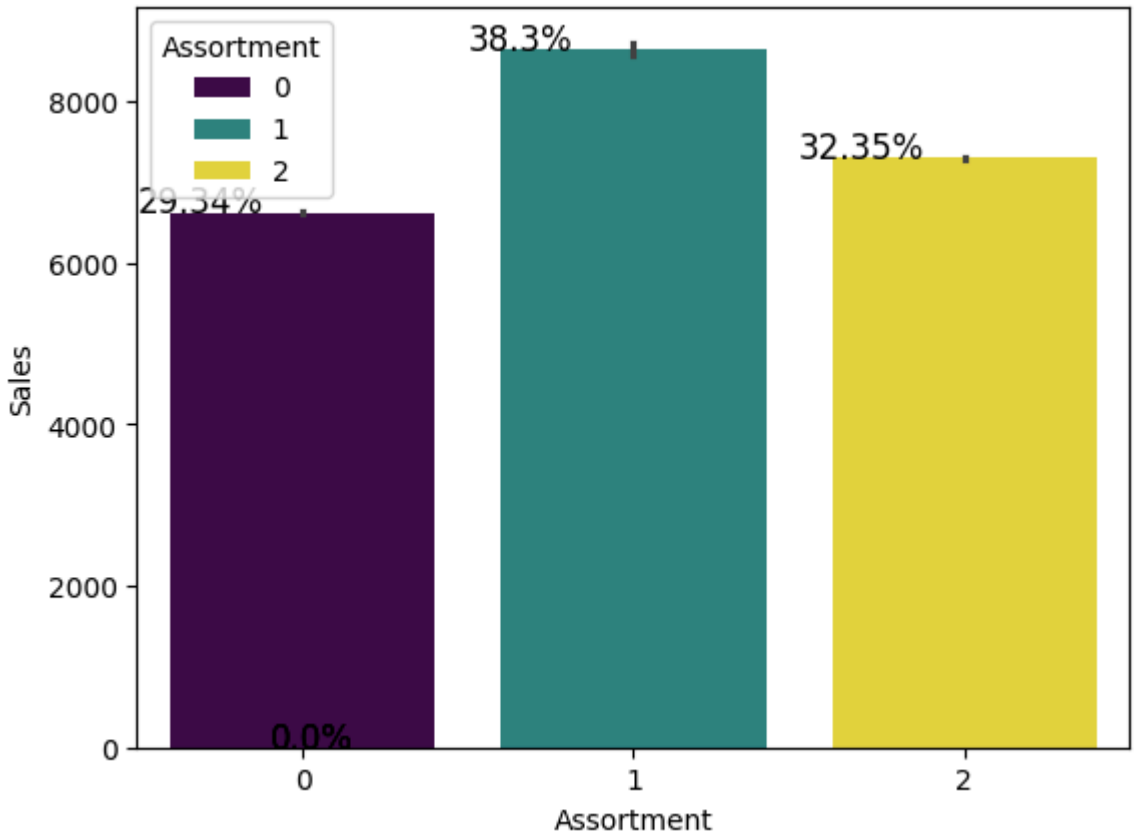

16.44%
14.18%
13.46%
13.54%
14.15%
11.76%
16.46%
0.0%
Sales
DayOfWeek
DayOfWeek
1
2
3
4
5
6
7

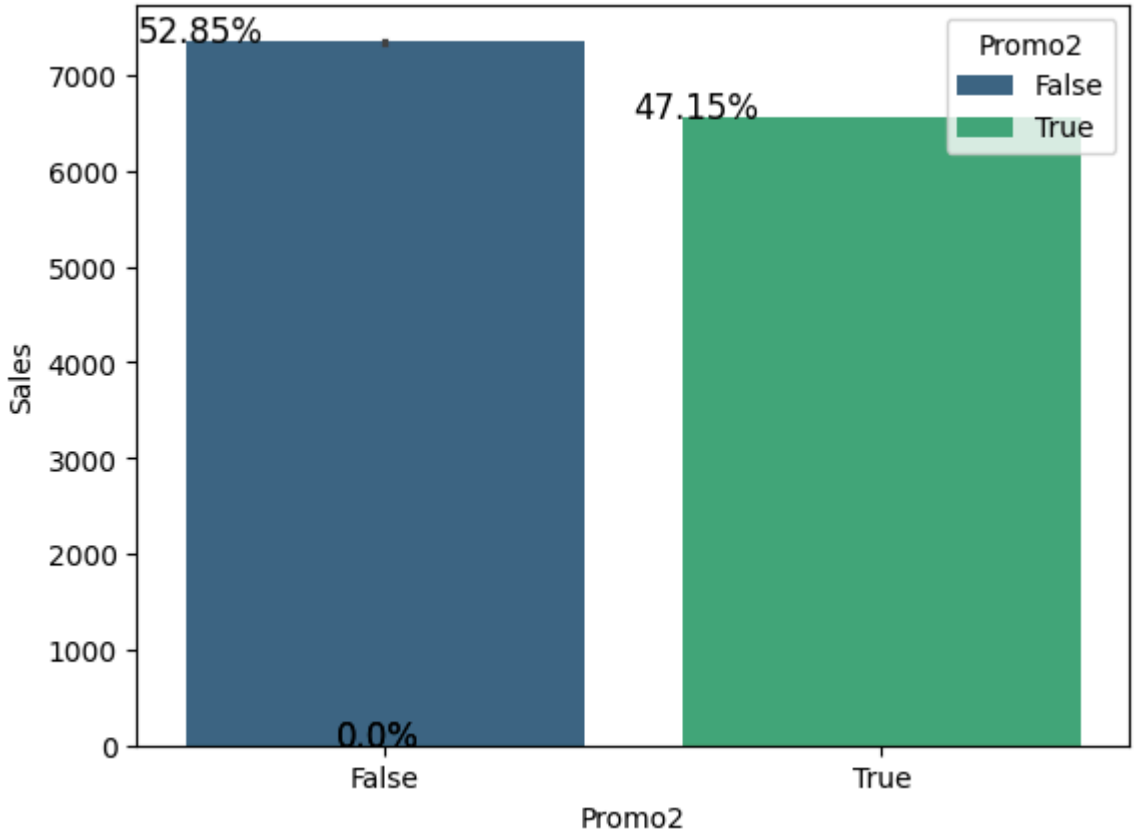

Promo
False
True
58.12%
41.88%
0.0%
Sales
False
True
Promo

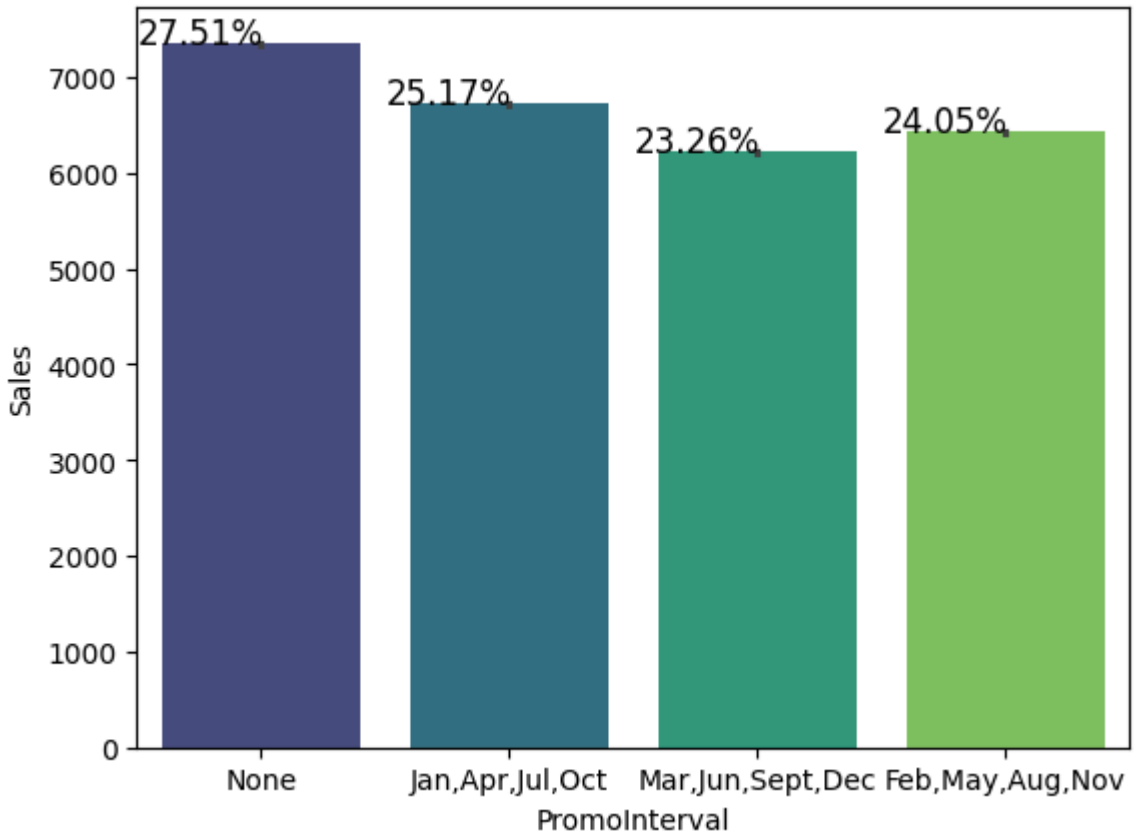

10000
8000
6000
4000
2000
0
Sales
19.83%
24.2%
28.19%
27.78%
0
a
b
c
StateHoliday

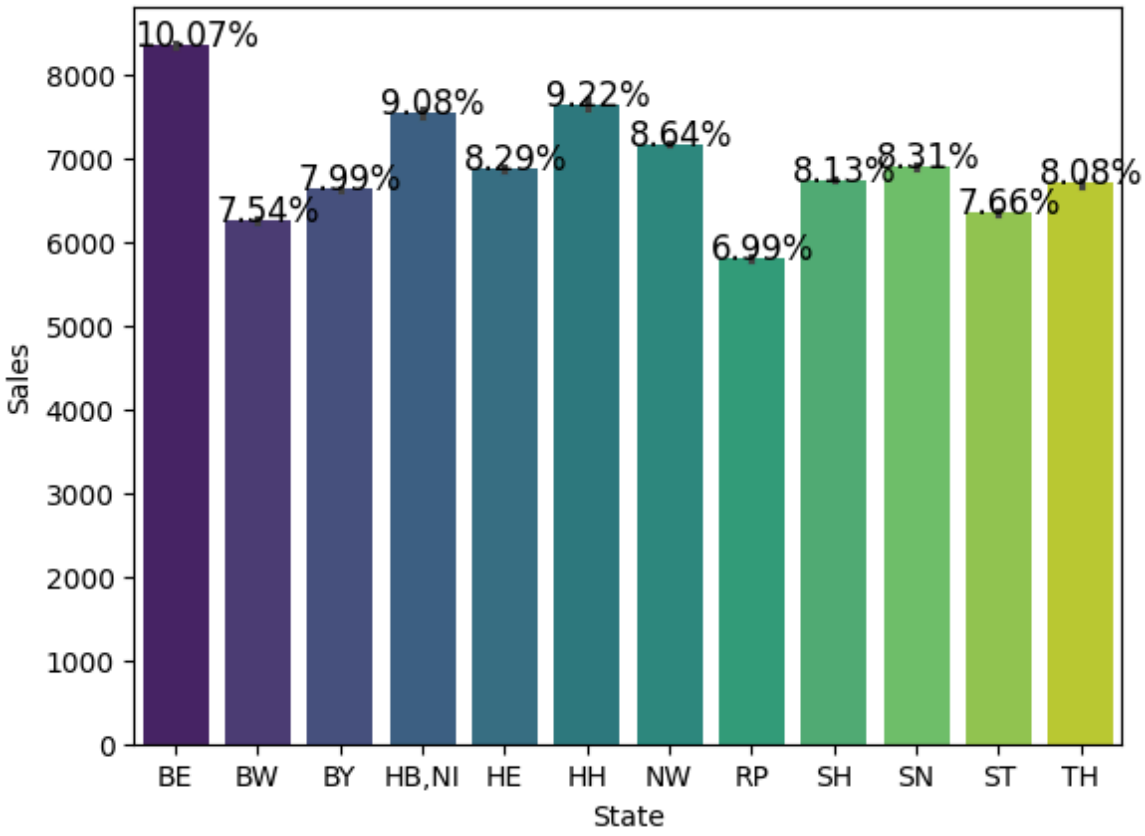

7000
6000
5000
4000
3000
2000
1000
0
Sales
48.92%
0.0%
51.08%
SchoolHoliday
False
True
False
True
SchoolHoliday


33.1%
22.43%
22.4%
22.07%
Sales
StoreType
c
a
b
d


Assortment
0
1
2
38.3%
32.35%
29.34%
0.0%
Sales
Assortment


52.85%
47.15%
0.0%
Promo2
False
True
Sales
0
1000
2000
3000
4000
5000
6000
7000
False
True
Promo2


27.51%
25.17%
23.26%
24.05%
Sales
0
1000
2000
3000
4000
5000
6000
7000
None
Jan,Apr,Jul,Oct
Mar,Jun,Sept,Dec
Feb,May,Aug,Nov
PromoInterval



The correct way to compare the values is via hypotheses tests.

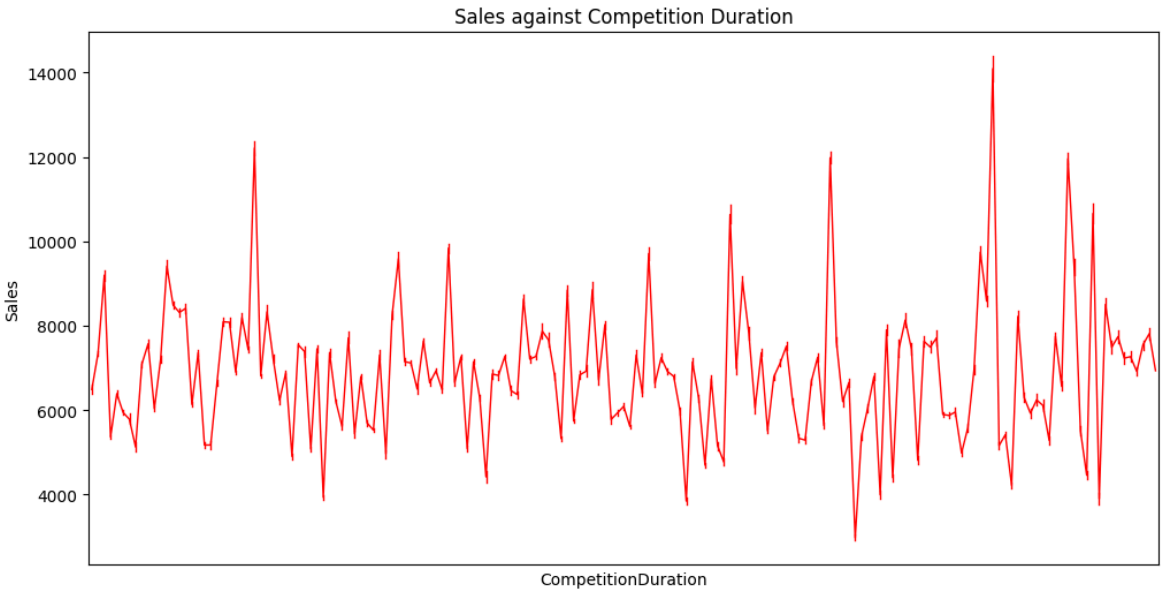
ANOVA for multiple.

Bar chart is used because it makes it easy to see the differences.

1. There were more sales on Mondays, probably because shops generally remain closed on Sundays.
2. It could be seen that the Promo leads to more sales.
3. Normally all stores, with a few exceptions, are closed on state holidays. Note that all schools are closed on public holidays and weekends. a) public holiday, b) easter holiday, c) Christmas, 0) None !!!!!! these don't coincide with the numerical values! Lowest sales are seen on state holidays, especially on Christmas.
4. More stores were open on school holidays than on state holidays, hence there were more sales compared to state holidays.
5. On average, stores of type B had the highest sales.
6. Highest average sales were seen with assortment level B, which corresponds to extra.
7. With promo2, there were slightly more sales, whihc is because many stores are participating in promo.
8. Berlin has the highest sale among the states.
9. Saturdays (corresponding to the 6th day of the week) have the lowest sales. Therefore, they are the best choice for refurbishment.

5.2. Effect of Competition Duration on Sales

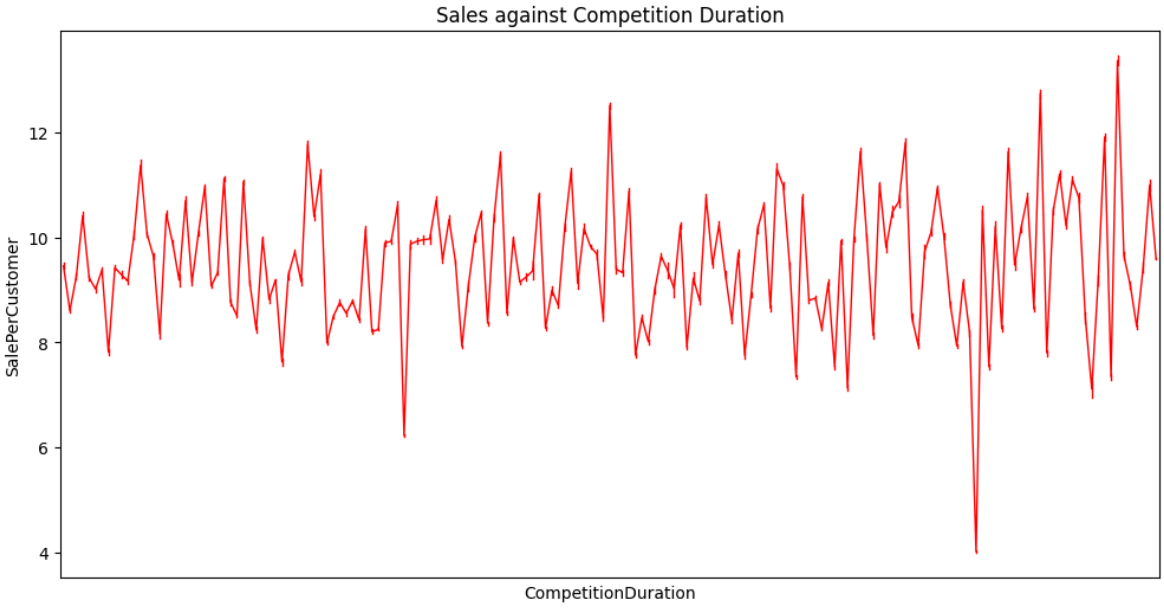
In [185…
```
plt.figure(figsize=(12, 6))
sns.pointplot(x='CompetitionDuration', y='Sales', data=final_df, color='red', ma
plt.xticks([])
plt.title('Sales against Competition Duration')
plt.show()
```



As it turns out 'CompetitionDuration' doesn't contain any trend by itself. Maybe it does if we include other inetrmediary or moderator variables (conisideirng the causal structure).

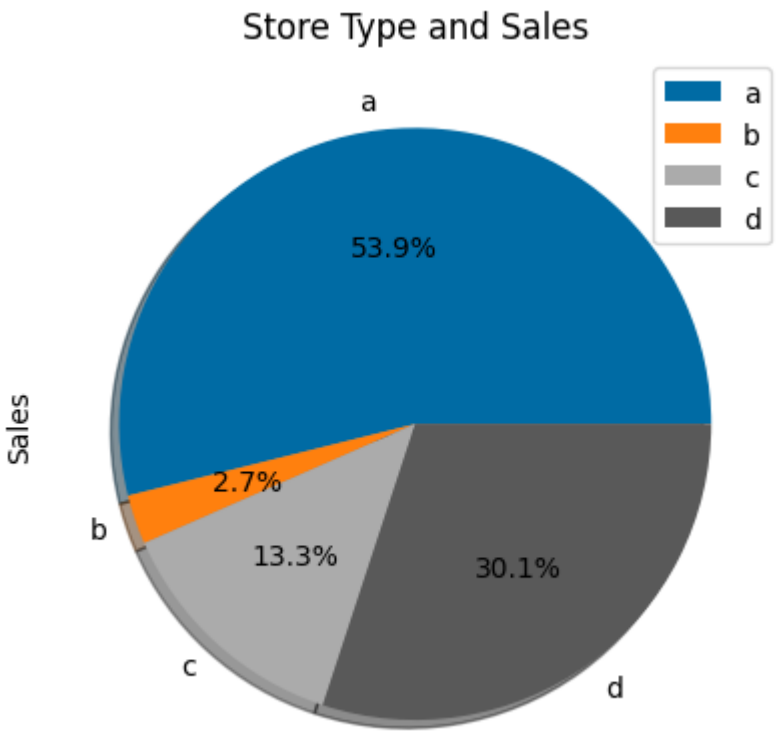
So individually, with the increase or decrease of competition duration, we can't expect any consistent change in sales.

In [186…
```
plt.figure(figsize=(12, 6))
sns.pointplot(x='CompetitionDuration', y='SalePerCustomer', data=final_df, color
plt.xticks([])
plt.title('Sales against Competition Duration')
plt.show()
```

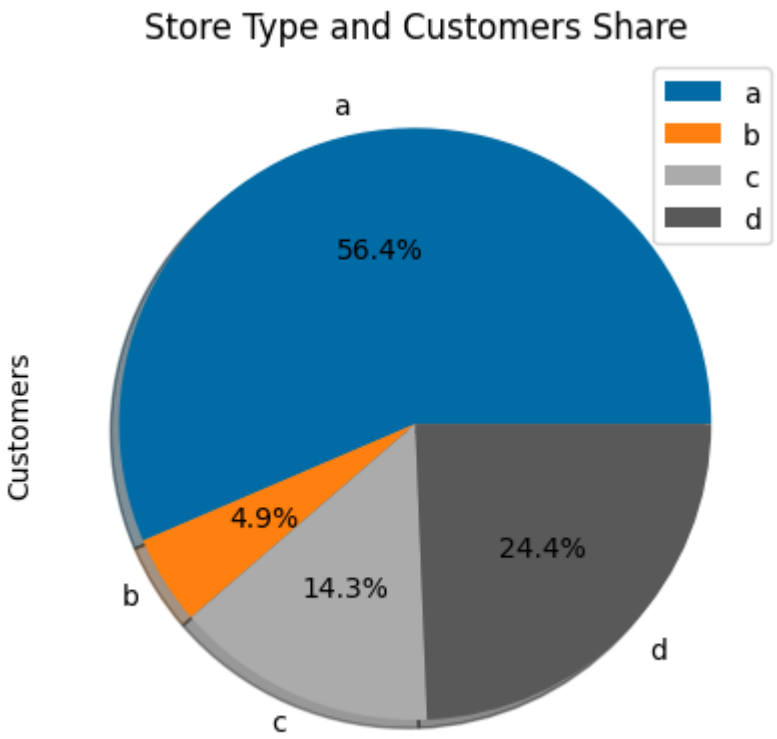


5.3. Share of Different Store Types in Sales, Customers, CustomerPerSale, and number of stores

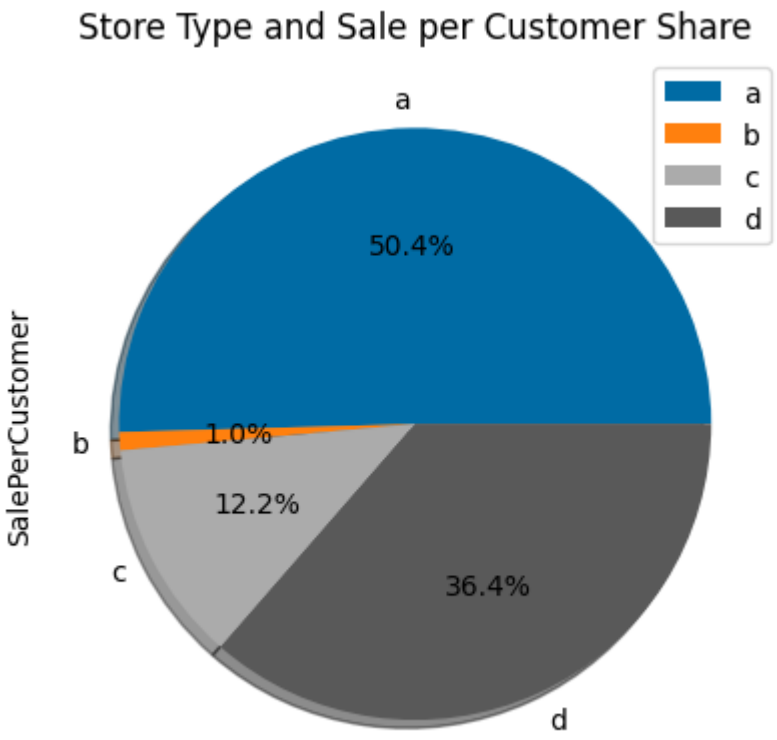
In [187…
```
final_df.groupby('StoreType')['Sales'].sum().plot.pie(title='Store Type and Sale
plt.show()
```



In [188…
```
final_df.groupby('StoreType')['Customers'].sum().plot.pie(title='Store Type and
plt.show()
```



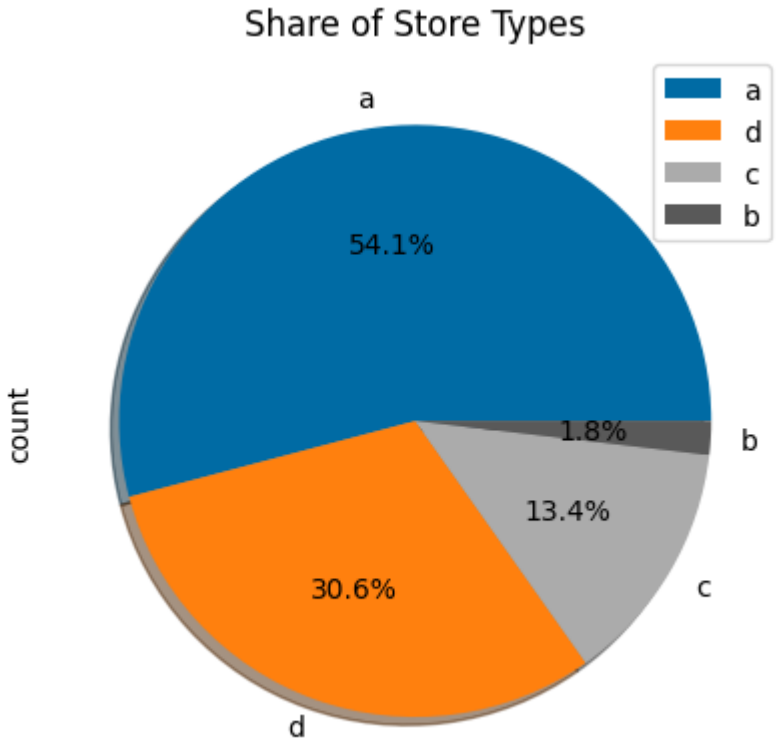
In [189…
```
final_df.groupby('StoreType')['SalePerCustomer'].sum().plot.pie(title='Store Typ
plt.show()
```



The three above are related and almost show the same thing.

The below chart shows the proportion of the store types (already discernible in the above barchart)

In [190…
```
final_df['StoreType'].value_counts().plot.pie(title='Share of Store Types', lege
plt.show()
```



A pie chart is a suitable way for summarising a set of nominal data or displaying the different values of a given variable.

- The majority of the stores are physical stores.
- There is a small percentage of online stores.
- There is a small percentage of other types of stores.

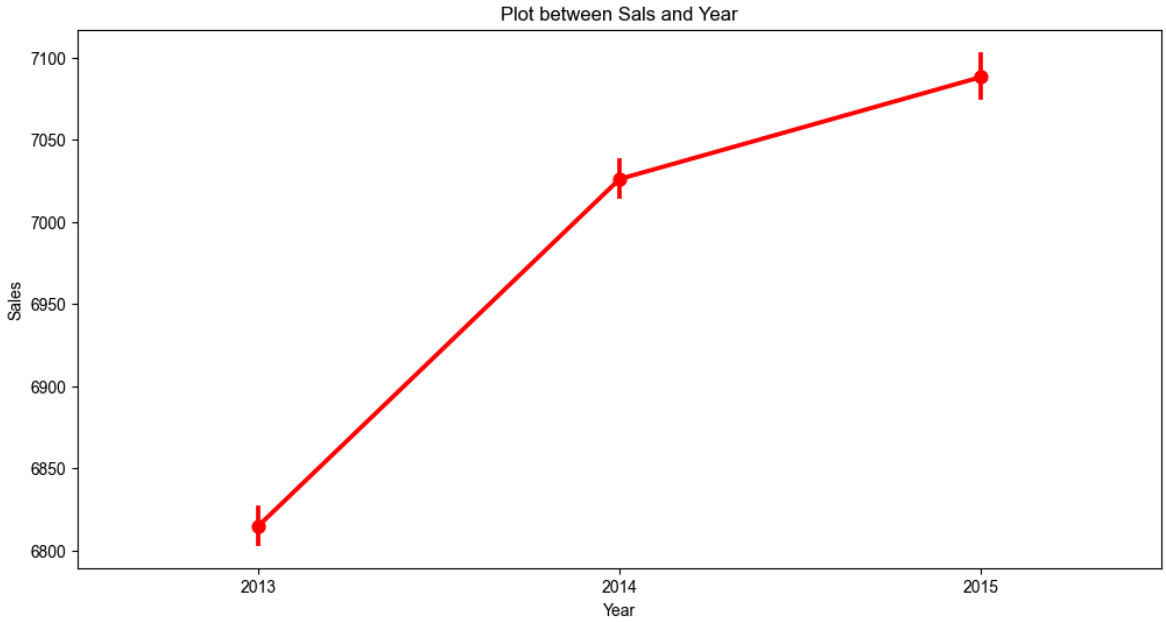
Upon further exploration it can be observed that the highest sales belonged to the store type a due to the high number of type a stores in the data set. Stores type a and c had a similar kind of salesand customer share.

Store type b has the highest average sales, and a reason for that could be that all three kinds of assortment strategies are involved in that category.

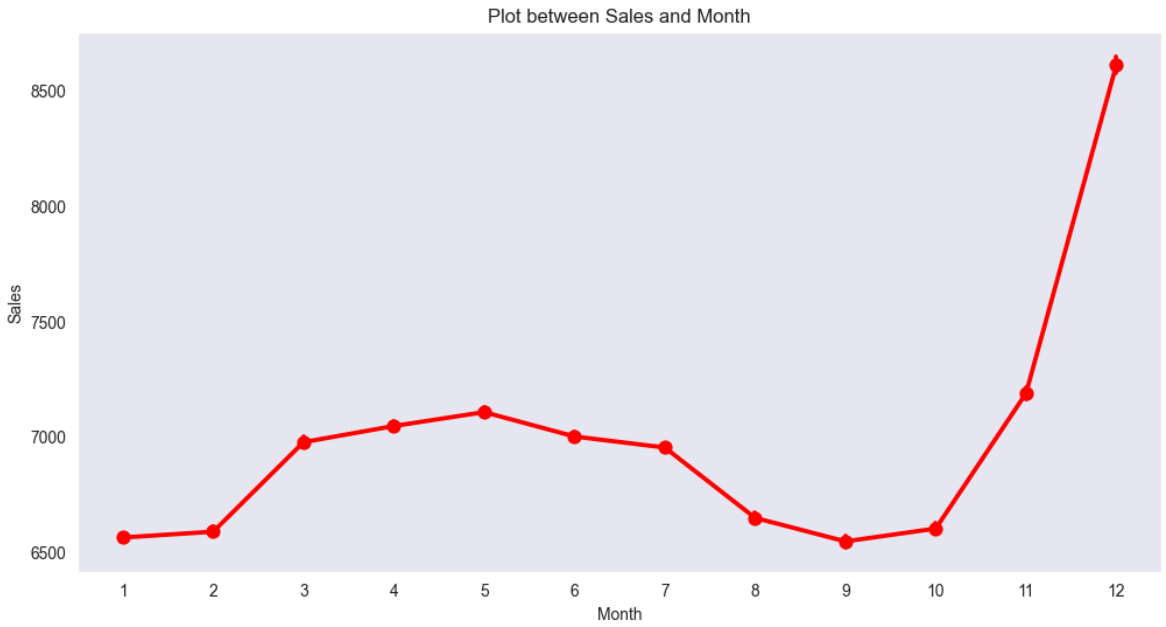
These insights can be used to help the business in making the decision on where to open new stores considering the existing store types, and what products or services to offer.

5.4. Sales Trend, Seasonality (Oscillation), and Noise Patterns

In [191…
```
plt.figure(figsize=(12, 6))
sns.pointplot(x='Year', y='Sales', data=final_df, color='red');
sns.set_style('dark')
plt.title('Plot between Sals and Year')
plt.show()
```

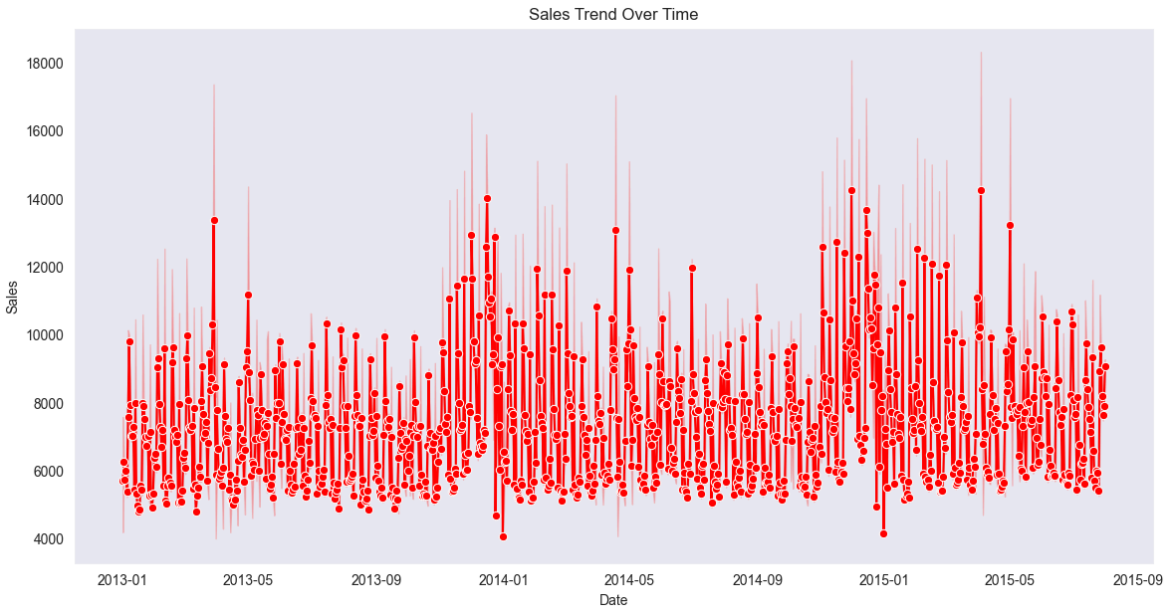


In [192…
```
plt.figure(figsize=(12, 6))
sns.pointplot(x='Month', y='Sales', data=final_df, color='red');
sns.set_style('dark')
plt.title('Plot between Sales and Month')
plt.show()
```



In [193…
```
df = final_df.copy()
df.set_index('Date', inplace=True)
```

In [194…
```
# Create the plot
plt.figure(figsize=(14, 7))
sns.lineplot(x=df.index, y='Sales', data=df, marker='o', color='red')

# Optional: Add a title and labels
plt.title('Sales Trend Over Time')
plt.xlabel('Date')
plt.ylabel('Sales')

# Show the plot
plt.show()
```



In [195…
```
# Free memory
del df
```

We can see that sale increases in the months 3-7 and 11-12.

This result coincides with the increase of sale in the Easter and Christmas holidays.

The important catch from these three charts is the existence of an increasing trend, oscillation, and noise. These should be taken into consideration if we want to use the auto-correlation in the data. Nevertheless, as the analysis shows, we can model the sale and customers with an event-driven approach, and avoid the costly (at evaluation time) nonparametric auto-correlation methods.

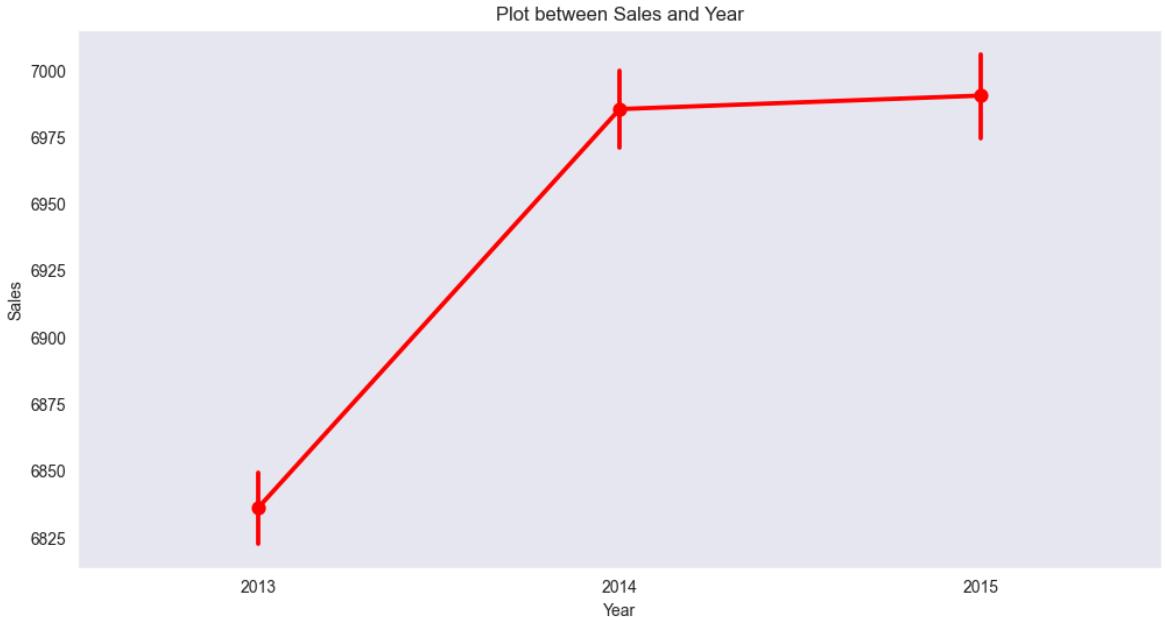
5.5 Physical vs Online Sales trend

In [196…
```
final_df.StoreType.unique()
```

Out[196…
```
array(['c', 'a', 'b', 'd'], dtype=object)
```

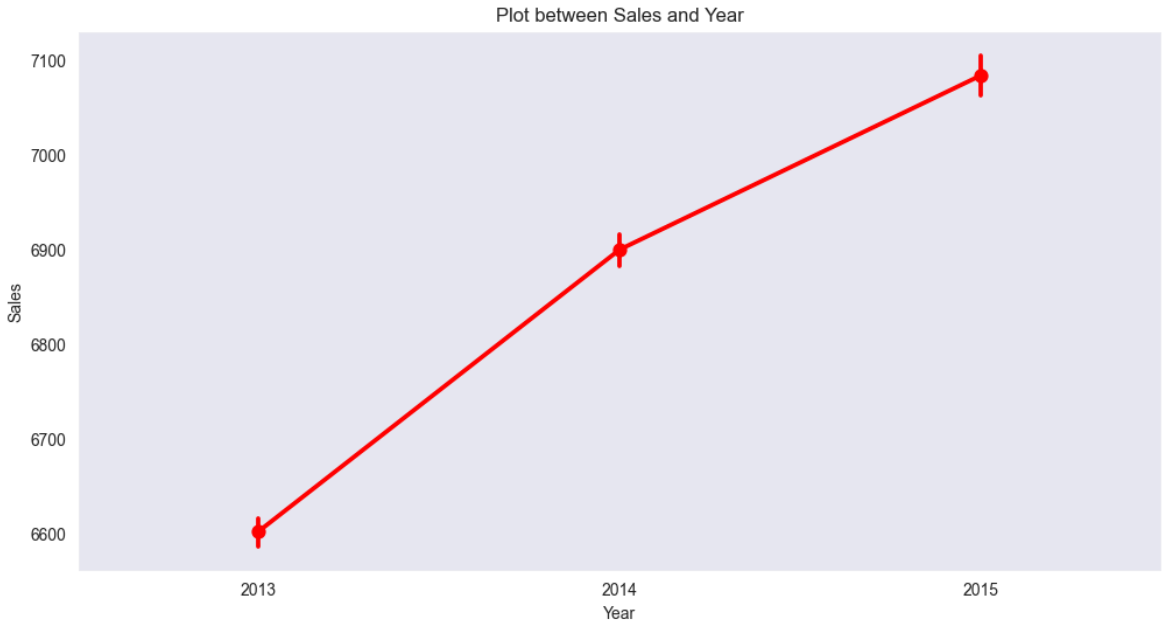
In [197…
```
total_physical = final_df.loc[(final_df['StoreType']=='a') | (final_df['StoreTyp
```

In [198…
```
plt.figure(figsize=(12, 6))
sns.pointplot(x='Year', y='Sales', data=total_physical, color='red');
sns.set_style('dark')
plt.title('Plot between Sales and Year')
plt.show()
```



In [199…
```
total_online = final_df.loc[(final_df['StoreType']=='d'), ['Year','Sales']]
```

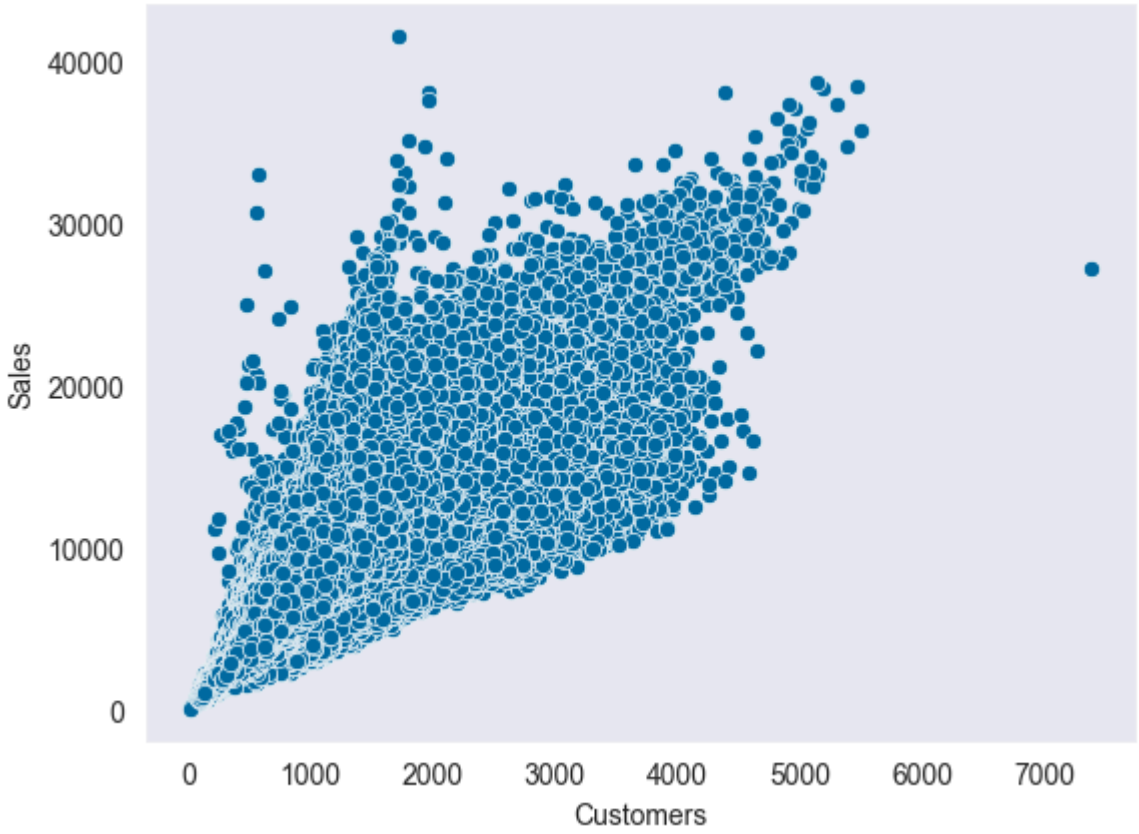
In [200…
```
plt.figure(figsize=(12, 6))
sns.pointplot(x='Year', y='Sales', data=total_online, color='red');
sns.set_style('dark')
plt.title('Plot between Sales and Year')
plt.show()
```



We observe that the status of sales for physical stores becomes steady, and for online shops, growing. If this trend continues, considering the heavy dependence of the business on physical stores, it might fall behind in the competition with its rivals.

5.6. Cutomers v.s. Sales and Independence of SalePerCustomer

In [201…
```
sns.scatterplot(x=final_df['Customers'], y=final_df['Sales']);
```

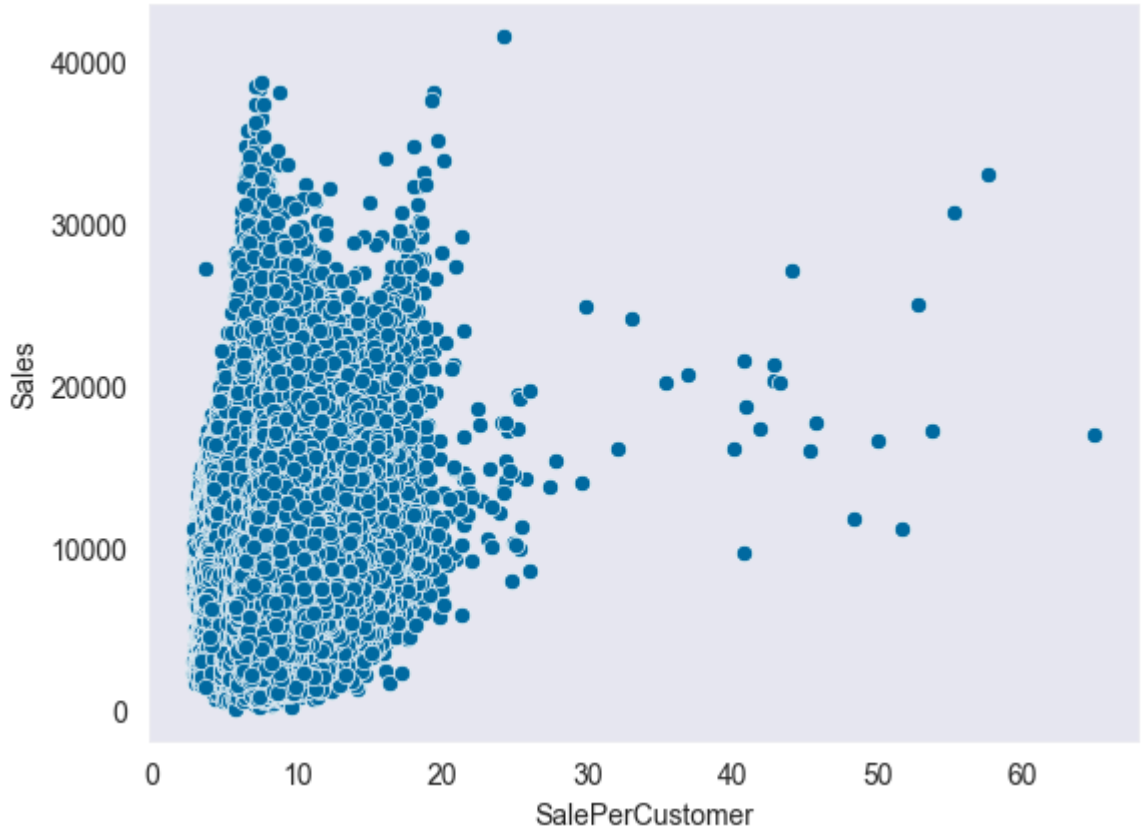


We can see a high positive correlation (linear relationship) between sales and customers. This can also be discovered in the correlation heatmap.

Since they are highly correlated, whatever conclusion that we make about 'Sales' it would also be valid for 'Customers'.

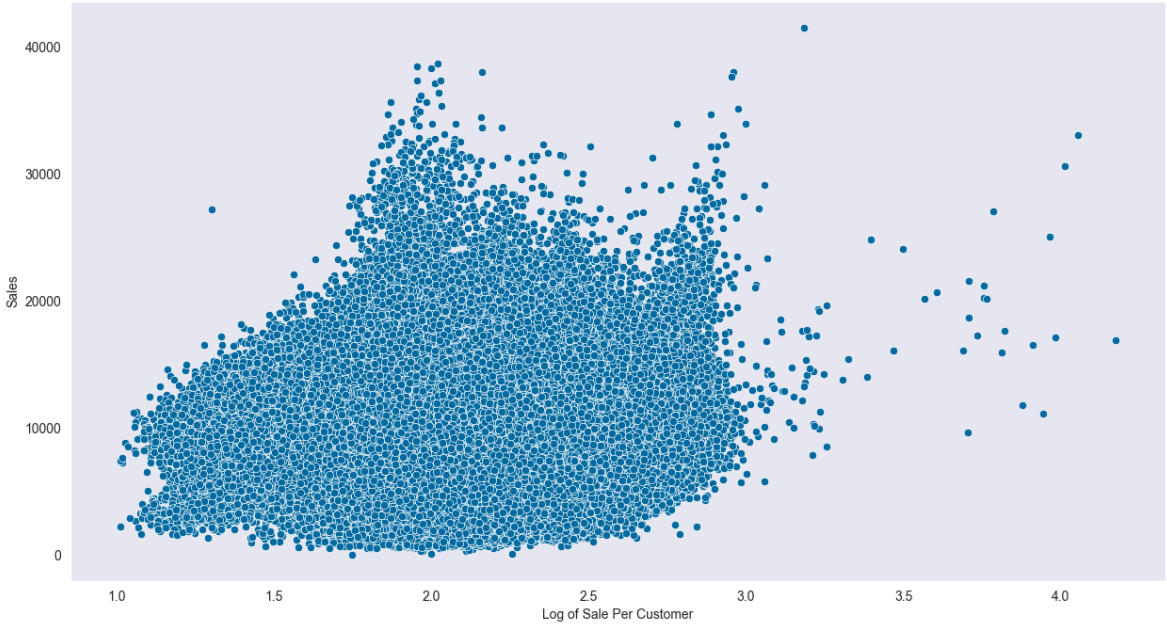
In [202…

```
sns.scatterplot(x=final_df['SalePerCustomer'], y=final_df['Sales']);
```



In [203…

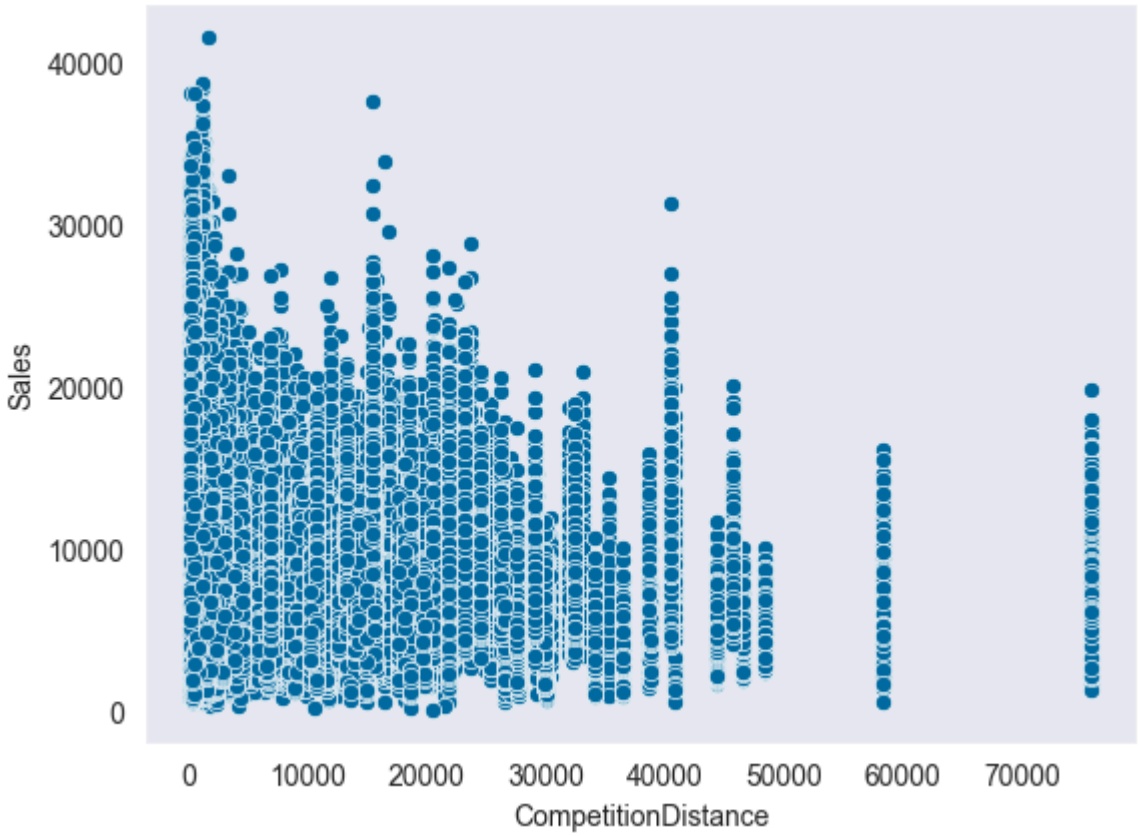
```
plt.figure(figsize=(15, 8))
ax = sns.scatterplot(x=np.log(final_df['SalePerCustomer']), y=final_df['Sales'])

# Add x-axis label
ax.set_xlabel('Log of Sale Per Customer')

# Add y-axis label (optional, if you want to set it as well)
ax.set_ylabel('Sales')

plt.show()
```



We see that Sales becomes almost linearly independent from sale per customer. This can be conformed by the correlation heatmap in 5.10.

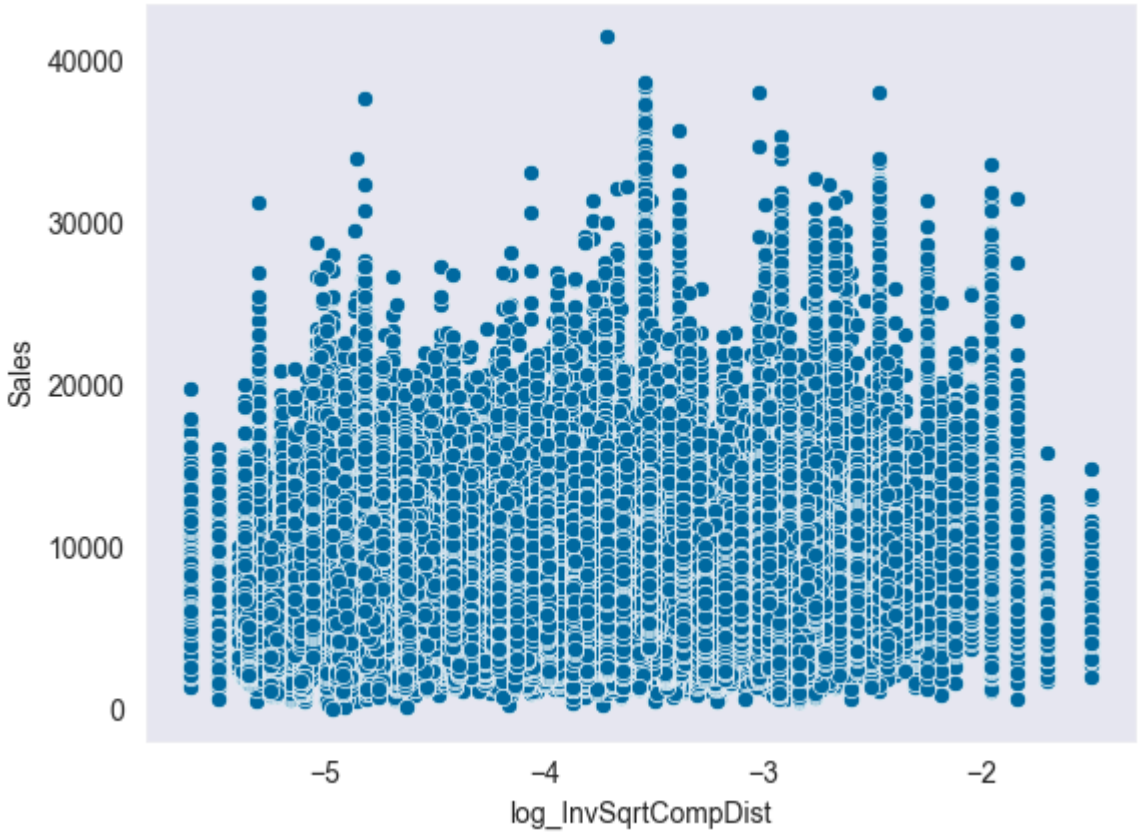
5.7. Effect of Competition Distance on Sales

In [204…
```
sns.scatterplot(x=final_df.CompetitionDistance, y=final_df.Sales);
```



Although the competition distance increases, the sale doesn't increase, this could be due to the highly populated regions

In [205…
```
sns.scatterplot(x=final_df.log_InvSqrtCompDist, y=final_df.Sales);
```

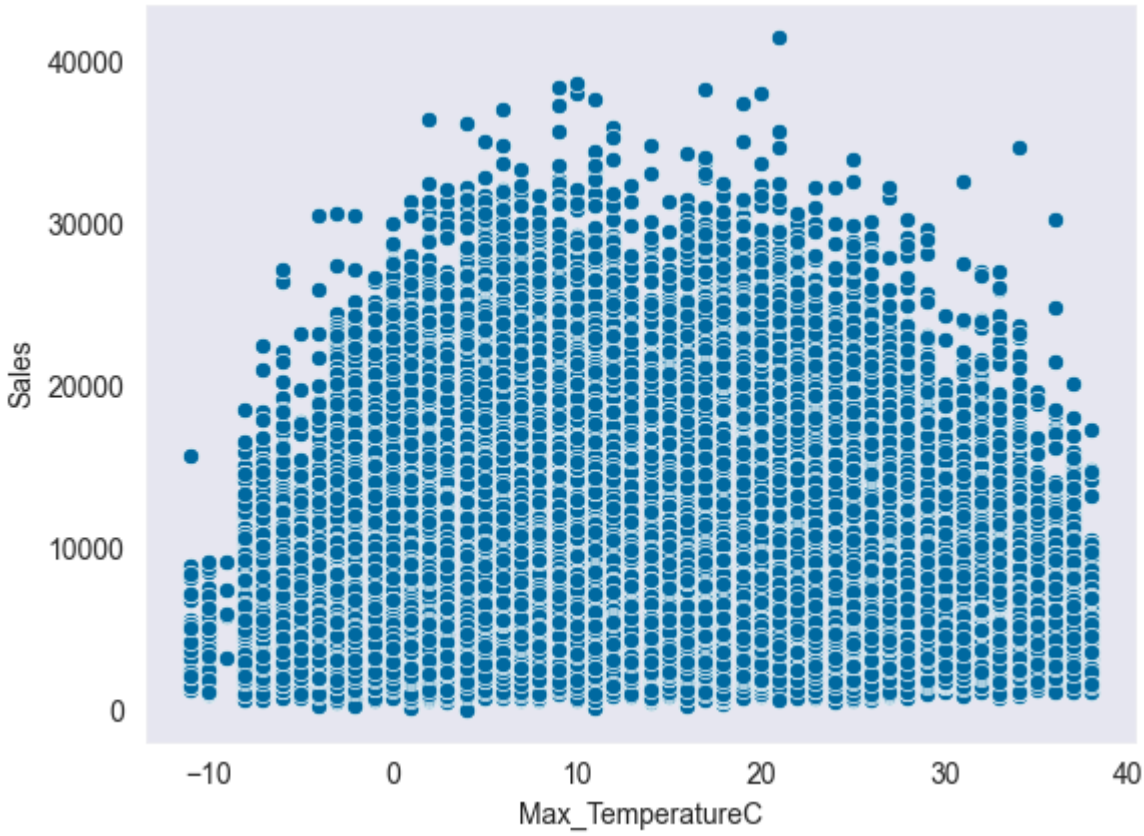


Nothing useful can be found by this chart.

5.8. Temperature vs Sale

In [206…

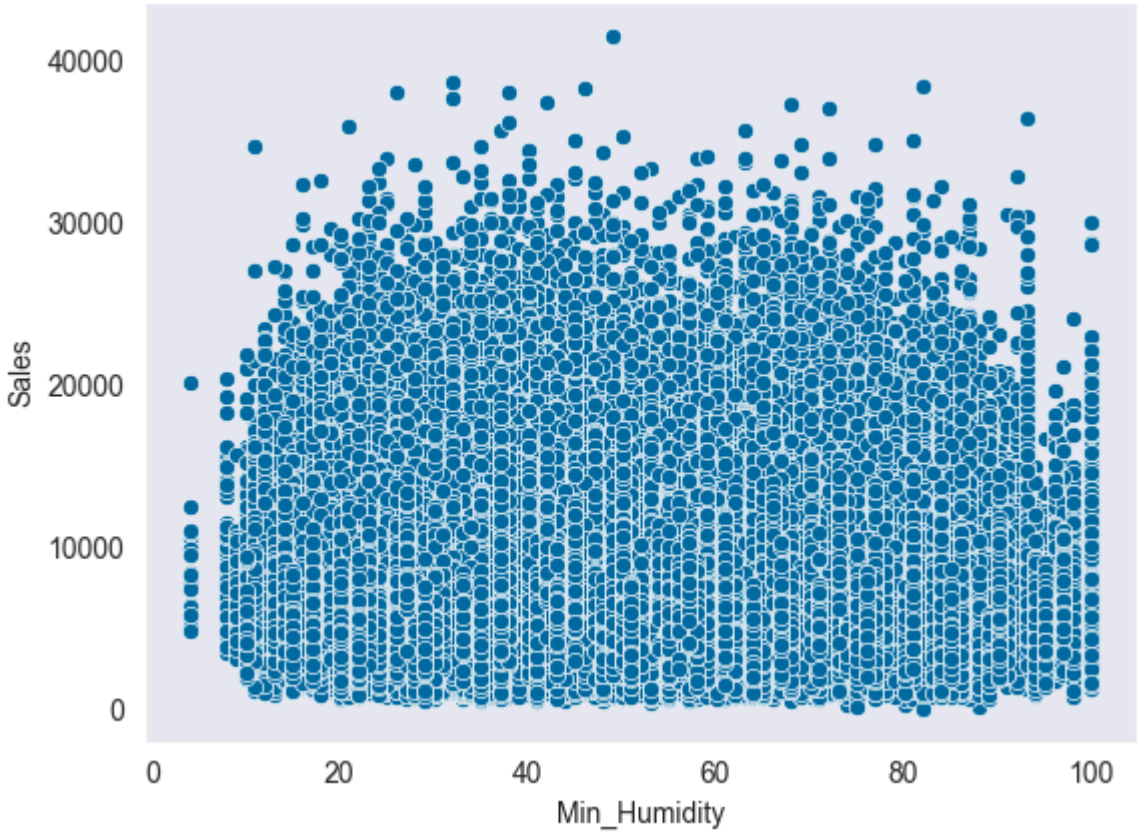
```
sns.scatterplot(x=final_df.Max_TemperatureC, y=final_df.Sales);
```



Not much can be understood by this scatter plots.

5.9. Humidity vs Sale

In [207…
```
sns.scatterplot(x=final_df.Min_Humidity, y=final_df.Sales);
```



Not much can be understood by this scatter plots.

5.10. Correlation

To be able to perform correlation analysis we need to convert the 'bool' variables to 'int8'

In [208…
```
temp_df = final_df.copy()
```

In [209…
```
final_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 844338 entries, 0 to 844337
Data columns (total 21 columns):
 #   Column               Non-Null Count   Dtype
---  ------               --------------   -----
 0   DayOfWeek            844338 non-null  int64
 1   Date                 844338 non-null  datetime64[ns]
 2   Sales                844338 non-null  int32
 3   Customers            844338 non-null  int16
 4   Promo                844338 non-null  bool
 5   StateHoliday         844338 non-null  object
 6   SchoolHoliday        844338 non-null  bool
 7   SalePerCustomer      844338 non-null  float64
 8   Year                 844338 non-null  int32
 9   Month                844338 non-null  int32
 10  StoreType            844338 non-null  object
 11  Assortment           844338 non-null  int8
 12  CompetitionDistance  844338 non-null  int32
 13  Promo2               844338 non-null  bool
 14  PromoInterval        844338 non-null  object
 15  log_InvSqrtCompDist  844338 non-null  float32
 16  Promo2Duration       844338 non-null  int16
 17  CompetitionDuration  844338 non-null  int16
 18  State                844338 non-null  object
 19  Max_TemperatureC     844338 non-null  float16
 20  Min_Humidity         844338 non-null  float16
dtypes: bool(3), datetime64[ns](1), float16(2), float32(1), float64(1), int16(3),
int32(4), int64(1), int8(1), object(4)
memory usage: 72.5+ MB
```

In [210…

```
final_df = final_df.astype(
    {
        'Promo': 'int8',
        'Promo2': 'int8',
        'SchoolHoliday': 'int8',
    }
)
```

In [211…

```
numeric_df = final_df.select_dtypes(include=['number'])
numeric_df.columns
```

Out[211…

```
Index(['DayOfWeek', 'Sales', 'Customers', 'Promo', 'SchoolHoliday',
       'SalePerCustomer', 'Year', 'Month', 'Assortment', 'CompetitionDistance',
       'Promo2', 'log_InvSqrtCompDist', 'Promo2Duration',
       'CompetitionDuration', 'Max_TemperatureC', 'Min_Humidity'],
      dtype='object')
```

In [212…

```
excl_vars = [
    'StateHoliday',
    'StoreType',
    'Year',
    'Month',
    'Store',
    'CompetitionDistance',
]
```

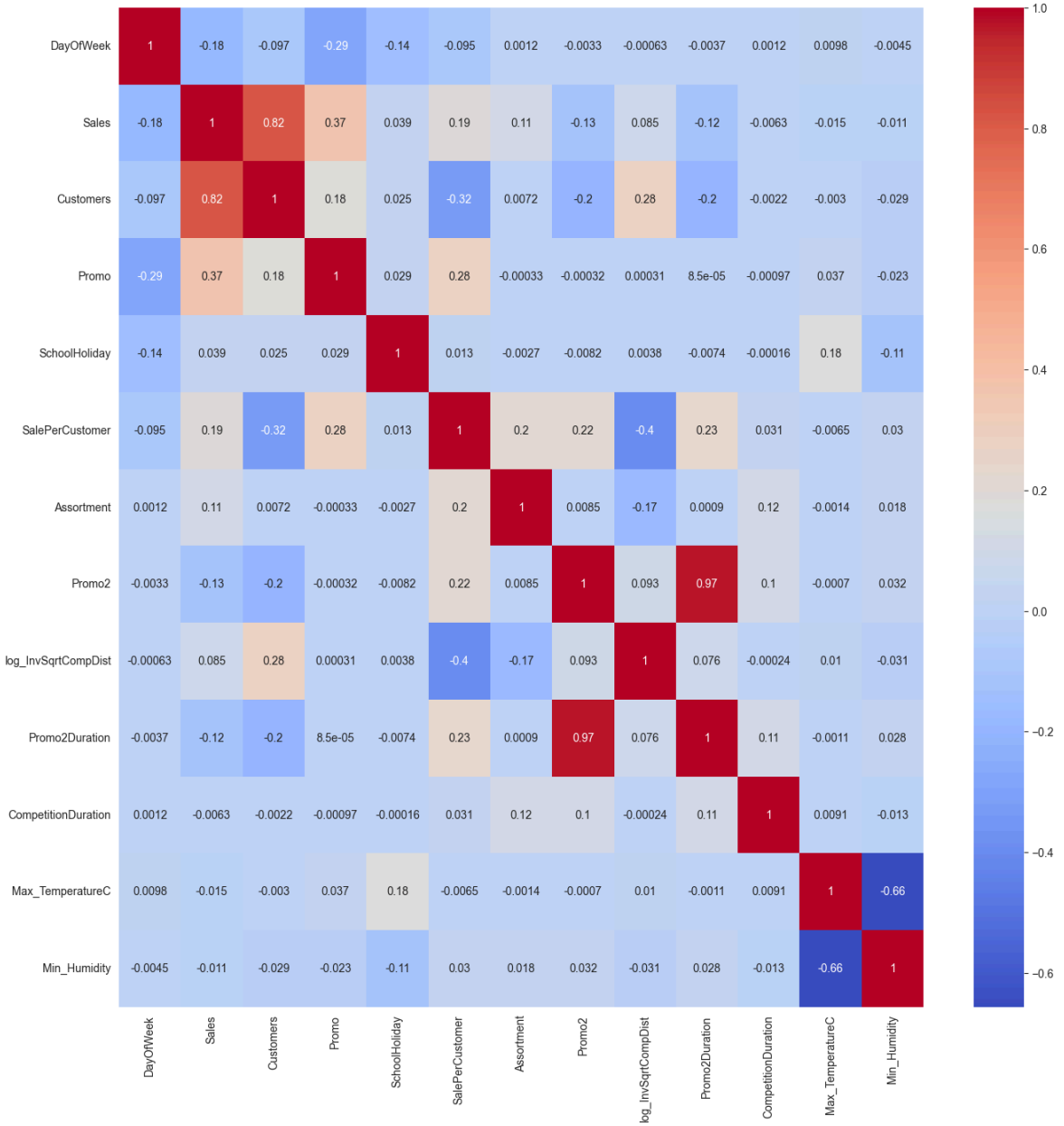
In [213…

```
cols = [col for col in numeric_df.columns if col not in excl_vars]
cols
```

Out[213…
```
['DayOfWeek',
 'Sales',
 'Customers',
 'Promo',
 'SchoolHoliday',
 'SalePerCustomer',
 'Assortment',
 'Promo2',
 'log_InvSqrtCompDist',
 'Promo2Duration',
 'CompetitionDuration',
 'Max_TemperatureC',
 'Min_Humidity']
```

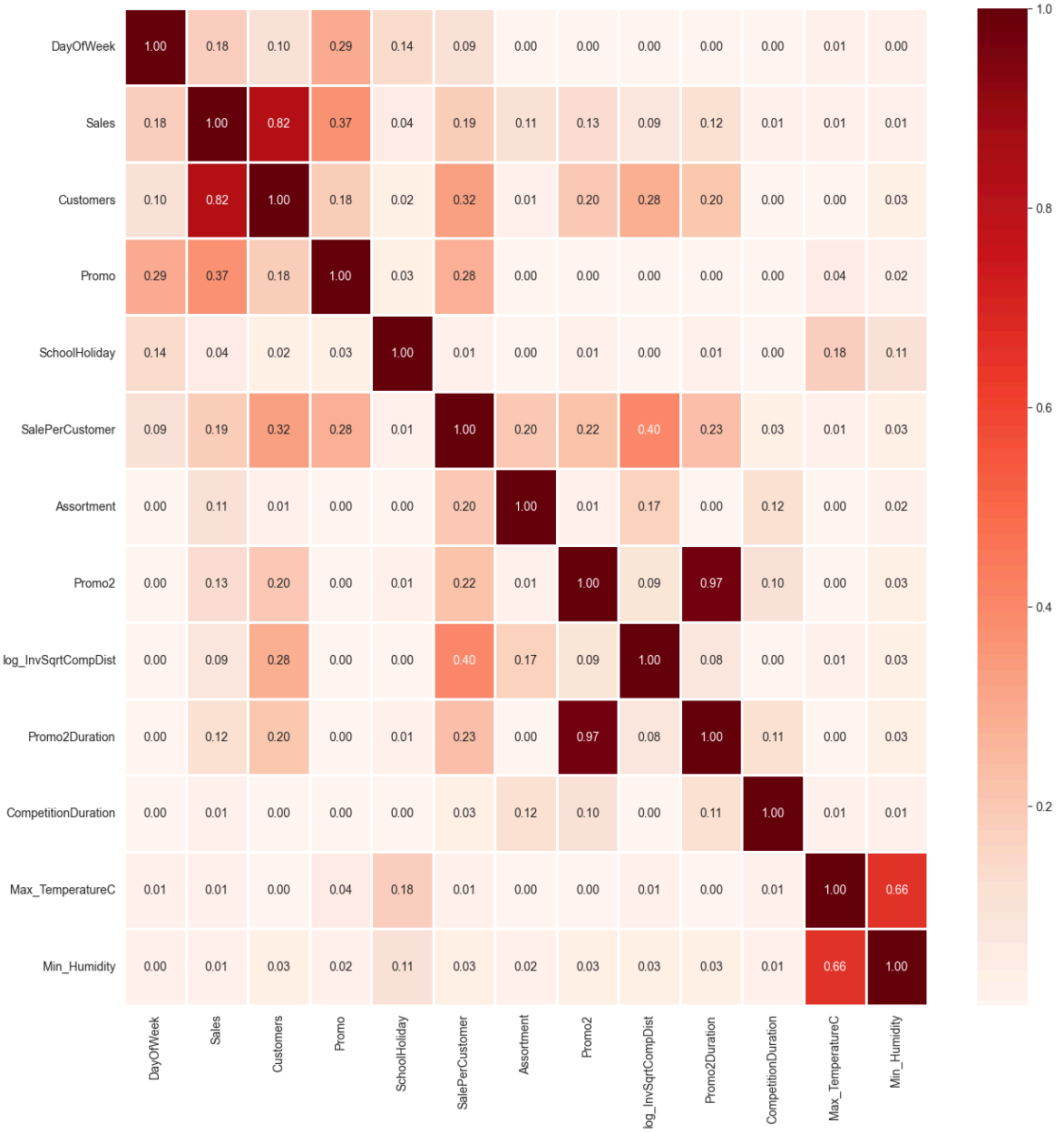
In [214…
```
numeric_df = numeric_df[cols]
```

In [215…
```
plt.figure(figsize=(16, 16))
sns.heatmap(numeric_df.corr(), cmap='coolwarm', annot=True);
```



In [216…
```
correlation = numeric_df.corr()
plt.figure(figsize=(15, 15))
```

```
sns.heatmap(abs(correlation), annot=True, cmap='Reds', linewidths=2, fmt='.2f');
```



The following factors have the highest effect on SalePerCustomer:

1. Promo2Duration, which subsumes the availability of Promo2;
2. log_InvSqrtCompDist;
3. Assortment;
4. Promo;

Nevertheless, correlation analysis can't recognise the nonlinear relationships.

While we saw that there are some patterns that affect sales and customers, the factors identified by the correlation analysis are event-driven.

6. Prediction

We are conisdering separate regression models for 'Sales' and 'Customers'.

6.0. Initialisation

6.0.1 Defining Different Inputs and Outputs & Checking Multicollinearity

Talk about the the importance of avoiding multicollinearity.

The variance inflation factor is a measure for the increase of the variance of the parameter estimates if an additional variable is added to the linear regression. It is a measure for multicollinearity of the design matrix.

One recommendation is that if VIF is greater than 5, then the explanatory variable is highly collinear with the other explanatory variables, and the parameter estimates will have large standard errors because of this.

In [217…
```
def calc_vif(X):
    vif = pd.DataFrame()
    vif['variables'] = X.columns
    vif['VIF'] = [variance_inflation_factor(X.values, i) for i in range(X.shape[
    return(vif)
```

In [218…
```
final_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 844338 entries, 0 to 844337
Data columns (total 21 columns):
 #   Column               Non-Null Count   Dtype
---  ------               --------------   -----
 0   DayOfWeek            844338 non-null  int64
 1   Date                 844338 non-null  datetime64[ns]
 2   Sales                844338 non-null  int32
 3   Customers            844338 non-null  int16
 4   Promo                844338 non-null  int8
 5   StateHoliday         844338 non-null  object
 6   SchoolHoliday        844338 non-null  int8
 7   SalePerCustomer      844338 non-null  float64
 8   Year                 844338 non-null  int32
 9   Month                844338 non-null  int32
 10  StoreType            844338 non-null  object
 11  Assortment           844338 non-null  int8
 12  CompetitionDistance  844338 non-null  int32
 13  Promo2               844338 non-null  int8
 14  PromoInterval        844338 non-null  object
 15  log_InvSqrtCompDist  844338 non-null  float32
 16  Promo2Duration       844338 non-null  int16
 17  CompetitionDuration  844338 non-null  int16
 18  State                844338 non-null  object
 19  Max_TemperatureC     844338 non-null  float16
 20  Min_Humidity         844338 non-null  float16
dtypes: datetime64[ns](1), float16(2), float32(1), float64(1), int16(3), int32
(4), int64(1), int8(4), object(4)
memory usage: 72.5+ MB
```

In [219…
```
numeric_df = final_df.select_dtypes(include=['number'])
```

In [220…
```
numeric_df.columns
```

Out[220…
```
Index(['DayOfWeek', 'Sales', 'Customers', 'Promo', 'SchoolHoliday',
       'SalePerCustomer', 'Year', 'Month', 'Assortment', 'CompetitionDistance',
       'Promo2', 'log_InvSqrtCompDist', 'Promo2Duration',
       'CompetitionDuration', 'Max_TemperatureC', 'Min_Humidity'],
      dtype='object')
```

In [221…
```
numeric_df.drop(['Sales', 'Customers', 'SalePerCustomer', 'CompetitionDistance']
```

In [222…
```
calc_vif(numeric_df)
```

Out[222…

	variables	VIF
0	DayOfWeek	5.766953
1	Promo	1.977007
2	SchoolHoliday	1.313455
3	Year	58.551478
4	Month	5.124331
5	Assortment	1.981701
6	Promo2	39.211819
7	log_InvSqrtCompDist	26.161544
8	Promo2Duration	38.119067
9	CompetitionDuration	1.914286
10	Max_TemperatureC	8.529338
11	Min_Humidity	14.796504

In [223…
```
df1 = numeric_df.drop(['Promo2', 'Year'], axis=1)
```

In [224…
```
calc_vif(df1)
```

Out[224...

	variables	VIF
0	DayOfWeek	5.200679
1	Promo	1.903219
2	SchoolHoliday	1.307804
3	Month	5.116195
4	Assortment	1.976145
5	log_InvSqrtCompDist	15.812156
6	Promo2Duration	1.947080
7	CompetitionDuration	1.890177
8	Max_TemperatureC	6.351865
9	Min_Humidity	9.883023

In [225...
```
df2 = df1.drop(['Min_Humidity'], axis=1)
```

In [226...
```
calc_vif(df2)
```

Out[226...

	variables	VIF
0	DayOfWeek	4.877884
1	Promo	1.857757
2	SchoolHoliday	1.306583
3	Month	4.273224
4	Assortment	1.975908
5	log_InvSqrtCompDist	9.797460
6	Promo2Duration	1.894756
7	CompetitionDuration	1.884026
8	Max_TemperatureC	4.202232

In [227...
```
df2.columns
```

Out[227...
```
Index(['DayOfWeek', 'Promo', 'SchoolHoliday', 'Month', 'Assortment',
       'log_InvSqrtCompDist', 'Promo2Duration', 'CompetitionDuration',
       'Max_TemperatureC'],
      dtype='object')
```

This final set of variables seem good for incorporating in linear regression.

It has to be noted that this has nothing to do with the provided test data set.

In [228...
```
# Defining the dependent variables 1 & 2
dependent_variables_1 = ['Sales', 'Customers']
dependent_variables_2 = ['SalePerCustomer']
```

In [229…
```
# Defining the independent variables 1
independent_variables_1 = list(df2.columns)
independent_variables_1
```

Out[229…
```
['DayOfWeek',
 'Promo',
 'SchoolHoliday',
 'Month',
 'Assortment',
 'log_InvSqrtCompDist',
 'Promo2Duration',
 'CompetitionDuration',
 'Max_TemperatureC']
```

In [252…
```
# By setting 'Date' as index, it won't be included in the columns.
final_df.set_index('Date', inplace=True)
final_df.head()
```

Out[252…

Date	DayOfWeek	Sales	Customers	Promo	StateHoliday	SchoolHoliday	SalePerCus
2015-07-31	5	13995	1498	True	0	True	9.
2015-07-31	5	11144	1162	True	0	True	9.
2015-07-31	5	6670	665	True	0	True	10.
2015-07-31	5	7791	971	True	0	True	8.
2015-07-31	5	6379	715	True	0	True	8.

In [255…
```
# By setting 'Date' as index, it won't be included in the columns.
new_total_df.set_index('Date', inplace=True)
new_total_df.head()
```

Out[255...

Date	Sales	Customers	Promo	SchoolHoliday	SalePerCustomer	Year	Assortment
2015-07-31	13995	1498	True	True	9.342457	2015	2
2015-07-31	11144	1162	True	True	9.590361	2015	0
2015-07-31	6670	665	True	True	10.030075	2015	0
2015-07-31	7791	971	True	True	8.023687	2015	0
2015-07-31	6379	715	True	True	8.921678	2015	0

5 rows × 99 columns

In [256...

```
# Defining the independent variables 2
# In this case we have to be careful to indicate the categorical variables in pa
independent_variables_2 = list(new_total_df.columns.drop(dependent_variables_1 +
independent_variables_2
```

Out[256…
```
['Promo',
 'SchoolHoliday',
 'Year',
 'Assortment',
 'CompetitionDistance',
 'Promo2',
 'log_InvSqrtCompDist',
 'Promo2Duration',
 'CompetitionDuration',
 'Max_TemperatureC',
 'Min_Humidity',
 'StateHoliday_0',
 'StateHoliday_a',
 'StateHoliday_b',
 'StateHoliday_c',
 'State_BE',
 'State_BW',
 'State_BY',
 'State_HB,NI',
 'State_HE',
 'State_HH',
 'State_NW',
 'State_RP',
 'State_SH',
 'State_SN',
 'State_ST',
 'State_TH',
 'StateHoliday_State_0_BE',
 'StateHoliday_State_0_BW',
 'StateHoliday_State_0_BY',
 'StateHoliday_State_0_HB,NI',
 'StateHoliday_State_0_HE',
 'StateHoliday_State_0_HH',
 'StateHoliday_State_0_NW',
 'StateHoliday_State_0_RP',
 'StateHoliday_State_0_SH',
 'StateHoliday_State_0_SN',
 'StateHoliday_State_0_ST',
 'StateHoliday_State_0_TH',
 'StateHoliday_State_a_BE',
 'StateHoliday_State_a_BW',
 'StateHoliday_State_a_BY',
 'StateHoliday_State_a_HB,NI',
 'StateHoliday_State_a_HE',
 'StateHoliday_State_a_HH',
 'StateHoliday_State_a_NW',
 'StateHoliday_State_a_RP',
 'StateHoliday_State_a_SH',
 'StateHoliday_State_a_SN',
 'StateHoliday_State_a_ST',
 'StateHoliday_State_a_TH',
 'StateHoliday_State_b_BE',
 'StateHoliday_State_b_BW',
 'StateHoliday_State_b_BY',
 'StateHoliday_State_b_HB,NI',
 'StateHoliday_State_b_HE',
 'StateHoliday_State_b_HH',
 'StateHoliday_State_b_NW',
 'StateHoliday_State_b_RP',
 'StateHoliday_State_b_SH',
```

```
 'StateHoliday_State_c_BE',
 'StateHoliday_State_c_BW',
 'StateHoliday_State_c_BY',
 'StateHoliday_State_c_HB,NI',
 'StateHoliday_State_c_HE',
 'StateHoliday_State_c_HH',
 'StateHoliday_State_c_NW',
 'StateHoliday_State_c_RP',
 'StateHoliday_State_c_SH',
 'PromoInterval_Feb,May,Aug,Nov',
 'PromoInterval_Jan,Apr,Jul,Oct',
 'PromoInterval_Mar,Jun,Sept,Dec',
 'PromoInterval_None',
 'StoreType_a',
 'StoreType_b',
 'StoreType_c',
 'StoreType_d',
 'DayOfWeek_1',
 'DayOfWeek_2',
 'DayOfWeek_3',
 'DayOfWeek_4',
 'DayOfWeek_5',
 'DayOfWeek_6',
 'DayOfWeek_7',
 'Month_1',
 'Month_2',
 'Month_3',
 'Month_4',
 'Month_5',
 'Month_6',
 'Month_7',
 'Month_8',
 'Month_9',
 'Month_10',
 'Month_11',
 'Month_12']
```

6.0.2 Cross_Validation: Separating Train, Test (Validation) Sets

Considering separate train and test sets is essential to ensure that a machine learning model can generalize well to unseen data, thereby providing an unbiased evaluation of its performance. Without this separation, a model might overfit to the training data, leading to overly optimistic performance metrics that do not reflect its true predictive capability.

Leave-one-out cross-validation (LOOCV) is a technique where each data point in the dataset is used once as a test set while the remaining data points form the training set, which helps in assessing model performance with high precision but can be computationally expensive. Cross-validation, particularly k-fold cross-validation, involves splitting the dataset into k subsets and using each subset as a test set while the remaining k-1 subsets form the training set, providing a robust estimate of model performance by averaging the results across all k trials.

In [234…
```
def train_test_vars(independent_variables: list, dependent_variables: list ,df:
    # This is not a Pandas data frame. This is a numpy array.
    X = df[independent_variables].values
    Y = df[dependent_variables].values

    # We do this for choosing the best model, since we don't have the values of
    # Although this kind of data splitting is not suitable if we want to test th
    # In the case of auto-correlation, a portion from the end of the data set sh
    return train_test_split(X, Y, test_size=test_portion, random_state=0)
```

In [235…
```
# Testing the function.
X_train, X_test, Y_train, Y_test = train_test_vars(independent_variables_1, depe
```

In [236…
```
def eval_model():
    score = model.score(X_train, Y_train)
    MSE = mse(Y_test, Y_pred)
    RMSE = np.sqrt(MSE)
    r2 = r2_score(Y_test, Y_pred)

    print(f'{score = }')
    print(f'{MSE = }')
    print(f'{RMSE = }')
    print(f'{r2 = }')

    # return pd.DataFrame(zip(Y_test, Y_pred), columns=['Actual', 'Prediction'])
```

6.1. (Generalised) Linear Regression

For modeling categorical variables (and interaction effects), we can directly use patsy in the linear regression model.

6.1.1. Linear Regression

In [258…
```
X_train, X_test, Y_train, Y_test = train_test_vars(independent_variables_1, depe
```

In [259…
```
model = LinearRegression()
```

In [260…
```
model.fit(X_train, Y_train)
```

Out[260…
```
LinearRegression
LinearRegression()
```

In [261…
```
Y_pred = model.predict(X_test)
```

In [262…
```
eval_model()
```

```
score = 0.321961977672092
MSE = 3.2589992525418467
RMSE = 1.8052698558780198
r2 = 0.3225886536226187
```

In [ ]:

In [263…
```
model.coef_
```

Out[263…
```
array([[-1.58018202e-02,  1.22714194e+00,  4.41877390e-02,
         1.83517552e-02,  2.93356299e-01, -1.10883927e+00,
         2.20407388e-03, -5.10864958e-05, -5.57072419e-03]])
```

In [264…
```
model.intercept_
```

Out[264…
```
array([3.924065])
```

6.1.2 Linear Regression 2

In [265…
```
independent_variables_2
```

Out[265…

```
['Promo',
 'SchoolHoliday',
 'Year',
 'Assortment',
 'CompetitionDistance',
 'Promo2',
 'log_InvSqrtCompDist',
 'Promo2Duration',
 'CompetitionDuration',
 'Max_TemperatureC',
 'Min_Humidity',
 'StateHoliday_0',
 'StateHoliday_a',
 'StateHoliday_b',
 'StateHoliday_c',
 'State_BE',
 'State_BW',
 'State_BY',
 'State_HB,NI',
 'State_HE',
 'State_HH',
 'State_NW',
 'State_RP',
 'State_SH',
 'State_SN',
 'State_ST',
 'State_TH',
 'StateHoliday_State_0_BE',
 'StateHoliday_State_0_BW',
 'StateHoliday_State_0_BY',
 'StateHoliday_State_0_HB,NI',
 'StateHoliday_State_0_HE',
 'StateHoliday_State_0_HH',
 'StateHoliday_State_0_NW',
 'StateHoliday_State_0_RP',
 'StateHoliday_State_0_SH',
 'StateHoliday_State_0_SN',
 'StateHoliday_State_0_ST',
 'StateHoliday_State_0_TH',
 'StateHoliday_State_a_BE',
 'StateHoliday_State_a_BW',
 'StateHoliday_State_a_BY',
 'StateHoliday_State_a_HB,NI',
 'StateHoliday_State_a_HE',
 'StateHoliday_State_a_HH',
 'StateHoliday_State_a_NW',
 'StateHoliday_State_a_RP',
 'StateHoliday_State_a_SH',
 'StateHoliday_State_a_SN',
 'StateHoliday_State_a_ST',
 'StateHoliday_State_a_TH',
 'StateHoliday_State_b_BE',
 'StateHoliday_State_b_BW',
 'StateHoliday_State_b_BY',
 'StateHoliday_State_b_HB,NI',
 'StateHoliday_State_b_HE',
 'StateHoliday_State_b_HH',
 'StateHoliday_State_b_NW',
 'StateHoliday_State_b_RP',
 'StateHoliday_State_b_SH',
```

```
 'StateHoliday_State_c_BE',
 'StateHoliday_State_c_BW',
 'StateHoliday_State_c_BY',
 'StateHoliday_State_c_HB,NI',
 'StateHoliday_State_c_HE',
 'StateHoliday_State_c_HH',
 'StateHoliday_State_c_NW',
 'StateHoliday_State_c_RP',
 'StateHoliday_State_c_SH',
 'PromoInterval_Feb,May,Aug,Nov',
 'PromoInterval_Jan,Apr,Jul,Oct',
 'PromoInterval_Mar,Jun,Sept,Dec',
 'PromoInterval_None',
 'StoreType_a',
 'StoreType_b',
 'StoreType_c',
 'StoreType_d',
 'DayOfWeek_1',
 'DayOfWeek_2',
 'DayOfWeek_3',
 'DayOfWeek_4',
 'DayOfWeek_5',
 'DayOfWeek_6',
 'DayOfWeek_7',
 'Month_1',
 'Month_2',
 'Month_3',
 'Month_4',
 'Month_5',
 'Month_6',
 'Month_7',
 'Month_8',
 'Month_9',
 'Month_10',
 'Month_11',
 'Month_12']
```

In [266…
```
X_train, X_test, Y_train, Y_test = train_test_vars(independent_variables_2, depe
```

In [267…
```
model = LinearRegression()
```

In [268…
```
model.fit(X_train, Y_train)
```

Out[268…
```
LinearRegression
LinearRegression()
```

In [269…
```
Y_pred = model.predict(X_test)
```

In [270…
```
eval_model()
```

```
score = 0.5932607748226083
MSE = 1.9561739093966553
RMSE = 1.3986328715558831
r2 = 0.5933922351534267
```

In [271…
```
model.coef_
```

Out[271…
```
array([[ 1.36255336e+00,  1.67387003e-01,  2.45144545e-01,
         2.40334040e-01, -2.33730075e-05, -3.17104977e-01,
        -1.02033129e+00,  2.54142937e-03, -1.20697980e-04,
        -1.32164526e-02,  1.49648432e-03, -1.71874396e-01,
        -1.76464834e-01, -6.59245117e-02,  4.14263741e-01,
         1.16576607e+00, -1.34898731e-01,  3.57026315e-01,
         3.46339557e-01,  4.45056532e-01,  9.25830314e-01,
         1.62986073e-01, -5.40031017e-02, -1.42438843e+00,
        -1.62542622e-01, -1.05307081e+00, -5.74101160e-01,
        -8.42824528e-01,  4.88859511e-01,  6.39969465e-01,
        -9.10280230e-01,  3.03615709e-01, -1.89291719e+00,
         3.99061691e-02,  2.88332437e-01,  7.91600124e-01,
        -4.98836936e-02,  7.32841917e-01,  2.38905910e-01,
         1.12173391e+00, -1.31742136e-01, -2.27576482e-01,
         6.96137208e-01,  1.86808565e-02,  1.18077559e+00,
         3.98853280e-02, -1.05982370e-01, -5.67980120e-02,
        -1.12658929e-01, -1.78591273e+00, -8.13007070e-01,
         3.06935966e-01, -4.32399991e-01, -2.30032526e-01,
         1.36237484e-01, -6.56507768e-02,  8.75657590e-01,
         5.68412398e-03, -1.03696464e-01, -5.58659917e-01,
         5.79920713e-01, -5.96161148e-02,  1.74665858e-01,
         4.24245096e-01,  1.88410743e-01,  7.62314322e-01,
         7.75104524e-02, -1.32656705e-01, -1.60053062e+00,
        -8.87156332e-02, -1.09659531e-01, -1.18729813e-01,
         3.17104977e-01,  4.96776255e-01, -2.88513192e+00,
         2.60427942e-01,  2.12792772e+00,  1.63031732e-01,
        -1.11739763e-01, -2.03287840e-01, -3.28340945e-01,
        -2.22587575e-01,  3.26602220e-01,  3.76322171e-01,
        -2.74542829e-01, -2.82606563e-01, -1.33522120e-01,
        -1.10442414e-01,  1.60228534e-02,  1.61377942e-01,
         2.14715183e-01, -2.19681127e-02, -1.16747830e-01,
        -1.50576344e-01,  1.08372643e-01,  5.89917592e-01]])
```

In [272…
```
model.intercept_
```

Out[272…
```
array([-490.06401858])
```

All the coefficients are almost in the same range, not too large and not too small. Therefore, we can expect the ridge method to also provide similar results, and the model have a low variance in cross-validation.

6.1. (Generalised) Linear Regression (cont.)

In [273…
```
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

In [274…
```
linear_regression = LinearRegression()
linear_regression.fit(X_train, Y_train)
```

Out[274…

▾ LinearRegression

```
LinearRegression()
```

In [275…
```
V_pred = linear_regression.predict(X_test)
V_pred
```

Out[275…
```
array([[ 8.42824268],
       [10.62471485],
       [ 9.12379932],
       ...,
       [10.22829151],
       [12.49562549],
       [ 7.1654253 ]])
```

In [277…
```
linear_regression.score(X_train, Y_train)
```

Out[277…
```
0.593237566739962
```

In [276…
```
regression_DataFrame = pd.DataFrame(zip(Y_test, Y_pred), columns=['Actual', 'Pre
regression_DataFrame
```

Out[276…

	Actual	Prediction
0	[8.693498452012383]	[8.423761678271376]
1	[11.74410480349345]	[10.626293550645642]
2	[9.53623188405797]	[9.117927679450759]
3	[6.729216152019003]	[8.522870602403884]
4	[10.16810344827586]	[9.985039750120507]
…	…	…
168863	[6.995522388059701]	[8.276607903643367]
168864	[9.82099596231494]	[11.542477214182327]
168865	[10.9623786407767]	[10.227205438633746]
168866	[12.862790697674418]	[12.496499030025177]
168867	[6.559380378657487]	[7.152743584669622]

168868 rows × 2 columns

In [282…
```
eval_model()
```

```
score = -51638.64334324601
MSE = 1.9561739093966553
RMSE = 1.3986328715558831
r2 = 0.5933922351534267
```

6.2. LASSO

In [283…
```
L1 = Lasso(alpha=0.4, max_iter=10000, selection='cyclic', tol=0.0001)
```

In [284…
```
L1.fit(X_train, Y_train)
```

Out[284…
```
Lasso
Lasso(alpha=0.4, max_iter=10000)
```

In [285…
```
Y_pred_lasso = L1.predict(X_test)
```

In [286…
```
L1.score(X_test, Y_test)
```

Out[286… 0.3746105308595904

In [288…
```
cv_scores = cross_val_score(L1, X_train, Y_train, cv=10)
mean_cv_score = cv_scores.mean()
```

In [289…
```
cv_scores
```

Out[289…
```
array([0.37639922, 0.37847972, 0.37877114, 0.37473452, 0.36930035,
       0.37639546, 0.37594796, 0.37022275, 0.3778562 , 0.37480747])
```

In [290…
```
mean_cv_score
```

Out[290… 0.3752914794857337

In [291…
```
# Define the range of alpha values
parameters = {'alpha': [0.1, 0.2, 0.3, 0.4, 0.5]}
```

In [292…
```
lasso_cv = GridSearchCV(L1, parameters, cv=5)
lasso_cv.fit(X_train, Y_train)
```

Out[292…
```
GridSearchCV
best_estimator_: Lasso
Lasso
```

In [293…
```
best_alpha_lasso = lasso_cv.best_params_['alpha']
best_score_lasso = lasso_cv.best_score_
```

In [294…
```
best_alpha_lasso
```

Out[294… 0.1

In [295…
```
best_score_lasso
```

Out[295… 0.5431331421728969

In [297…
```
pd.DataFrame(zip(Y_test, Y_pred_lasso), columns=['Actual', 'Prediction'])
```

Out[297…

	Actual	Prediction
0	[8.693498452012383]	8.925210
1	[11.74410480349345]	9.706693
2	[9.53623188405797]	8.927943
3	[6.729216152019003]	8.731511
4	[10.16810344827586]	9.118410
…	…	…
168863	[6.995522388059701]	8.914022
168864	[9.82099596231494]	10.997252
168865	[10.9623786407767]	9.698234
168866	[12.862790697674418]	11.089024
168867	[6.559380378657487]	8.320253

168868 rows × 2 columns

In [298…
```
L2 = Ridge(alpha=0.5)
```

In [299…
```
L2.fit(X_train, Y_train)
```

Out[299…

Ridge

Ridge(alpha=0.5)

In [300…
```
L2.predict(X_test)
```

Out[300…
```
array([[ 8.42376115],
       [10.62628701],
       [ 9.11792568],
       ...,
       [10.22719939],
       [12.49649612],
       [ 7.15275072]])
```

In [301…
```
L2.score(X_test, Y_test)
```

Out[301…
```
0.5933922350358165
```

6.3. Ridge

In [302…
```
ridge = Ridge(max_iter=10000, solver='auto')
```

In [303…
```
parameters = {'alpha': [0.1, 0.2, 0.3, 0.4, 0.5]}
```

In [304…
```
ridge_cv = GridSearchCV(L2, parameters, cv=5)
ridge_cv.fit(X_train, Y_train)
```

Out[304…

GridSearchCV

best_estimator_: Ridge

Ridge

In [305…
```
best_alpha = ridge_cv.best_params_['alpha']
best_score = ridge_cv.best_score_
```

In [306…
```
ridge_best = Ridge(alpha=best_alpha, max_iter=10000, solver='auto')
cv_scores = cross_val_score(ridge_best, X_train, Y_train, cv=5)
```

In [307…
```
max_score = cv_scores.max()
```

In [310…
```
print(f'{best_alpha = }')
print(f'{best_score = }')
print(f'{max_score = }')
```

```
best_alpha = 0.5
best_score = 0.5931942088199398
max_score = 0.5962718503733638
```

Keeping best_score, when trying different models

6.4. Elastic Net

In [311…
```
elastic_net = ElasticNet(max_iter=10000)
```

In [312…
```
parameters = {'alpha': [0.1, 0.2, 0.3, 0.4, 0.5], 'l1_ratio': [0.1, 0.3, 0.5, 0.
```

In [313…
```
elastic_net_cv = GridSearchCV(elastic_net, parameters, cv=5)
elastic_net_cv.fit(X_train, Y_train)
```

Out[313…

GridSearchCV

best_estimator_: ElasticNet

ElasticNet

In [314…
```
best_alpha = elastic_net_cv.best_params_['alpha']
best_l1_ratio = elastic_net_cv.best_params_['l1_ratio']
best_score_elastic = elastic_net_cv.best_score_
```

In [316…
```
# Create an Elastic Net model with the best hyperparameters
elastic_net_best = ElasticNet(alpha=best_alpha, l1_ratio=best_l1_ratio, max_iter
elastic_net_best.fit(X_train, Y_train)
```

Out[316…

ElasticNet

```
ElasticNet(alpha=0.1, l1_ratio=0.1, max_iter=10000)
```

In [317…
```
test_score = elastic_net_best.score(X_test, Y_test)
```

In [318…
```
print(f'{best_alpha = }')
print(f'{best_l1_ratio = }')
print(f'{best_score = }')
print(f'{test_score = }')
```

```
best_alpha = 0.1
best_l1_ratio = 0.1
best_score = 0.5931942088199398
test_score = 0.5847087173686343
```

6.5. Decision Tree

In [324…
```
decision_tree = DecisionTreeRegressor(max_depth=5)
decision_tree.fit(X_train, Y_train)
Y_pred_DT = decision_tree.predict(X_test)
Y_train_DT = decision_tree.predict(X_train)
```

In [325…
```
R2 = r2_score(Y_test, Y_pred_DT)
print(f'{R2 = }')
```

```
R2 = 0.5576326807274778
```

In [326…
```
MSE = mse(Y_test, Y_pred_DT)
RMSE = np.sqrt(MSE)
print(f'{MSE = }')
print(f'{RMSE = }')
```

```
MSE = 2.128211714444687
RMSE = 1.4588391667502922
```

In [327…
```
decision_tree_DataFrame = pd.DataFrame(zip(Y_test, Y_pred_DT), columns=['Actual'
decision_tree_DataFrame
```

Out[327…

	Actual	Prediction
0	[8.693498452012383]	8.526171
1	[11.74410480349345]	12.222683
2	[9.53623188405797]	8.526171
3	[6.729216152019003]	8.029220
4	[10.16810344827586]	10.622635
...	...	...
168863	[6.995522388059701]	8.526171
168864	[9.82099596231494]	11.187413
168865	[10.9623786407767]	12.222683
168866	[12.862790697674418]	13.035106
168867	[6.559380378657487]	7.227275

168868 rows × 2 columns

6.6. Random Forest Regression

In [328…
```
# Create a random forest regressor
random_forest = RandomForestRegressor(n_estimators=500, max_depth=8, n_jobs=2)
```

In [331…
```
Y_train.shape
```

Out[331… (675470, 1)

In [333…
```
# Fit the random forest to the training data
random_forest.fit(X_train, Y_train.ravel())
```

Out[333…

RandomForestRegressor

RandomForestRegressor(max_depth=8, n_estimators=500, n_jobs=2)

In [334…
```
Y_pred_RF = random_forest.predict(X_test)
```

In [335…
```
MSE = mse(Y_test, Y_pred_RF)
RMSE = np.sqrt(MSE)

print(f'{MSE = }')
print(f'{RMSE = }')
```

```
MSE = 1.6012518842052323
RMSE = 1.2654058179909053
```

In [336…
```
R2 = r2_score(Y_test, Y_pred_RF)
print(f'{R2 = }')
```

```
R2 = 0.6671658657415243
```

6.7. Adaboost

In [338…
```
adaboost = AdaBoostRegressor(n_estimators=500, learning_rate=0.01)
adaboost.fit(X_train, Y_train.ravel())
```

Out[338…
```
AdaBoostRegressor(learning_rate=0.01, n_estimators=500)
```

In [339…
```
Y_pred_ada = adaboost.predict(X_test)
```

In [340…
```
MSE = mse(Y_test, Y_pred_ada)
RMSE = np.sqrt(MSE)

print(f'{MSE = }')
print(f'{RMSE = }')
```

```
MSE = 2.6749670028288817
RMSE = 1.63553263581895
```

In [341…
```
R2 = r2_score(Y_test, Y_pred_ada)
print(f'{R2 = }')
```

```
R2 = 0.44398483752701823
```

6.8. XGBoost

In [343…
```
xgboost = xgb.XGBRegressor(max_depth=9, n_jobs=2)
xgboost.fit(X_train, Y_train)
```

Out[343…
```
XGBRegressor(base_score=None, booster=None, callbacks=None,
             colsample_bylevel=None, colsample_bynode=None,
             colsample_bytree=None, device=None, early_stopping_rou
nds=None,
             enable_categorical=False, eval_metric=None, feature_ty
pes=None,
             gamma=None, grow_policy=None, importance_type=None,
             interaction_constraints=None, learning_rate=None, max_
bin=None,
```

In [344…
```
Y_pred_xgb = xgboost.predict(X_test)
```

In [345…
```
MSE = mse(Y_test, Y_pred_xgb)
RMSE = np.sqrt(MSE)

print(f'{MSE = }')
print(f'{RMSE = }')
```

```
MSE = 0.32296082706954526
RMSE = 0.5682964253534816
```

In [346…
```
R2 = r2_score(Y_test, Y_pred_xgb)
print(f'{R2 = }')
```

```
R2 = 0.9328697824886892
```

6.9. LSTM

In [347…
```
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler
from keras.models import Sequential
from keras.layers import LSTM, Dense
from keras.optimizers import Adam
import matplotlib.pyplot as plt
```

In [352…
```
%pwd
```

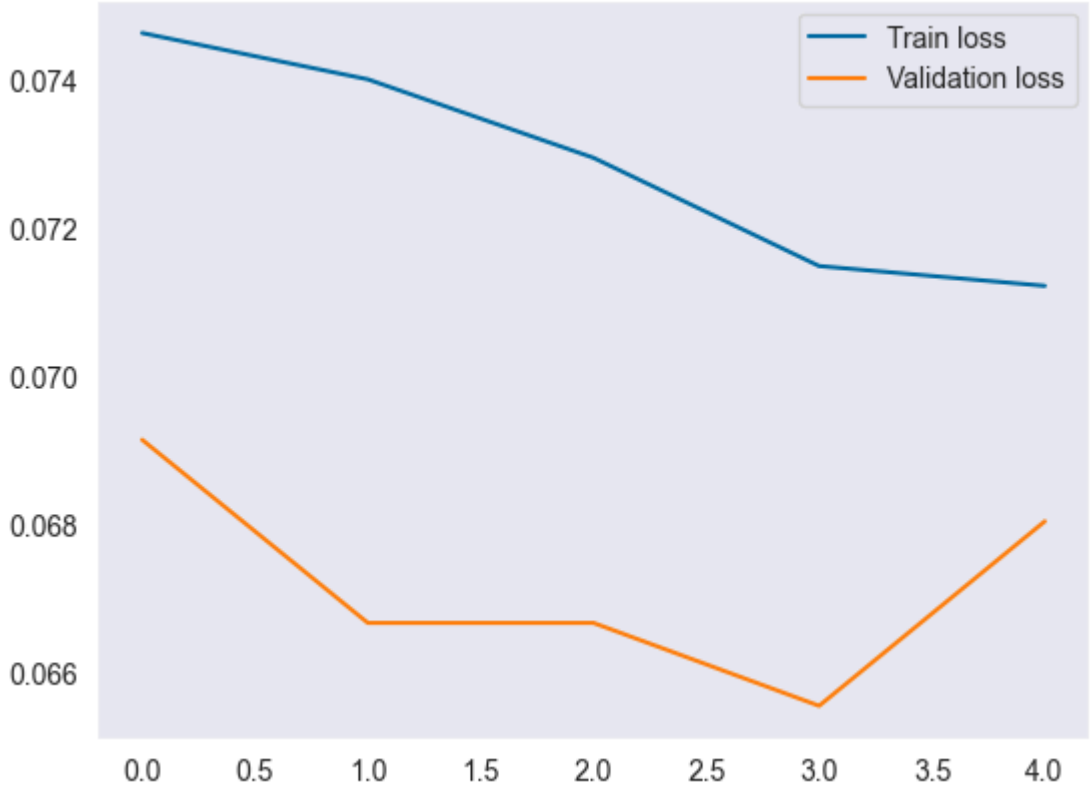
Out[352…
```
'C:\\Drive D\\Python\\Rossmann\\datasets'
```

In [355…
```
# Load the data
data = pd.read_csv(r'train_df.csv', parse_dates=['Date'], low_memory=False)
data.sort_values('Date', inplace=True)

# Select the relevant features
features = ['Store', 'DayOfWeek', 'Sales', 'Customers', 'Promo', 'StateHoliday',

# Preprocess the data
data['StateHoliday'] = data['StateHoliday'].astype('category').cat.codes
data['Store'] = data['Store'].astype('category').cat.codes

# Scale the features
scaler = MinMaxScaler()
data[features] = scaler.fit_transform(data[features])

# Prepare the training data
def create_dataset(data, time_step=1):
    X, Y = [], []
    for i in range(len(data)-time_step-1):
        a = data[i:(i+time_step), :-2]
        X.append(a)
        Y.append(data[i + time_step, -2:])
    return np.array(X), np.array(Y)

time_step = 60  # Number of previous days to consider for prediction
data_values = data[features].values

X, Y = create_dataset(data_values, time_step)

# Split into training and test sets
train_size = int(len(X) * 0.8)
test_size = len(X) - train_size
X_train, X_test = X[0:train_size], X[train_size:len(X)]
Y_train, Y_test = Y[0:train_size], Y[train_size:len(Y)]
```

In [362…
```
# Define the input layer
input_layer = Input(shape=(time_step, X_train.shape[2]))
```

```
# Add the LSTM and Dense layers
x = LSTM(50, return_sequences=True)(input_layer)
x = LSTM(50, return_sequences=False)(x)
x = Dense(25)(x)
output_layer = Dense(2)(x)

# Create the model
model = Model(inputs=input_layer, outputs=output_layer)

# Compile the model
model.compile(optimizer=Adam(learning_rate=0.001), loss='mean_squared_error')

# Train the model
history = model.fit(X_train, Y_train, epochs=5, batch_size=64, validation_data=(

# Plot the training and validation loss
plt.plot(history.history['loss'], label='Train loss')
plt.plot(history.history['val_loss'], label='Validation loss')
plt.legend()
plt.show()
```

```
Epoch 1/5
10554/10554 ━━━━━━━━━━━━━━━━━━━━ 341s 32ms/step - loss: 0.0755 - val_loss: 0.0692
Epoch 2/5
10554/10554 ━━━━━━━━━━━━━━━━━━━━ 341s 32ms/step - loss: 0.0743 - val_loss: 0.0667
Epoch 3/5
10554/10554 ━━━━━━━━━━━━━━━━━━━━ 342s 32ms/step - loss: 0.0735 - val_loss: 0.0667
Epoch 4/5
10554/10554 ━━━━━━━━━━━━━━━━━━━━ 352s 33ms/step - loss: 0.0719 - val_loss: 0.0656
Epoch 5/5
10554/10554 ━━━━━━━━━━━━━━━━━━━━ 349s 33ms/step - loss: 0.0716 - val_loss: 0.0681
```



In [363…

```
# Make predictions
train_predict = model.predict(X_train)
test_predict = model.predict(X_test)
```

```
# Inverse transform the predictions
train_predict = scaler.inverse_transform(np.hstack((train_predict, np.zeros((tra
test_predict = scaler.inverse_transform(np.hstack((test_predict, np.zeros((test_
Y_train_inverse = scaler.inverse_transform(np.hstack((Y_train, np.zeros((Y_train
Y_test_inverse = scaler.inverse_transform(np.hstack((Y_test, np.zeros((Y_test.sh

# Calculate RMSE
train_rmse = np.sqrt(np.mean((train_predict[:, :2] - Y_train_inverse[:, :2])**2,
test_rmse = np.sqrt(np.mean((test_predict[:, :2] - Y_test_inverse[:, :2])**2, ax

print(f'Train RMSE: {train_rmse}')
print(f'Test RMSE: {test_rmse}')
```

```
21107/21107 ━━━━━━━━━━━━━━━━━━━━ 163s 8ms/step
5277/5277 ━━━━━━━━━━━━━━━━━━━━ 42s 8ms/step
Train RMSE: [15.39124441  2.26019048]
Test RMSE: [14.96598038  2.21217256]
```

In [ ]:

In [ ]:

pitfalls:

- interpretability

7. Conclusion and Future research

7.1. Results

- The 'Sales' column contained 172817 rows with 0 sale. Therefore, I created a new data frame in which the 0 sale rows were removed.
- Different models were tried (although we could also try omptimizing the parameters of the models using BO)
- If we include the 0 sales data set we would obtain a far better performance in methods such decision tree, random forest, adaboost, xgboost, and this is because there would be the obviously correct solution of sale being 0 when the store is closed!
- The best accuracy is obtained from the ?? model (after optimization of the prameters/ hyperparameters)
-

7.2. Future Research

- Considering the v-structure, it is better to exploit the dependence of the outputs and use multi-tasking. In this respect considering the:
  - Trend (increasing/ decreasing)
  - Seasonality (Oscillation)

    - Random noise

  we can apply Gaussian processes.
- The other idea would be to extract the causal structure. Having a causal graphical model, we can integrate out the effect of the variables that we don't have. We can also introduce more structure in terms of latent variables.
-

In [ ]: